KELP

Numerical Methods and Implementation

A Reference Manual

Lagrangian particle tracking on the unstructured meshes
of the FESOM family (FESOM2 and FESOM-C)

Engine reference — numerics and source linkage

Abstract

We document KELP, a Lagrangian particle-tracking engine that integrates the trajectories of passive particles through the velocity fields of unstructured-mesh ocean circulation models, with primary support for the FESOM family — FESOM2 and its predecessor FESOM 1.4, on purely triangular meshes, and FESOM-C, on mixed triangular–quadrilateral meshes — and with readers for ocean fields remapped to the HEALPix grid and for structured NEMO grids. This manual is a numerical and implementation reference: for each component of the engine we state the governing equations, give the discrete formulae exactly as implemented, illustrate the underlying geometry with schematics, and cite the location of the corresponding Python source. We treat the engine as the object of study rather than any particular application; the FESOM2 *core2* global mesh recurs only as a concrete example.

The material is organized along the path a single time step travels through the solver: locating a particle on the unstructured mesh, with a proof that the nearest node found is the nearest; interpolating the velocity in space (horizontally by barycentric or least-squares schemes, vertically between depth interfaces or along sigma surfaces, per layer or linearly in depth) and in time (by linear or monotone piecewise-cubic reconstruction); advancing the position with an explicit or adaptive Runge–Kutta scheme; and applying boundary conditions. Stochastic diffusion, the particle lifecycle, the forcing reader, trajectory output, the configuration surface, verification, and computational cost — measured across thread layouts, mesh sizes, particle counts and node counts — are each treated in turn, and a final part describes the agent skills through which experiments are set up, run and analysed. We point to the engine's physics-validation tests — the analytic-flow benchmarks, the vertical streamfunction (continuity) conservation tests, the reversibility round-trip, and the checks against real model output — by name, as evidence that the numerics behave as specified, without reproducing their internals.

Contents

Abstract i

How to use this manual x

Notation and conventions xi

I Foundations 1

1 Introduction 2
1.1 The Lagrangian transport problem . . . 2
1.2 Unstructured-mesh ocean models and the FESOM family . . . 3
1.3 Scope and purpose . . . 4
1.4 How this manual is organized . . . 4
1.5 Conventions . . . 4

2 Solver Architecture and the Per-Step Pipeline 5
2.1 From the continuous equation to a discrete pipeline . . . 5
2.2 Principal data structures . . . 5
2.3 Run assembly . . . 6
2.4 The per-step loop . . . 7
2.5 Forward and backward integration . . . 8
2.6 Fused and unfused execution paths . . . 8

II Mesh and Localization 9

3 The Unstructured Mesh 10
3.1 Nodes, elements, and interfaces . . . 10
3.2 Vertical structure: interfaces and layers . . . 10
3.3 Reading a mesh file . . . 11
3.4 The node-to-element table and its reconstruction . . . 12
3.5 Element bottoms . . . 13
3.6 Other mesh sources . . . 13
3.7 The FESOM 1.4 mesh . . . 15
3.8 Convex quadrilaterals and derived tables . . . 17

4 Particle Localization 18
4.1 Nearest node by spatial hashing . . . 18
4.2 Choosing the cell . . . 19
4.3 Exact nearest nodes . . . 20
4.4 Where localization runs . . . 20

4.5 The containing element . . . 21
4.6 Tracking the element across edges . . . 22

III Interpolation in Space 23

5 Horizontal Velocity Interpolation 24
5.1 Projection to a local plane . . . 24
5.2 Barycentric interpolation . . . 25
5.3 Least-squares interpolation (element layout) . . . 26
5.4 Edge least-squares interpolation . . . 27
5.5 Interpolation on mixed meshes: quadrilateral elements . . . 28
5.6 Common properties . . . 29

6 Vertical Structure and Vertical-Velocity Interpolation 31
6.1 The sign of the vertical velocity . . . 31
6.2 Vertical coordinate: z-levels and sigma . . . 32
6.3 Finding the layer . . . 33
6.4 Horizontal velocity in depth: layers or linear . . . 33
6.5 The vertical velocity profile . . . 38
6.6 Horizontal combination of the vertical velocity . . . 39
6.7 The step on a sigma mesh . . . 40
6.8 Depth clamping at the surface and the floor . . . 41

7 Scalar Field Sampling Along Trajectories 42
7.1 What is sampled, and when . . . 42
7.2 Vertical placement by field shape . . . 43
7.3 Horizontal combination and element type . . . 43
7.4 Independence from the integration direction . . . 43

8 Coordinate Frame and Rotation 45
8.1 The rotated frame . . . 45
8.2 Rotating points and rotating vectors . . . 45
8.3 The user-facing contract . . . 46
8.4 Rotation on HEALPix meshes . . . 46
8.5 Accuracy and when rotation is needed . . . 47

IV Time Integration 48

9 Time-Integration Schemes 49
9.1 The displacement metric . . . 49
9.2 The three schemes . . . 49
9.3 The adaptive scheme . . . 50
9.4 Failure within a step . . . 51
9.5 Order, and what limits it . . . 51

10 Temporal Reconstruction of the Velocity Field 52
10.1 Discrete and interpolated modes . . . 52
10.2 Linear reconstruction . . . 52
10.3 The order cap, and monotone-cubic reconstruction . . . 53
10.4 Read-ahead . . . 54

11 Fused and Unfused Integration Kernels **55**
11.1 Two implementations of one step . . . 55
11.2 Which path applies . . . 55
11.3 Bit-identity . . . 57

V Stochastic Transport and Boundaries **58**

12 Stochastic Diffusion **59**
12.1 The random-walk model . . . 59
12.2 Kicks on the sphere . . . 59
12.3 The three horizontal schemes . . . 60
12.4 Sampling the diffusivity . . . 60
12.5 Vertical diffusion . . . 61
12.6 Reproducibility: counter-based random numbers . . . 61
12.7 Composition with the integrator . . . 61

13 Particle Lifecycle and Boundary Conditions **63**
13.1 The four statuses . . . 63
13.2 Why a particle died . . . 63
13.3 Lateral removal versus vertical clamping . . . 63
13.4 Periodic windows . . . 64
13.5 Beaching and resuspension . . . 64
13.6 Delayed and pulsed release . . . 65

VI Data, Configuration, and Validation **67**

14 Forcing Input and Time Control **68**
14.1 The reader layer . . . 68
14.2 One cadence at a time . . . 69
14.3 The time axis . . . 70
14.4 The integration window . . . 70
14.5 Time-averaged forcing . . . 71
14.6 The velocity handed to a step . . . 72
14.7 Sharing snapshots between processes . . . 72

15 Trajectory Output and Restart **74**
15.1 Two backends . . . 74
15.2 What is written . . . 74
15.3 Checkpoints and restart . . . 77
15.4 Merging a sharded release . . . 77
15.5 Outputs of earlier engines . . . 78

16 Verification and Validation **79**
16.1 Closed-form analytic flows . . . 79
16.2 Conservation and statistics . . . 79
16.3 The depth-linear horizontal velocity . . . 80
16.4 Contracts with model data . . . 81
16.5 Reversibility and determinism . . . 81
16.6 Time-dependent benchmark flows . . . 82

17 Computational Cost and Performance **84**
17.1 The per-step budget . . . 84
17.2 The per-particle kernel cost . . . 85
17.3 The nearest-node anchor . . . 86
17.4 Threads and processes on one node . . . 87
17.5 Mesh size and particle count . . . 88
17.6 Many nodes . . . 88
17.7 Memory . . . 88
17.8 Input and output . . . 89
17.9 Sharding a release . . . 90
17.10 Measuring the cost . . . 90

VII Working with KELP **91**

18 Agent Skills: the Operating Layer **92**
18.1 What a skill is . . . 92
18.2 The seven skills . . . 92
18.3 How the skill text is written . . . 93
18.4 The check scripts . . . 95
18.5 Skills and the documentation . . . 95
18.6 Maintaining a skill . . . 95

A Configuration Key Reference **96**

B Notation and Symbols **103**

C Source-Code Map **105**

D Glossary **108**

List of Figures

2.1 The per-step pipeline executed by `run_single_realization`. The main loop (left) activates any particles whose release time has arrived, assembles the velocity for the step, integrates, folds particles back into periodic windows, applies the optional beaching hook, advances the clock, and records and checkpoints at the configured cadence. The side panel lists the operations performed for each particle inside the integration step; stages marked *(3-D)* are present only in three-dimensional runs. 7

3.1 Anatomy of the horizontal mesh. Triangle vertices are *nodes* (filled circles), where node-layout velocities and the scalar fields live; triangle centroids are *elements* (the orange square), where element-layout velocities live. The six shaded triangles sharing the central node n form its one-ring, the adjacency the localization search exploits (Chapter 4). 11

3.2 Anatomy of the vertical column. The K interfaces (grey lines, open circles) bound $K-1$ mid-layers (orange crosses). The vertical velocity W is stored at the interfaces; the three-dimensional horizontal velocity and the scalar fields are stored at the mid-layers. The local sea floor is given per node by `zbar_n_bottom`; layers below it do not exist for that node and a particle cannot descend past it (Chapter 6). 11

3.3 Three mesh sources on one `Mesh`. (a) The HEALPix dual mesh joins the four pixel centres around a pixel corner into a quadrilateral (three centres give a triangle). (b) The pixel mesh makes each wet pixel an element, with nodes at its corners and the velocity at its centre. (c) A structured grid keeps the quadrilaterals whose four T-points are wet; one land point removes the quadrilateral in the corner. 15

3.4 Triangle recovery from FESOM 1.4 cell polygons. The median-dual cell of each node (shaded, one colour per node) is bounded by the dashed segments joining the edge midpoints (dots) to the triangle centroid G (square). Within one triangle each node's cell contributes the quadrilateral node, midpoint, centroid, midpoint, so G is the only point that the polygons of exactly three nodes share: every cluster of polygon vertices owned by three nodes is a triangle of the mesh. . . . 16

4.1 Nearest-node search by spatial hashing. Nodes are bucketed into uniform cells when the mesh is loaded. A query (the drifter) is first matched against the nodes of its own cell and the eight neighbouring cells (shaded); Section 4.3 gives the test that decides whether this window is enough. 19

4.2 The exactness certificate. The window returns a node at angular distance θ; a closer node can only lie in the cap of radius θ around the query. Here the cap crosses the window's eastern edge, so the window cannot prove its answer; it widens by one column, finds the true nearest node, and the smaller cap of the new answer fits inside the searched interval. 21

4.3 Containing-element search. The engine scans the one-ring of the nearest node and tests each element for containment. The drifter lies in the shaded triangle; the dotted segments split it into three sub-triangles whose signed areas, normalized by the whole, are the barycentric coordinates that decide containment. 21

5.1 Barycentric interpolation of node-layout velocity. The three triangle vertices n_1, n_2, n_3 carry the node velocities (u_i, v_i). The drifter P splits the triangle into three sub-triangles; the weight w_i is the area of the sub-triangle *opposite* vertex i divided by the total area, so the nearer the particle lies to a vertex, the larger that vertex's weight. The interpolated velocity is $w_1(u_1, v_1) + w_2(u_2, v_2) + w_3(u_3, v_3)$. 26

5.2 Least-squares interpolation of element-layout velocity. The data are the velocities at the centres (orange squares) of the one-ring of elements about the nearest node. A plane $f = a_0 + a_1x + a_2y$ is fitted to them in a frame centred on the drifter, so the interpolated velocity is the fitted constant a_0. The scheme is exact for linearly varying fields over this stencil. 27

5.3 The edge least-squares stencil. The drifter lies in the shaded triangle; the plane is fitted to its centre **c** and the centres of the elements across its two interior edges. Its third edge is on the coast and contributes a ghost at the mirror image **g** of **c**, carrying the mirrored velocity: the component along the wall is kept and the component into the wall reversed (free slip), or the whole velocity reversed (no slip). 28

5.4 Wachspress interpolation on a quadrilateral element. The drifter p splits the element into four fan triangles of doubled area $A_1, \ldots, A_4$. The weight at vertex k is proportional to $C_kA_{k+1}A_{k+2}$, where C_k is the doubled area of the corner triangle (u_{k-1}, u_k, u_{k+1}): the two fan triangles *not* adjacent to vertex k grow as p approaches it. 29

6.1 The depth-linear rule at one particle (star, depth z) whose containing triangle has the vertices A, B, C, drawn as three columns side by side. Diamonds are the data points — here layer centres between the levels (grey lines) — down to each column's last usable point p_{ℓ_c}; green segments are the beds. Vertex A is wet at z and blends its two bracketing points. The bed of B lies above z, so B is dry there: a wall, left out of the sum. Vertex C is wet, but its bottom layer is cut by its bed (a partial layer, open diamond, never read), so it contributes the value of its last usable point. The node-layout result is $U = s_{\mathrm{fb}}(w_A^+u_A + w_C^+u_C)/(w_A^+ + w_C^+)$ (Equation (6.5)); for element layout the columns are the elements of the stencil, under the same rule. 35

6.2 Vertical interpolation of the vertical velocity, identical in form for the z-level and sigma families. The interface values W_k, W_{k+1} (open circles; downward arrows mark the positive-down sign) bracket the drifter at depth z. The fraction $\alpha = (z - z_k)/(z_{k+1} - z_k)$ is formed in physical depth, and the profile value is $(1-\alpha)W_k + \alpha W_{k+1}$. No extrapolation is performed beyond the ends of the column. 39

6.3 Sigma-following evaluation of the vertical velocity. The particle sits at the fraction $\xi = z/H_p$ of its column; the neighbouring column is evaluated at ξH_j on the same sigma surface (filled node), not at the particle's depth (open node), which on a sloping floor lies on a different sigma level. 40

8.1 Rotation of a horizontal velocity into the rotated frame. The velocity (u, v) is a tangent vector; its components are re-expressed in the rotated local east–north basis (dashed), turned through the local angle θ, by Equation (8.2). The map preserves the speed. Node coordinates and element centres, being points rather than vectors, are instead mapped directly by the rotation. 46

9.1 The four stages of RK4. The slope $\mathbf{d}_1$ is sampled at the start (t_n); $\mathbf{d}_2$ and $\mathbf{d}_3$ at provisional midpoints (time $t_n + \frac{1}{2}\Delta t$); $\mathbf{d}_4$ at a provisional endpoint ($t_n + \Delta t$). The step is their weighted average Equation (9.5) (dashed), a fourth-order approximation to the true path (blue). Each stage requires the velocity at a different position *and* time, which is why the temporal reconstruction of Chapter 10 matters. 50

10.1 Temporal reconstruction within the interval $[t_n, t_{n+1}]$. The linear reconstruction (dashed) joins the two bracketing snapshots; the monotone-cubic PCHIP reconstruction (solid) uses four consecutive snapshots and follows the curvature of the sampled series. At a sub-step time t the two differ, and only the cubic lets a fourth-order spatial scheme realize its order. PCHIP is monotone, so it introduces no overshoot between samples. 54

11.1 The two implementations of a single step. The fused kernel (left) performs all per-particle operations inside one compiled loop; the unfused path (right) runs the same operations as separate parallel passes over all particles. Where both exist they produce bit-identical trajectories. 56

12.1 One diffusive step. The deterministic advective displacement $u\,\Delta t$ (blue) carries the particle from x_n; a random kick of typical size $\sqrt{2K_h|\Delta t|}$ (dashed circle) is then added, scattering the realized endpoint (stars). The advective term reverses under backward integration; the kick, scaling with $\sqrt{|\Delta t|}$, spreads the ensemble either way. 60

13.1 The particle status state machine. A particle is released from `NOT_RELEASED` into `ALIVE` at its scheduled time; it is removed to `OUTSIDE` if it leaves the domain (its step undone). With beaching enabled, `ALIVE` and `BEACHED` interconvert, and a particle removed in the same step with a shallow or coastal anchor is reclassified as `BEACHED` (dashed). Reaching the surface or floor does not change the status: the depth is clamped and the particle remains `ALIVE`; nor does a fold back into a periodic window (the self-loop). 64

14.1 An example of the cadences of one FESOM2 output configuration: the variables share one time-units string but are sampled at different rates. A run must select variables of a single cadence; Lagrangian advection uses the daily node-based fields (top). Mixing cadences — for example the daily `unod`/`vnod` with the monthly element `u`/`v` — cannot be aligned on one time axis and is rejected when the forcing is opened (a finer attached file that contains every velocity stamp is thinned with a warning instead). 70

14.2 The node-shared snapshot cache. The process that wins the lock reads a snapshot from the forcing files, writes it under a temporary name and renames it into place; the other processes map the published arrays read-only. The namespace is derived from the configuration, the input files, the engine version and the job, so no other run reuses it. 73

15.1 The output data model: two-dimensional arrays indexed by particle (trajectory) and by record (time), one such array per stored variable, with the time and the trajectory identifier as one-dimensional coordinate variables and the release position and time as one-dimensional arrays over the particles. The position and status are always present, and so are the velocities of a tracking run (NaN where none was evaluated); failure reasons and sampled scalars are added when available. In a Zarr store each array is split into chunks of Equation (15.1). . . 76

15.2 A resumable chain. Each job resumes from the newest numbered restart, writes its own segment and drops restarts as it goes; the segments are concatenated at the end. Because every cadence and every random draw is keyed to the global step, the result equals one uninterrupted run. 78

16.1 A controlled reversibility check on the *core2* mesh, measured in June 2026: one hundred particles are integrated forward for one year and then backward for one year with the same scheme (RK2, linear reconstruction in time), and the return error between the start and end positions is mapped. The median return error is 0.16 km (ninetieth percentile 1.17 km), far below the local grid scale of tens of kilometres (a three-dimensional run at 5 m depth in the Weddell Sea, engine of June 2026; the vertical motion was negligible). Larger separations over long windows reflect Lagrangian chaos and boundary clamping rather than a defect of the scheme. 82

17.1 The per-step budget of Equation (17.1) and the lever that acts on each term. The kernel term splits into the nearest-node anchor (c_{anchor}, Section 17.3) and the stencil work, repeated once per velocity evaluation of the scheme (n_{eval}). . . 85
17.2 Particle count: steady tracking step against the number of particles, zoom-10 HEALPix mesh (8.9 million wet surface nodes), one 128-thread process, fused three-dimensional midpoint kernel. Dashed: the fitted power law with slope 0.88. 89
17.3 Scaling over nodes on the zoom-11 HEALPix mesh, one 128-thread process per node, fused three-dimensional midpoint kernel. Left: weak scaling (dashed, the one-node step). Right: strong scaling of 24.1 million particles (dashed, ideal). The strong curve is not smooth because the cost of a process depends on which particles its contiguous share holds. 89

18.1 How requests reach the skills. A request is matched against the skills' descriptions; the selected `SKILL.md` then points to the neighbouring skill it builds on (solid) and to the documentation it defers to (dashed). Reference files under each skill's `references/` are opened only from the step that names them. 94

How to use this manual

This manual is designed for selective, non-linear reading: a reader with a specific question should be able to turn directly to the relevant chapter without reading the preceding material. To that end we adopt a consistent internal structure and several navigation aids.

Chapter structure. Each chapter opens with a short statement of what the component does and where it sits in the per-step pipeline (Chapter 2). It then develops the governing equations and the discrete formulae as implemented, accompanied by a schematic that shows which mesh entities (nodes, element centres, vertical interfaces) supply values and how they are combined. Each substantive claim about the implementation is followed by a **Source** callout naming the file and the function or line range that realizes it, so that the manual and the code can be read side by side.

Finding a topic. Three routes are provided. The table of contents lists every section. Chapter C is a source-code map: it lists, per subsystem, the authoritative source symbols and the manual section that explains them, so a reader starting from the code can find the corresponding text. The index collects symbols, configuration keys, and function names.

Conventions and symbols. The sign and unit conventions used throughout (positive-downward depth, the integration-direction factor, the spherical metric, the distinction between fields stored at nodes, at element centres, and at vertical interfaces) are stated once in the front matter section that follows, and the complete symbol list is given in Chapter B.

Worked configuration. Where a concrete size clarifies a choice, we refer to the FESOM2 *core2* global mesh; the ready configurations shipped with the engine cover two-dimensional global runs, a three-dimensional regional run, a structured-grid run, HEALPix runs and a three-dimensional run on FESOM 1.4 output. The examples illustrate the engine; they are not the subject of this document.

Setting up a run. A reader who wants to run an experiment rather than study the numerics can start from Chapter 18, which describes how a described experiment becomes seeds, a configuration and a run command, and how the output is turned into statistics and figures; Chapter A lists every configuration key.

Notation and conventions

We collect here the conventions that recur throughout the manual. The complete symbol list is given in Chapter B.

Position and velocity. A particle position is the triple $\mathbf{x} = (\lambda, \phi, z)$ of longitude, latitude (both in degrees), and depth. The model velocity is $\mathbf{u} = (u, v, W)$, with horizontal components u, v in metres per second and vertical component W in metres per second.

Depth and vertical velocity are positive downward. Depth z increases from 0 at the surface to the sea floor, and the vertical velocity W is positive for downwelling. Consequently the vertical position update $z \mathrel{+}= W\,\Delta t$ deepens a particle in downwelling, so that sinking is followed with the correct sign.

Integration direction. The factor $s_{\mathrm{fb}} \in \{+1, -1\}$ (the configuration key `time_dir`) selects forward $(+1)$ or backward (-1) integration. The time step Δt is always a positive quantity; the direction enters by multiplying the interpolated velocity, so that backward integration is a sign reversal of the sampled velocity rather than a separate code path (Chapters 2 and 9).

The spherical metric. Velocities in metres per second are converted to angular increments in degrees by the local metric. With $M_\phi = 111\,120\,\mathrm{m}\,^{\circ -1}$ the (constant) metres per degree of latitude and $z_0 = \cos(\phi)\, M_\phi$ the metres per degree of longitude, the horizontal update reads $\lambda \mathrel{+}= \Delta t\, u / z_0$ and $\phi \mathrel{+}= \Delta t\, v / M_\phi$. The value of M_ϕ follows the nautical-mile convention (1852×60), the value used by Parcels; the original Fortran tracker used $111\,300\,\mathrm{m}\,^{\circ -1}$, so the two differ by 0.16 % in every displacement (Chapter 16). Diffusive kicks are the exception: they move along great circles (Section 12.2).

The FESOM family. KELP targets the unstructured-mesh ocean models of the FESOM family: *FESOM2*, whose meshes are purely triangular and whose vertical coordinate is a set of fixed depth levels (z-levels); its finite-element predecessor *FESOM 1.4*, on the same kind of mesh and levels, whose fields live on the levels at the nodes; and *FESOM-C*, whose meshes are *mixed* — each element being a triangle or a quadrilateral — and whose vertical coordinate is terrain-following (sigma). The engine additionally reads ocean fields remapped to the HEALPix grid and structured NEMO grids, which it represents as meshes of quadrilaterals (with a few triangles). Accordingly, throughout this manual an *element* is a triangle or a quadrilateral, and a "FESOM mesh" means a mesh of any of these models; statements that hold for only one of them name it explicitly. Sizes quoted as examples are those of the FESOM2 *core2* (triangular, z-level) mesh.

Where fields live. On a FESOM mesh, scalar fields and node-layout velocities are defined at the element *vertices* (nodes); element-layout velocities are defined at element *centroids* (elements); and the vertical velocity and other interface quantities are defined at the *vertical interfaces* that bound the depth layers, the three-dimensional horizontal velocity at the layer

centres (FESOM 1.4: on the levels, with the vertical velocity). The distinction governs which interpolation stencil applies and is made explicit in every schematic.

Typographic conventions. Source files, functions, and configuration keys are set in a monospaced font: a configuration key such as `uv_interp`, a function such as `run_tracking`, a file such as `numerics/interpolate.py`. A **Source** callout ties a passage of prose to the implementation that realizes it.

Part I

Foundations

Chapter 1

Introduction

KELP is a Lagrangian particle-tracking engine for the velocity fields produced by unstructured-mesh ocean circulation models, with first-class support for the FESOM family: the Finite-volumE Sea ice–Ocean Model in its second version (FESOM2; Danilov et al., 2017), on purely triangular meshes, its finite-element predecessor FESOM 1.4 (Wang et al., 2014), and its coastal relative FESOM-C, on mixed triangular–quadrilateral meshes. It also reads ocean fields remapped to the HEALPix grid and, in a first two-dimensional version, structured NEMO grids. Given a time sequence of stored model velocity snapshots and a set of release positions, it integrates the trajectories of passive particles either forward or backward in time, optionally in three dimensions and optionally subject to a stochastic eddy-diffusion term. This manual documents the numerical methods that the engine implements and links each method to the Python source that realizes it.

1.1 The Lagrangian transport problem

In the Lagrangian description of transport one follows individual fluid parcels along their paths, in contrast to the Eulerian description, which records the flow at fixed locations. A passive particle carried by a velocity field $\mathbf{u}(\mathbf{x}, t)$ obeys the ordinary differential equation

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{u}(\mathbf{x}(t),\, t), \qquad \mathbf{x}(t_0) = \mathbf{x}_0, \tag{1.1}$$

where the position $\mathbf{x} = (\lambda, \phi, z)$ collects longitude, latitude and depth, and the velocity $\mathbf{u} = (u, v, W)$ collects the two horizontal components and the vertical component. We adopt throughout the convention that depth z and vertical velocity W are positive downward, so that a positive W deepens a particle. The particle is passive: it does not modify the flow, and Equation (1.1) is integrated as a one-way coupling from the stored model fields to the trajectory.

Solving Equation (1.1) numerically poses two distinct subproblems. First, the right-hand side must be evaluated at the particle position $\mathbf{x}(t)$, which in general falls between the mesh vertices at which the model stores its fields and between the discrete times at which snapshots are available; this requires interpolation in space and in time. Second, the differential equation must be advanced from one time level to the next with a controlled truncation error; this requires a time-integration scheme. The spatial-interpolation subproblem is the subject of Chapters 5 to 7, the temporal-reconstruction subproblem of Chapter 10, and the time-integration subproblem of Chapter 9. When the stochastic term is included, a backward run integrates the adjoint process — the advection reversed, the diffusion not — so that backward integration remains a well-posed random walk (Chapter 12). The remaining chapters treat the supporting machinery: locating a particle on the mesh (Chapter 4), the stochastic-diffusion extension of Equation (1.1) (Chapter 12), the treatment of domain boundaries and the sea floor (Chapter 13), and the

reading, writing, configuration and verification of the whole (Chapters 14 to 17); a final part describes the agent skills through which runs are set up and analysed (Chapter 18).

For a broad survey of Lagrangian ocean analysis — its applications, its numerical pitfalls, and the conventions adopted in the field — we refer the reader to van Sebille et al. (2018); the present manual is concerned specifically with how KELP solves Equation (1.1) on an unstructured mesh.

1.2 Unstructured-mesh ocean models and the FESOM family

FESOM2 discretizes the ocean horizontally on an unstructured triangular mesh and vertically on a set of fixed depth levels (Danilov et al., 2017); its coastal relative FESOM-C uses a *mixed* mesh, in which an element may be a triangle or a quadrilateral, together with a terrain-following vertical coordinate. Its predecessor FESOM 1.4 (Wang et al., 2014) uses the same kind of triangular surface mesh and fixed levels, but it is a finite-element model of the P1 type: u, v, W, temperature and salinity are piecewise linear on the tetrahedra of a three-dimensional mesh and live together *on the levels at the nodes*, rather than at layer centres and element centroids. KELP supports all three, and represents the other sources on the same kind of mesh: a HEALPix product becomes either a mesh whose nodes are the pixel centres (quadrilaterals, with triangles at the eight points where only three pixels meet) or a mesh whose elements are the pixels themselves, and a structured NEMO grid becomes a mesh of quadrilaterals (Chapter 3). We therefore treat an element as a triangle or a quadrilateral throughout. The unstructured horizontal mesh permits the spatial resolution to vary smoothly across the domain — fine where the dynamics demand it, coarse elsewhere — but it removes the regular row-and-column addressing of a latitude–longitude grid. As a result, identifying the element that contains a given particle position is itself a search problem rather than an arithmetic indexing operation; we treat it in Chapter 4.

A second feature of the unstructured mesh bears directly on interpolation: the model stores different quantities at different geometric locations. The two horizontal velocity components may be defined either at the element vertices ("node-layout" fields, such as the FESOM2 variables `unod` and `vnod`, the FESOM 1.4 velocities, the HEALPix pixel-centre mesh and the NEMO grids) or at the element centroids ("element-layout" fields, such as the FESOM2 variables `u` and `v` and the FESOM-C velocities). The vertical velocity and other interface quantities are defined at the vertical interfaces that bound the depth layers, while scalar fields such as temperature and salinity are defined at the nodes. In the vertical, most producers store the three-dimensional horizontal velocity once per layer, which the engine reads either as constant within the layer or linearly in depth between the layer centres; the FESOM 1.4 velocities, stored on the levels, are always read linearly in depth (Section 6.4). Each storage location calls for a different interpolation stencil, a distinction that recurs throughout Chapters 5 to 7 and that we render explicitly in every schematic.

The engine ships with ready configurations that make these choices concrete: global two-dimensional runs on node velocities, a regional three-dimensional run with vertical velocity and sampled temperature and salinity, a structured-grid run, HEALPix runs and a three-dimensional run on level-wise FESOM 1.4 output, its mesh recovered from the cell polygons of the data files. Where this manual quotes a size it is usually that of the FESOM2 *core2* global mesh (126 858 nodes, 48 vertical interfaces reaching to about 6250 m). The object of this document is the engine, not any experiment run with it.

1.3 Scope and purpose

This manual is a numerical and implementation reference. For each component we state the governing equations, give the discrete formulae exactly as implemented (including the sign conventions, the clamping rules, and the degenerate-case fallbacks that a working code must contain), illustrate the geometry of the computation with a schematic, and cite the source location. We do not report scientific results, and we do not attempt to assess the physical realism of any particular simulation; our goal is to characterize what the engine computes and how, not to interpret the trajectories it produces.

The engine is accompanied by an extensive automated test suite of about 4700 tests. The large majority of these guard low-level invariants — array shapes, configuration parsing, byte-for-byte reproducibility — and are not individually informative. A small number, by contrast, verify the numerical physics against independent references: convergence under step refinement in time-dependent eddy flows, conservation of a vertical streamfunction under continuity-consistent flow, reversibility of the integrator under time reversal (to within a small tolerance), agreement with closed-form analytic flows, the convergence order of the depth-linear horizontal velocity, and the reading of published FESOM 1.4 output. We discuss these by name in Chapter 16 as evidence that the numerics behave as specified, without reproducing their internals.

1.4 How this manual is organized

The chapters follow the order in which the engine processes a single integration step, so that reading the manual front to back traces one time step through the solver. Chapter 2 gives the overall architecture — the data structures and the per-step pipeline — and serves as the map to which the later chapters refer. The pairs and groups that follow treat, in turn, the mesh and particle localization; spatial interpolation of velocity and of scalars; time integration and temporal reconstruction; stochastic diffusion and the particle lifecycle; and finally the data, configuration, validation and performance layers. A last part, “Working with KELP”, describes the agent skills that turn a described experiment into a run and its output into figures and statistics (Chapter 18). The appendices provide an exhaustive configuration-key reference, the symbol list, the source-code map, and a glossary. Readers seeking a particular topic should consult the front-matter guide “How to use this manual” or the source-code map in Chapter C.

1.5 Conventions

We summarize the principal conventions here and state them in full in the front-matter section “Notation and conventions”. Depth and vertical velocity are positive downward. Angles are in degrees and velocities in SI units unless stated otherwise. The integration direction is carried by the factor $s_{\mathrm{fb}} = \pm 1$, with the time step Δt kept positive. The spherical metric converts advective velocities to angular increments through $z_0 = \cos(\phi)\, M_\phi$ and $M_\phi = 111\,120\,\mathrm{m}\,^{\circ -1}$; diffusive kicks move along great circles instead (Section 12.2). Model files that store the vertical velocity positive upward are converted when they are read (Section 6.1). Fields are identified throughout as living at nodes, at element centres, or at vertical interfaces. The complete symbol list is given in Chapter B.

Chapter 2

Solver Architecture and the Per-Step Pipeline

This chapter provides the map to which the remainder of the manual refers. We describe how the continuous problem of Equation (1.1) is reduced to a discrete pipeline, the principal data structures that the pipeline operates on, the one-time assembly that precedes the time loop, and the ordered sequence of operations performed at each step. Later chapters expand individual stages of this pipeline; here we are concerned only with their arrangement.

2.1 From the continuous equation to a discrete pipeline

The engine advances Equation (1.1) by an explicit time-stepping scheme: time is divided into steps of length Δt, and at each step the velocity is sampled along the path and used to displace the particle. Evaluating the velocity at the particle position decomposes into a fixed sequence of operations that is the same at every step and for every scheme. The particle must first be *located* on the unstructured mesh — the nearest node and the containing element must be identified — because the interpolation stencils are expressed in terms of mesh entities. The velocity is then *interpolated in space*, from the surrounding nodes or element centres to the particle position, and *reconstructed in time*, from the bracketing stored snapshots to the substep time required by the integrator. The integrator *advances* the position, and finally the *boundary conditions* are applied: a particle that has left the domain is flagged and its step undone, while a particle that has reached the surface or the sea floor is clamped. The result is recorded at a prescribed cadence.

This decomposition — locate, interpolate in space, reconstruct in time, integrate, apply boundary conditions, record — is realized directly in the source and is depicted in Figure 2.1. The chapters of Chapters 3 to 13 correspond one-to-one to its stages.

2.2 Principal data structures

Three objects carry the state of a run. The `Mesh` holds the static geometry: the node coordinates, the element connectivity, the per-node sea-floor depth, the vertical interfaces, the frame and radius, and the search structures and adjacency tables built once and cached on it. The `VelocityField` holds the model fields for a single time level, already shaped according to where they live (at nodes, at element centres, or at interfaces) and tagged with their layout. The `ParticleState` holds the evolving per-particle quantities: position, status, the cached nearest node, the most recently evaluated velocity, and — when the run uses them — the failure reason, the release step and the carried element. Table 2.1 summarizes their principal members; the chapters indicated develop each in detail.

Table 2.1: The three principal data structures and their principal members. Shapes are given for N nodes, E elements, K vertical interfaces (hence $K-1$ layers), and P particles.

Structure	Member	Meaning
`Mesh`	`nodes_lon/lat`	node coordinates, degrees (N); see Chapter 3
	`elem_nodes`	element connectivity $(E,3)$ or $(E,4)$, -1-padded
	`elem_center_lon/lat`	element centroids (E)
	`depth`	per-node sea-floor depth, positive down (N)
	`sigma`	vertical interfaces (K): depths on z-level meshes, σ values on sigma meshes
	`vertical_coord`	`zlevel` or `sigma`
	`node_to_elems`	node→element adjacency; see Chapter 4
	`rotation_angles`	frame of the coordinates, or `None`; see Chapter 8
	(cached)	grid hash and node table, element neighbours, areas, coastal flags
`VelocityField`	`u`, `v`	horizontal velocity; node $(N)/(K-1,N)$ or element $(E)/(K-1,E)$
	`w`	vertical velocity at interfaces (K,N), or `None` (2-D)
	`eta`	sea-surface height (N)
	`uv_layout`	`"node"` or `"element"`; selects the stencil
	`scalars`	sampled fields (e.g. temperature, salinity)
	`kh_*`, `az`	horizontal and vertical diffusivities, when used
`ParticleState`	`lon`, `lat`, `z`	position (P)
	`status`	lifecycle state; see Chapter 13
	`nearest_node`	cached localization result (P)
	`u`, `v`, `w`	last evaluated velocity (P)
	`failure_reason`, `release_step`, `elem`	optional: cause of death, release step, carried element

Source. The data structures are defined in `core/mesh.py` (`Mesh`), `core/io.py` (`VelocityField`) and `core/particles.py` (`ParticleState`).

2.3 Run assembly

Before the time loop begins, a single assembly phase prepares the run, in this order. The reader appropriate to the mesh type is selected and the mesh is loaded (Chapter 3); the exact-anchor guard is set and the grid-hash cell is chosen — pinned, measured or the reader's (Section 4.2) — and in three dimensions the vertical interfaces are trimmed to `max_levels`. The forcing is opened lazily, so that snapshots are read from disk only as they are needed, and the release positions are read. The time axis is resolved: the interval between stored snapshots is detected, the step is set, and for time-averaged forcing each timestamp is shifted from the end of its averaging window to its midpoint (Chapter 14). Only then is the release schedule built, since release steps are offsets divided by the step; it pre-allocates the full particle array and freezes not-yet-released particles (Chapter 13). The seeds are rotated into the computational frame (Chapter 8). A set of probes then checks, before any output file exists, the combinations that would fail later — the sampled fields, the velocity layout against the interpolation scheme, the diffusion, beaching, periodic and adaptive-scheme settings — so that a misconfigured run fails in seconds; a few remain for the first step (spatially varying diffusion in three dimensions, the

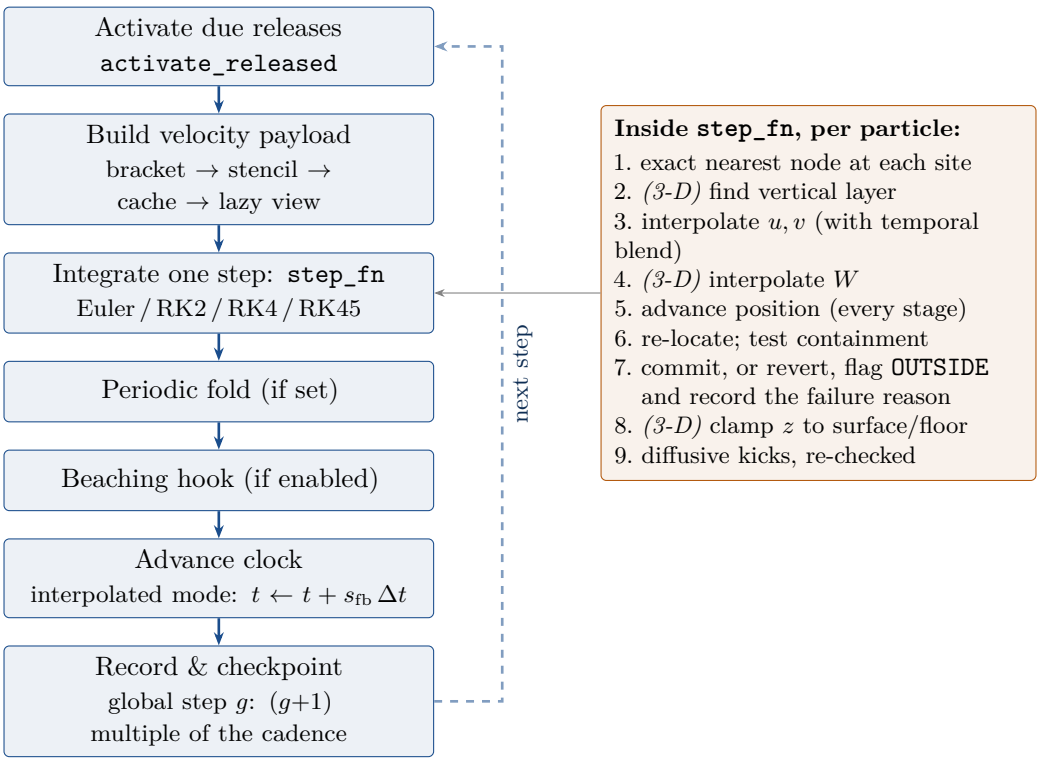


Figure 2.1: The per-step pipeline executed by `run_single_realization`. The main loop (left) activates any particles whose release time has arrived, assembles the velocity for the step, integrates, folds particles back into periodic windows, applies the optional beaching hook, advances the clock, and records and checkpoints at the configured cadence. The side panel lists the operations performed for each particle inside the integration step; stages marked *(3-D)* are present only in three-dimensional runs.

adaptive scheme on a pixel mesh with the exact guard off). For each realization the output writer is opened with its provenance attributes (Chapter 15), the shared snapshot cache is attached when configured (Section 14.7), and the time loop runs.

Source. The assembly order is implemented in `run_tracking` (`tracking.py`); the time loop in `run_single_realization` (`tracking.py`).

2.4 The per-step loop

At each step the engine performs the sequence shown in Figure 2.1. It first activates any particles whose scheduled release time has arrived, so that they are advected during the same step (Chapter 13). It then assembles the velocity payload for the step: the stored snapshots that bracket the current time are identified, the reconstruction stencil is resolved (two snapshots for linear reconstruction, four for monotone piecewise-cubic), the snapshots are taken from the cache or read, and the step receives either a snapshot or a lazy view that blends a field only when it is read, together with the target snapshot for the Runge–Kutta stages (Sections 10.2 and 14.6). On a sigma mesh the surface elevation at the start and the end of the step is supplied as well (Section 6.7). The integration step is invoked once; the operations it performs for each particle are listed in the side panel of Figure 2.1 and are the subject of Chapters 4 to 9. Particles that left a periodic window are folded back, and if beaching is enabled a post-step hook evaluates the stranding and resuspension criteria (Chapter 13). In the interpolated mode the clock advances by $s_{\mathrm{fb}}\,\Delta t$; in the discrete mode the snapshot index advances by s_{fb}, one record forward or backward. The state is recorded, and checkpointed, after every step whose global index g satisfies that $g+1$ is a multiple of `output_freq`, respectively `checkpoint_freq`, so that a

resumed run records the same steps as an uninterrupted one (Chapter 15); one more checkpoint follows the last executed step. A record is labelled with the time at the end of its step, samples the requested scalars at that time, un-rotates positions and velocities to geographic coordinates, writes the rows of not-yet-released particles as missing values, and writes the velocity as missing wherever the particle was not alive through the whole step.

The boundary conditions are applied inside the step, immediately after the position is advanced. The containing-element test serves as the domain check: a particle whose new position is not contained by any element — whether it has crossed an open boundary or left a coastline — has its step undone and its status set to `OUTSIDE`. Vertical limits, by contrast, are enforced by clamping rather than removal: a particle that would cross the surface or the local sea floor is held at that limit (Chapters 6 and 13). With the option `-v` the loop prints, per step, the time spent reading (`io=`), stepping (`step=`), beaching (`beach=`) and recording (`rec=`) (Section 17.10).

Source. The step sequence is the loop body of `run_single_realization` (`tracking.py`); the per-particle operations are implemented in the integration kernels of `numerics/integrators.py`.

2.5 Forward and backward integration

The engine integrates equally in either time direction, and it does so without a separate backward code path. The direction factor $s_{\mathrm{fb}} \in \{+1, -1\}$ is folded into the interpolated velocity: every interpolation routine returns $s_{\mathrm{fb}}\,u$, $s_{\mathrm{fb}}\,v$ and $s_{\mathrm{fb}}\,W$ rather than the raw model velocity, while the time step Δt remains positive. Backward integration is therefore a sign reversal of the sampled velocity, and the same integrator advances the position in both cases. The step counter increments in both directions, and the simulation clock advances by $s_{\mathrm{fb}}\,\Delta t$. With stochastic diffusion the backward run is the adjoint process: the advection is reversed, the diffusive drift and noise are not (Section 12.1). This symmetry is what the reversibility tests of Chapter 16 exploit: integrating forward and then backward over the same interval, with the same scheme, returns a particle to its starting point to within a small tolerance.

2.6 Fused and unfused execution paths

The per-particle operations of Figure 2.1 are realized by two interchangeable implementations. A set of *fused* kernels, compiled just-in-time, performs the whole step — localization, interpolation, integration and the boundary check — within a single compiled loop over the particles, for the common configurations to which they apply: two-dimensional least-squares runs, time-dependent two-dimensional barycentric runs on node velocities (RK2 and RK4, linear interpolation in time, no element tracking), three-dimensional Euler with least-squares velocities on z-level meshes (vertical combine `nearest` or `leastsq`), and three-dimensional RK2 with node-layout barycentric velocities when the vertical velocity is combined by `nearest` or `barycentric`. An *unfused* path performs the same operations as separate parallel passes over all particles and covers every other configuration. Where both exist they produce bit-for-bit identical trajectories, statuses and failure reasons under the default exact anchor guard (`mesh.exact_nearest`); the details are in Chapter 11.

Part II

Mesh and Localization

Chapter 3

The Unstructured Mesh

This chapter describes the static geometry on which the engine operates: the unstructured mesh, the quantities a model stores on it, and the conventions under which each source — FESOM2, FESOM 1.4, FESOM-C, HEALPix products and structured NEMO grids — is read into the `Mesh` structure of Section 2.2. We use the triangular FESOM2 *core2* mesh as the worked example and describe the other sources where they differ. The localization search of Chapter 4 and every interpolation stencil of Chapters 5 to 7 are expressed in terms of the entities introduced here, so we also fix the vocabulary — node, element, interface, layer — used throughout the manual.

3.1 Nodes, elements, and interfaces

The horizontal mesh is a tessellation of the ocean surface: a set of N *nodes* joined into E *elements*. In FESOM2 the elements are triangles; in FESOM-C they are mixed, each element being a triangle or a quadrilateral; structured grids and the HEALPix pixel mesh produce quadrilaterals only, and the HEALPix dual mesh quadrilaterals with a few triangles (Section 3.6). A triangle in a mesh with room for four vertices is stored with its fourth slot set to -1. For the FESOM2 *core2* configuration used as the running example the elements are triangular, with $N = 126\,858$ nodes and $E = 244\,659$ elements. Each element stores the indices of its vertices — three for a triangle, four for a quadrilateral — and the engine precomputes, for each node, the list of elements that contain it (its *one-ring*), which the localization search relies upon (Chapter 4).

The model does not store all fields at the same location, and the distinction is fundamental to interpolation (Figure 3.1). *Node-layout* fields are defined at the vertices: the FESOM2 horizontal velocity components `unod` and `vnod`, the sea-surface height, and the sampled scalars (temperature, salinity) are all node quantities. *Element-layout* fields are defined at the element centroids: the alternative monthly-mean FESOM2 velocities `u` and `v` are element quantities. A node-layout field carries one value per vertex and is interpolated by the barycentric scheme of Chapter 5; an element-layout field carries one value per element and is interpolated by either least-squares scheme or, through area-weighted vertex values, the barycentric scheme of the same chapter. The `VelocityField` records which layout applies in its `uv_layout` attribute, and the engine selects the matching stencil accordingly.

3.2 Vertical structure: interfaces and layers

In the vertical, the water column is divided by K *interfaces* into $K-1$ *layers* (Figure 3.2). The interfaces are the level boundaries; the layers are the volumes between them. The distinction matters because, again, different fields live in different places: the vertical velocity W is defined

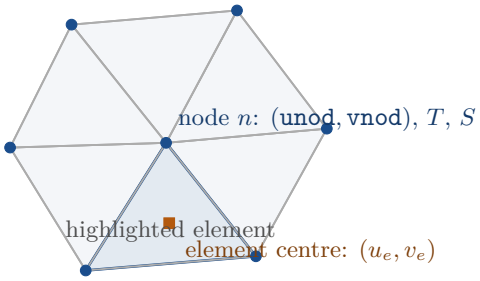


Figure 3.1: Anatomy of the horizontal mesh. Triangle vertices are *nodes* (filled circles), where node-layout velocities and the scalar fields live; triangle centroids are *elements* (the orange square), where element-layout velocities live. The six shaded triangles sharing the central node n form its one-ring, the adjacency the localization search exploits (Chapter 4).

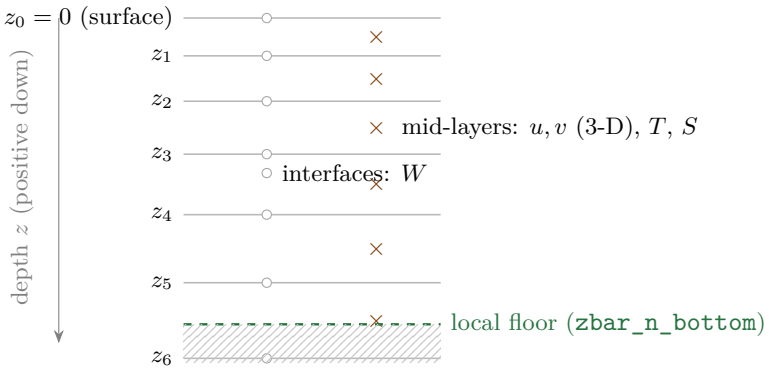


Figure 3.2: Anatomy of the vertical column. The K interfaces (grey lines, open circles) bound $K-1$ mid-layers (orange crosses). The vertical velocity W is stored at the interfaces; the three-dimensional horizontal velocity and the scalar fields are stored at the mid-layers. The local sea floor is given per node by `zbar_n_bottom`; layers below it do not exist for that node and a particle cannot descend past it (Chapter 6).

at the interfaces, whereas the three-dimensional horizontal velocity and the scalar fields are defined *at the mid-layers*. FESOM 1.4 is the exception: it stores every field on the levels, so its horizontal velocity shares the interfaces with W (Section 3.7). For the *core2* mesh there are 48 interfaces and hence 47 layers, spanning depths from 0 to approximately 6250 m. The configuration key `max_levels` retains only the first N interfaces from the surface downward (at least two in three dimensions, on either vertical family), which both saves memory and caps the depth a particle can reach; a run that follows water to the sea floor keeps the full column.

The engine treats FESOM2 levels as fixed depths (a *z-level* vertical coordinate), the same everywhere. In FESOM2 itself the interfaces follow the free surface (the zstar coordinate), and the recorded depth is therefore the reference depth, within $|\eta|$ of the physical one (Section 6.1). The alternative supported family, used by the FESOM-C model, is a terrain-following *sigma* coordinate in which the interface depths are a fixed fraction of the local water-column thickness and therefore vary in space and with the sea-surface height. The two families differ only in how an interface depth is computed; we defer the sigma geometry to Chapter 6. FESOM 1.4, HEALPix products and NEMO grids use z-levels.

3.3 Reading a mesh file

The FESOM2 reader accepts three mesh sources: a mesh-diagnostics NetCDF file, an ASCII mesh directory, and a Zarr store written by the region-extraction tool. It resolves the quantities of a diagnostics file by variable *name* rather than by dimension, which lets it read three naming

Table 3.1: Quantities read from the FESOM2 mesh-diagnostic file and their conversion to the `Mesh` structure.

File variable	`Mesh` member	Conversion / convention
`lon`, `lat` or `nodes` $(2, N)$	`nodes_lon/lat`	degrees; packed radians → degrees
`elements`, `elem` or `face_nodes`	`elem_nodes`	oriented to $(E, 3)$; one-based → zero-based
`nz` or `zbar`	`sigma`, `z_levels`	$\lvert\cdot\rvert$, positive-down interface depths
`zbar_n_bottom`	`depth`	$\lvert\cdot\rvert$, per-node sea floor (positive-down)
`elem_area`	element areas	used directly when present
`nod_in_elem2D` (+ count)	`node_to_elems`	one-based → zero-based; rebuilt if degenerate (Section 3.4); required in a diagnostics file
`zbar_e_bottom`, `nlevels`	element bottoms (metadata)	each element's bed and last whole layer, when present (Section 3.5)
`ulevels`	element first levels (metadata)	above 1 under an ice-shelf cavity

conventions — node coordinates as `lon/lat` in degrees or packed as one `nodes` array in radians, and the connectivity as `elements`, `elem` or `face_nodes`, oriented to $(E, 3)$ by its axis of length three — and applies a small number of unit and indexing conversions (Table 3.1). Packed coordinates are converted to degrees, trusting an explicit `units` attribute when present and otherwise inferring radians when the longitude magnitude does not exceed 2π and the latitude magnitude does not exceed $\pi/2$. The connectivity is shifted to zero-based indexing by its `start_index` attribute (one, as FESOM2 writes it from Fortran, when absent). Depth-like quantities — the interface levels and the per-node sea-floor depth — are stored with a negative-downward sign and are converted to the positive-down convention by taking absolute values. Element centres are computed from the vertices, with the longitude averaged on the circle so that an element across the dateline gets a centre inside it. The model's element areas are kept when the file provides them.

Source. Mesh-variable resolution and conversions are in `_resolve_mesh_vars`, `_load_mesh_nc` and `_compute_elem_centres` (`readers/fesom2.py`); the `Mesh` dataclass and its `z_levels` property are in `core/mesh.py`.

3.4 The node-to-element table and its reconstruction

The localization search requires, for each node, the list of elements that contain it; this is the `nod_in_elem2D` table in the mesh file. On the *core2* mesh this table is effectively empty: of the 733 977 node–element incidences it should hold (three per element) only 54 are present. The consequence, if the table were used as read, is severe and silent. After the routine one-based to zero-based shift, every unpopulated entry becomes -1, so that 126 852 of the 126 858 nodes report no adjacent elements. The containing-element search of Chapter 4 would then fail for every particle on the first step, flag each as having left the domain, and terminate normally — producing an output file in which no particle ever moved, with no error raised.

To prevent this, the reader detects a degenerate table and rebuilds it from the element connectivity, which is intact. The detector flags the table as degenerate on any of three independent signals: more than half of the nodes report no first element; an entry strictly inside a node's declared neighbour count is empty (the table and the count array disagree); or a neighbour count is negative or exceeds the maximum connectivity. On a positive verdict the engine prints a notice and reconstructs the node-to-element adjacency by a two-pass count-and-fill over the elements, after which the localization search behaves correctly. We note this behaviour explicitly because it is a precondition for any meaningful run on the *core2* mesh, and

because its failure mode — a completed run with no motion — is easy to misread as a physical result rather than a missing table.

Source. The degeneracy detector `_nod_in_elem2d_is_degenerate` and the rebuild call are in `readers/fesom2.py`; the reconstruction `build_node_to_elems` (a two-pass count-and-fill) is in `core/mesh.py`.

3.5 Element bottoms

FESOM2 fixes the number of levels of each element first and gives a node the deepest of the elements around it, so the per-node floor `zbar_n_bottom` tells where the deepest element at a node ends, not where a given element ends. The depth-linear reading of element-layout velocities (Section 6.4) needs the bed of every element, and the FESOM2 reader takes it from the mesh diagnostic — NetCDF or Zarr — when the diagnostic holds the element level variables: `nlevels` (one-based; the element has `nlevels` -1 wet layers), `zbar_e_bottom` (its bed, negative down) and `ulevels` (its first level, above 1 under an ice-shelf cavity). From them it declares, in file element order, the bed d_e and the last whole layer D_e of each element,

$$d_e = \begin{cases} |\texttt{zbar_e_bottom}_e| & \text{when present,} \\ z_{\texttt{nlevels}_e - 1} & \text{otherwise,} \end{cases} \qquad D_e = \max\{k : z_{k+1} \le d_e\} \quad (-1 \text{ when } z_1 > d_e), \tag{3.1}$$

that is, the last layer whose lower level lies at or above the bed. A bottom layer that the bed cuts — a FESOM2 *partial bottom cell* — is therefore not counted, and the depth-linear rule never reads it, the same rule as the node table of Equation (6.3). The element first levels are kept as well. A variable read here that is not one finite value per element, or that holds levels outside $1, \ldots, K$, refuses the mesh at load — in layer runs too — with a message that names the file and the fix, and the loader prints one line saying whether the bottoms came from the file. A mesh without either variable — an ASCII mesh directory, a diagnostic that lacks them, a Zarr store without a level table, or a region extracted before the extraction tool kept them — declares no bottoms, and `mode.uv_vertical: linear` on its element velocities is refused before the output file exists; `ulevels` alone declares nothing. A mesh with `ulevels` > 1 anywhere is refused for the same option: ice-shelf cavities are not supported in the code yet and are planned. The region-extraction tool `utils/extract_fesom_region.py` writes `nlevels`, `ulevels` and `zbar_e_bottom` of the kept elements, and `nlevels_nod2D` of the kept nodes, whenever the source has them, with the values and signs of the source; a subset written before it did so must be extracted again to use the option on element data.

Source. `_elem_level_metadata` in `readers/fesom2.py`, called by the NetCDF and the Zarr mesh loaders, declares `mesh.metadata["elem_bed"]`, `["uv_data_last_elem"]`, `["elem_ulevels"]` and `["elem_levels_source"]`; the element table that reads them is `depth_points` with `layout="element"` (`numerics/vertical_uv.py`); the subset writer is `utils/extract_fesom_region.py`.

3.6 Other mesh sources

The other readers produce the same `Mesh` from different inputs (Table 3.2 and Figure 3.3); the FESOM 1.4 reader has a section of its own (Section 3.7).

FESOM-C. The FESOM-C reader takes the node coordinates, depths and vertex lists from the model's single output file, together with the element centres when present and the sigma levels. Its connectivity is sanitised on load: indices are shifted to zero-based, empty and repeated vertices are removed, and each row is compacted, so a triangle stored as a quadrilateral with a repeated vertex becomes $[a, b, c, -1]$.

source	nodes	elements	velocity layout	vertical	radius (m)
FESOM2	model nodes	triangles	node or element	z-level	6 367 500
FESOM 1.4	model nodes	triangles, from the mesh files or the cell polygons	node, on the levels	z-level	6 367 500
FESOM-C	model nodes	triangles, quadrilaterals	element (node when stored so)	sigma	6 400 000
HEALPix, dual	wet pixel centres	corner cells: quadrilaterals, some triangles	node	z-level	6 371 000
HEALPix, pixel	pixel corners	wet pixels (quadrilaterals)	element	two-dim.	6 371 000
NEMO	wet T-points	quadrilaterals of four wet T-points	node	two-dim.	6 371 000

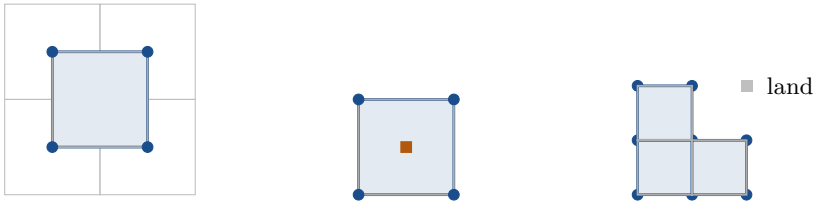
Table 3.2: Mesh sources. The radius is the default of `mesh.r_earth`; it enters the fallback computation of element areas and, without effect on the values, the barycentric projection.

HEALPix, dual mesh. A HEALPix product stores its fields at the centres of equal-area pixels. The default *dual* mesh takes the wet pixel centres as nodes and builds one element around each pixel corner, joining the pixels that meet there: four pixels give a quadrilateral, and at the eight points of the sphere where only three pixels meet, a triangle. The cells are built from the pixel neighbour relations in integer arithmetic, each kept once; an all-wet sphere must give exactly two more elements than pixels, which the builder checks. A pixel is wet when the velocity at the first record is finite on the tracked level, restricted by `mesh.crop_bbox` and by a bathymetry variable when the store has one. The velocities live at the nodes, so the dual mesh needs `uv_interp: barycentric`.

HEALPix, pixel mesh. With `mesh.healpix_geometry: pixel` the elements are the wet pixels themselves, quadrilaterals whose nodes are the pixel corners; the velocities live at the element centres, so both least-squares schemes apply. A quadrilateral with straight edges approximates a pixel with curved edges, so near an edge a point may be assigned to the adjacent pixel. The pixel mesh is two-dimensional; the eight pixels that touch a pole are degenerate in the geographic frame and produce a warning.

Structured grids and NEMO. A logically rectangular grid becomes a mesh of quadrilaterals whose corners are four adjacent wet cell centres (T-points); a quadrilateral with a land corner is dropped, which draws the coastline, and a wet T-point that belongs to no quadrilateral is left out, since it could only be a dead end. The NEMO reader averages the C-grid velocities at the cell faces to the T-points, setting missing face values to zero first, which gives node-layout velocities. Its first version is two-dimensional and requires a grid whose axes are within 2° of east and north.

Source. `_sanitize_connectivity` and `FesomCReader.load_mesh` (`readers/fesomc.py`); `build_healpix_dual_mesh`, `build_healpix_pixel_mesh` and `HealpixReader.load_mesh` (`readers/healpix.py`); `build_structured_mesh` and `c_grid_to_t` (`readers/structured.py`); the NEMO adapter in `readers/nemo.py`.



(a) dual: nodes at pixel centres, cell at a corner (b) pixel: element = pixel, velocity at its centre (c) structured: quads of four wet T-points

Figure 3.3: Three mesh sources on one `Mesh`. (a) The HEALPix dual mesh joins the four pixel centres around a pixel corner into a quadrilateral (three centres give a triangle). (b) The pixel mesh makes each wet pixel an element, with nodes at its corners and the velocity at its centre. (c) A structured grid keeps the quadrilaterals whose four T-points are wet; one land point removes the quadrilateral in the corner.

3.7 The FESOM 1.4 mesh

FESOM 1.4 (Wang et al., 2014) is a finite-element model on a triangular surface mesh that is the same at every level. Every surface node has a column of three-dimensional nodes, one per level, down to its bed, and all prognostic fields — u, v, W, temperature and salinity — are continuous, piecewise-linear (P1) functions on the tetrahedra between those nodes: they live *on the levels at the nodes*. The reader therefore returns node-layout fields on the levels (Section 6.4), and it builds the `Mesh` from either of two sources, decided by what `mesh.meshpath` is.

ASCII mesh directory. A directory that holds `nod3d.out` is read as the model's mesh files: `nod2d.out` (the node coordinates and flags), `elem2d.out` (the triangles, one-based), `nod3d.out` (every three-dimensional node with its depth, numbered level by level) and `aux3d.out` in FESOM 1.4's own format — the number of levels, the table of the three-dimensional node at each level of each column (a value ≤ 0 marks a dry slot) and the surface node of every three-dimensional node; its last block, on the three-dimensional elements, is not parsed. This `aux3d.out` is not FESOM2's, and the FESOM2 reader would misread it. Each file is read as one stream of numbers, as a Fortran list-directed read sees it, so the spread of the records over lines does not matter; Fortran `D` exponents are accepted, and an integer field takes any number whose value is an integer. The levels are the depths of `nod3d.out` in file order — every change of value starts a new level — stored positive down, with the surface exactly at zero. Before anything is returned the files are checked against each other: indices in range, every three-dimensional node in exactly one slot, the wet slots a prefix of each column, and every slot's surface node and depth consistent with the node tables. A failure names the file and the first offending index.

A FESOM 1.4 run may store its mesh in the rotated frame of the model. The key `mesh.input_rotation` takes the Euler angles $[\alpha, \beta, \gamma]$ of the run's namelist, in degrees, and rotates the node coordinates back to geographic on load, with the convention of Section 8.1. The angles are never guessed, since a rotated mesh looks like a geographic one; the reader warns when a global ASCII mesh (more than 350° of longitude) without a node north of 85° N is loaded without them, because a geographic FESOM mesh reaches the Arctic. The key is refused for a mesh from cell polygons, whose coordinates are geographic, and for every other mesh type; it is independent of `mesh.rotation`, the frame in which the engine tracks (Chapter 8).

Cell polygons of level-wise data. Published FESOM 1.4 output, such as the CMIP6 files, holds no element table. It holds, next to the node coordinates `lon`/`lat`, each node's *median-dual cell* as a polygon (`lon_bnds`/`lat_bnds`, padded to a fixed vertex count by repeating the last vertex). An interior node's polygon alternates the midpoints of its edges and the centroids of its triangles; a coastal node's polygon also holds the midpoints of its boundary edges and the

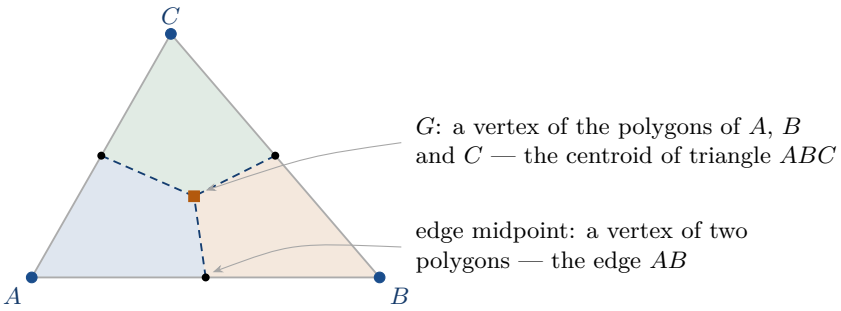


Figure 3.4: Triangle recovery from FESOM 1.4 cell polygons. The median-dual cell of each node (shaded, one colour per node) is bounded by the dashed segments joining the edge midpoints (dots) to the triangle centroid G (square). Within one triangle each node's cell contributes the quadrilateral node, midpoint, centroid, midpoint, so G is the only point that the polygons of exactly three nodes share: every cluster of polygon vertices owned by three nodes is a triangle of the mesh.

node itself. A polygon vertex shared by exactly three nodes is therefore the centroid of their triangle (Figure 3.4), and the reader recovers the triangles exactly from that rule. The polygon vertices of all nodes are clustered on the unit sphere within 10^{-6} degrees (about 0.1 m); distinct clusters must lie at least a thousand times farther apart, so that no point split by rounding can pass for two. At a *pinch node*, whose triangles form several fans that touch only at the node, the published polygon lists one fan only; a centroid listed by just two nodes then names its third node, and that triangle is recovered as well. The result is validated before it is used: a shared point within 1 cm of the spherical centroid of its three nodes, every edge in one or two triangles with a polygon vertex at its midpoint, the Euler characteristic $V - E + F = 2C - L$ of C edge-connected components with L boundary loops, and an accounting check that every distinct polygon vertex is a node, the midpoint of a recovered edge or the centroid of a recovered triangle. The levels are the file's depth coordinate (finite, starting at the surface, increasing), and the wet mask comes from the first record of u: a value counts as data when it is finite and smaller than 10^{29} in magnitude, and the data of every column must be a prefix of its levels, starting at the surface. When a file holds `deptho` (for instance the published fixed-field file in the same directory), its value must equal the depth of each column's last level with data within 0.01 m. This is the only guard against level-wise files that write zero rather than a fill value below the bed, whose first record would look wet everywhere; without `deptho` that case cannot be detected.

The mesh in either case. `mesh.sigma` holds the full level table, positive down, and `mesh.depth` the depth of each column's last wet level, exactly. The reader keeps the static wet mask of every column (against which every record of every variable is later compared, Section 14.1), the geographic node coordinates, and the index of each column's last wet level, which it declares as the data prefix of the depth-linear rule (D_i in Equation (6.3)). Element centres are circular means of the vertices, as for FESOM2, and the default radius is 6 367 500 m, FESOM 1.4's value. A mesh whose columns do not all start at the surface — a mesh with ice-shelf cavities — is refused by the reader: an ASCII mesh whose top wet node lies below $z = 0$, or level-wise data with a column that holds no surface value. Ice-shelf cavities are not supported in the code yet and are planned.

Source. The mesh is built by `Fesom14Reader.load_mesh` in `readers/fesom14/reader.py`, with the helpers `_from_ascii`, `_from_polygons`, `_check_deptho` and `_assemble` of the same file; the ASCII files are parsed by `read_ascii_mesh` (`readers/fesom14/_ascii_mesh.py`) and the triangles recovered by `recover_triangles` (`readers/fesom14/_polygons.py`); the validity predicate `is_data` and `wet_mask_levelwise` (`readers/fesom14/_layout.py`); the Euler rotation `euler_matrix` (`core/rotation.py`).

3.8 Convex quadrilaterals and derived tables

The quadrilateral weights of Section 5.5 have no fall-back, so every quadrilateral must be convex. The FESOM-C reader, the structured-grid builder and the HEALPix dual builder check this when the mesh is loaded (the HEALPix pixel builder too, except for the eight pixels that touch a pole, which it only reports with a warning): each quadrilateral is projected onto the plane tangent to the sphere at its centre, and it fails when the cross products of consecutive edges change sign or one is zero within a relative tolerance of 10^{-9}. The test does not depend on the winding direction or on a rotation of the frame, and a failure names the offending elements.

Three further tables are built from the connectivity when a run needs them and are cached on the mesh:

Element neighbours.
For each element, the element across each of its edges, or none at the coast; an edge shared by more than two elements is an error. Used by the edge least-squares scheme and by element tracking (Sections 4.6 and 5.4), and built when the run starts if either is on.

Coastal flags.
An element is coastal when one of its edges belongs to no other element; used by the beaching model (Section 13.5).

Element areas.
Read from the model's mesh file when present (FESOM2 mesh diagnostics provide them; ASCII meshes and regional extracts do not); otherwise the spherical excess of the element's fan of triangles, times the square of the mesh radius. Used only as ratios, in the element-to-vertex averages of the barycentric scheme.

The search structures of Chapter 4 are built at the end of every loader.

Source. `find_nonconvex_quads`, `validate_quad_convexity`, `build_elem_neighbors`, `build_boundary_info` and `build_elem_areas` in `core/mesh.py`.

Chapter 4

Particle Localization

Before the velocity can be interpolated to a particle, the particle must be located on the mesh: the engine must identify the nearest node and, for most interpolation methods, the element that contains the position. On a regular latitude–longitude grid this is an indexing operation; on an unstructured mesh it is a search, performed for every particle at every evaluation site — the start of a step, every Runge–Kutta stage (Chapter 9) and the final position. This chapter describes the nearest-node search by spatial hashing and the guarantee that the node it returns is the nearest one, the automatic choice of the hash cell, the containing-element test, and the alternative of tracking the containing element across edges.

4.1 Nearest node by spatial hashing

A linear scan over all N nodes would make every evaluation cost $O(N)$. Instead the engine builds a uniform *grid hash* once, when the mesh is loaded, and scans only the nodes of a small window of cells around the query (Figure 4.1).

Construction. Node longitudes are first placed on one branch — $[0°, 360°)$ or $[-180°, 180°)$, whichever gives the smaller span; a mesh whose span exceeds $350°$ on both branches is treated as cyclic in longitude. With cell side c, the grid starts one cell below the southernmost node, $\phi_{\min} = \min\phi - c$, and has $n_y = \lceil (\max\phi + c - \phi_{\min})/c \rceil$ rows; a cyclic grid starts at $\lambda_{\min} = 0$ with $n_x = \lceil 360/c \rceil$ columns, whose last ("seam") cell is only $360 - (n_x - 1)c$ wide, and a non-cyclic grid carries one padding cell on each side. A query at (λ, ϕ) falls in the cell

$$i = \left\lfloor \frac{(\lambda - \lambda_{\min}) \bmod 360}{c} \right\rfloor, \qquad j = \left\lfloor \frac{\phi - \phi_{\min}}{c} \right\rfloor, \tag{4.1}$$

clamped to the grid; on a non-cyclic grid a query outside the domain that lies angularly closer to the western edge is folded to the negative side first, so the clamp picks the nearer edge. The nodes of every cell are stored contiguously (a compressed offset array and a node-index array), so a row of cells is one contiguous slice.

Distance. Distances are compared as squared chord lengths between points on the unit sphere, $\|\mathbf{r}_p - \mathbf{r}_n\|^2$ with $\mathbf{r} = (\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi)$. Minimising the chord minimises the great-circle distance, without the dateline discontinuity or the polar distortion of a planar metric. The node vectors are computed once, when the hash is built, into a table of three numbers per node, so a scan reads coordinates instead of evaluating trigonometric functions per candidate; the table is filled with exactly the arithmetic the scans used before, so the nearest node found is bit-for-bit unchanged, ties included (the first node found wins a tie) — provided Numba runs without Intel's vector math library, which the hash build checks on sampled nodes

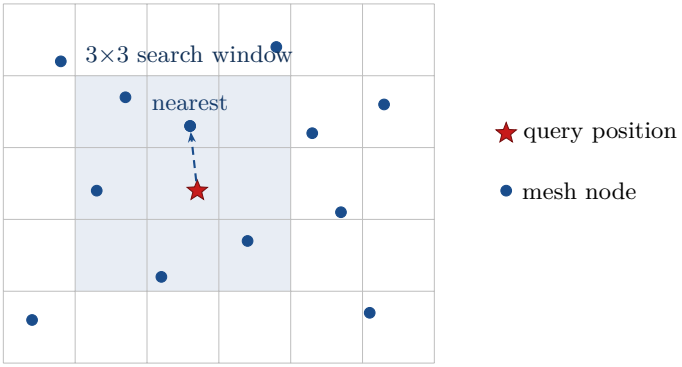


Figure 4.1: Nearest-node search by spatial hashing. Nodes are bucketed into uniform cells when the mesh is loaded. A query (the drifter) is first matched against the nodes of its own cell and the eight neighbouring cells (shaded); Section 4.3 gives the test that decides whether this window is enough.

and warns about. The plain three-by-three window scan returns the nearest node among the nodes of the query cell and its eight neighbours.

Source. Construction `build_grid_hash` (with `_gh_lon_branch`, `_gh_grid_geometry`, `_gh_cell_of`); node table `_node_xyz_fill` and its self-check `_node_xyz_selfcheck`; window scan `_grid_hash_nearest_3x3`; batch query `find_nearest_nodes`; all in `core/mesh.py`.

4.2 Choosing the cell

The reader of each mesh family proposes a cell of twice the mean node spacing,

$$c_{\text{legacy}} = \max\left(0.05^\circ,\ 2\sqrt{A/N}\right), \tag{4.2}$$

with A the area of the mesh's longitude–latitude bounding box in square degrees. On a quasi-uniform mesh this puts a few dozen nodes in the window at every resolution. On a mesh whose resolution varies by orders of magnitude it does not: a cell sized for the coarse part holds thousands of nodes where the mesh is fine, and the scan turns into a linear pass over them.

Every run therefore chooses the cell by measurement: the key `mesh.grid_hash_cell_rule` defaults to `occupancy`. The rule samples up to 200 000 element centres, finds each one's exact nearest node once, and evaluates the candidate cells $c_k = c_{\text{legacy}}/\sqrt{2}^{\,k}$, $k = 0, \ldots, 12$, without rebuilding the hash: for every sample it counts the nodes the window holds, tests whether the window proves its answer (Section 4.3), and estimates the cost of the query — the nodes scanned, the cells visited and one certificate per widening, a certificate weighing as much as twenty cells. With the exact-anchor guard of Section 4.3 on (the default), the rule keeps the reader's cell unless the cheapest eligible candidate (at most 20 % of the queries widened, at most 128 MB of offsets) is predicted to be at least 2.75 times cheaper; with the guard off it takes the smallest cell that leaves no window empty, holds a median of at least eight nodes per window and has at most half a percentage point more approximate anchors than the reader's cell. The ladder stops early where a candidate would need more than 64 rows of widening budget or too many cells. With the guard on, quasi-uniform meshes keep the reader's cell and structure-resolving meshes take a smaller one (Section 17.3). An explicit `mesh.grid_hash_cell_size` overrides the rule, and `grid_hash_cell_rule: legacy` keeps the reader's cell. The resolved cell and the rule are written to the output metadata, so any run can be repeated with its cell pinned.

Source. `choose_grid_hash_cell` (the per-sample statistics in `_gh_sample_stats`), `apply_grid_hash_cell_rule`, `set_grid_hash_cell` and the one-line report `describe_grid_hash`, in `core/mesh.py`; applied by `run_tracking` after the mesh is loaded.

4.3 Exact nearest nodes

A non-empty three-by-three window does not guarantee that the node it returns is the nearest one: a closer node can lie just outside the window, most easily at high latitude, where a degree of longitude shrinks, or with a cell smaller than the local node spacing. A near-nearest anchor changes the interpolation stencil and, over time, the trajectory. With `mesh.exact_nearest: true` (the default) the engine proves every anchor.

The certificate. Let the window return a node at angular distance $\theta = \arccos(\sin\phi_p \sin\phi_n + \cos\phi_p \cos\phi_n \cos\Delta\lambda)$ from the query. Any closer node lies inside the spherical cap of radius θ around the query, whose half-extent is θ in latitude and

$$\Delta\lambda_{\mathrm{cap}} = \arcsin\left(\frac{\sin\theta}{\cos\phi_p}\right) \tag{4.3}$$

in longitude, unbounded when $|\phi_p| + \theta \geq 90°$ (the cap reaches a pole). The answer is proven when the cap fits inside the latitude and longitude intervals the window actually searched — taking into account the narrower seam cell of a cyclic grid, a window clipped at the grid edge (there are no nodes beyond it), and, on a non-cyclic grid, the $\pm 360°$ images of the cap (Figure 4.2). The test returns which sides fail: the rows, the columns, or — when the cap reaches a pole or an image of it meets the node range — all columns.

Widening. An unproven window grows in the failing direction — rows and columns independently, all columns only where required — and is scanned again, new rows as contiguous slices, until the best node found is proven. An empty window grows as a square until the first node is found, or by whole rows in the polar cap, where a row's length $360° \cos\phi$ is at most 32 cell widths. The growth has a budget: a query whose nearest node is farther away than the longest element edge (plus two cells) lies outside every element, so it keeps the best node seen and dies in that step; the budget, between 4 and 4096 rows, is derived from the mesh when the hash is built.

One search for every path. The batch query and every fused kernel's evaluation site call the same exact search, so fused and unfused paths return the same anchor bit for bit, ties included, and the fused kernels need no round trip through the batch path. With the guard off (`exact_nearest: false`; together with `grid_hash_cell_rule: legacy` this reproduces the nearest-node anchors of runs made before September 2026) the batch path keeps the plain window and repairs empty windows through a k-d tree, and the fused kernels widen an empty window ring by ring — an approximate anchor. The two-dimensional barycentric kernels, and the least-squares kernels on HEALPix pixel meshes, are then enabled only on meshes that pass a density certificate (every cell of every element's bounding box has a non-empty window); the three-dimensional RK2 kernel instead flags a particle whose window is empty and re-runs it with the anchor of the batch search (Section 11.3).

> **Source.** Cap extents `_cap_extents`; certificate `_anchor_provable_rect` (and `_anchor_provable` for the plain window); exact search `_grid_hash_nearest_exact`; budget `_gh_far_rows_for`; guard flag `set_exact_nearest`; guard-off single-particle search `_grid_hash_nearest_widened`; density certificate `grid_hash_dense_for_elements`; all in `core/mesh.py`. The fused kernels anchor each site through `_fused_site_anchor` in `numerics/integrators.py`.

4.4 Where localization runs

Every evaluation site of the step re-queries the hash; no step path updates the anchor from the previous one (only the offsets of the diffusivity gradient, Chapter 12, start a local walk from the

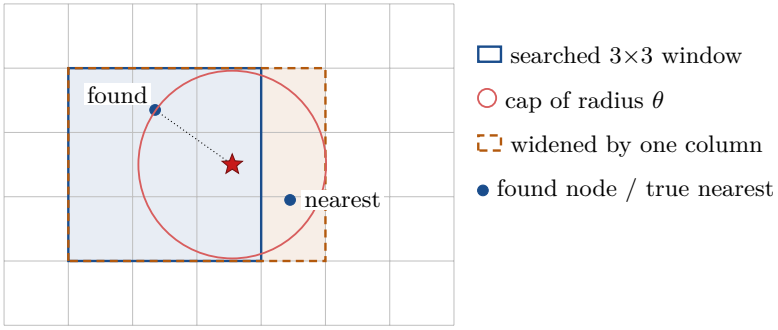


Figure 4.2: The exactness certificate. The window returns a node at angular distance θ; a closer node can only lie in the cap of radius θ around the query. Here the cap crosses the window's eastern edge, so the window cannot prove its answer; it widens by one column, finds the true nearest node, and the smaller cap of the new answer fits inside the searched interval.

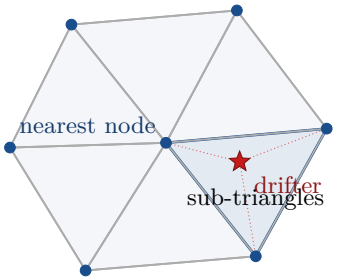


Figure 4.3: Containing-element search. The engine scans the one-ring of the nearest node and tests each element for containment. The drifter lies in the shaded triangle; the dotted segments split it into three sub-triangles whose signed areas, normalized by the whole, are the barycentric coordinates that decide containment.

particle's node). A fused kernel anchors its start, its stages and its final position through the exact search and tests the final position for containment; the unfused paths query the batch search for the particles that moved and test their containment. A particle folded back into a periodic window (Chapter 13) is re-anchored after the fold. On a HEALPix mesh in the geographic frame, points poleward of the first ring of pixel centres lie outside every element and are lost; the rotated frame of Chapter 8 removes this limit.

4.5 The containing element

The nearest node does not by itself determine every interpolation stencil; several methods need the element that *contains* the particle. The engine scans the elements of the nearest node's one-ring and tests each for containment (Figure 4.3). If none contains the particle — which can occur on an irregular mesh, where the nearest node is not always a vertex of the containing element — the search widens to a second ring, scanning the elements adjacent to every vertex of the one-ring elements. A particle for which neither ring yields a containing element is taken to lie outside the domain; its step is undone and its status set to `OUTSIDE` (Chapter 13).

The containment test proceeds in three steps. First, the valid vertices of the candidate element are compacted, discarding the -1 padding and, on the mixed meshes of FESOM-C, the repeated vertex. Second, a dateline-safe bounding-box prefilter, in longitude differences wrapped to $[-180°, 180°)$, rejects candidates that cannot contain the point. Third, the element and the point are projected to a local plane — an equirectangular plane around the mean vertex, $x = \Delta\lambda\, R \cos\phi_0$, $y = \Delta\phi\, R$, with the mean longitude taken on the wrapped differences when the element straddles the dateline — and the point is declared inside when its barycentric

coordinates are all non-negative to within a small negative tolerance, so that a point exactly on an edge is inside and no particle falls between two neighbours. A near-degenerate triangle, whose area falls below a second tolerance, is rejected rather than divided by. A quadrilateral is tested on a fan of triangles, which is exact because quadrilaterals are required to be convex (checked when the mesh is loaded, Chapter 3); its interpolation weights are the Wachspress coordinates of Section 5.5, computed separately.

Source. The ring search is `_find_containing_element_local`; the containment test is `_element_contains` (bounding box) and `_polygon_contains_bool` (barycentric signs) on `_project_to_local_xy`; the tolerances `_BARY_EPS` $= -10^{-6}$ and `_AREA_TOL` $= 10^{-10}$ are module constants. All in `core/mesh.py`.

4.6 Tracking the element across edges

With `numerics.elem_search: walk` the containing element is searched once, when the run starts (or taken from the checkpoint of a resumed run), and afterwards carried from step to step: at every evaluation site the engine starts from the carried element and walks across edges towards the position. In an element with signed area S and compacted vertices (x_k, y_k), the edge coordinates of the point are

$$w_k = \frac{\operatorname{sgn} S}{|S|}\left[(x_{k+1} - x_k)(p_y - y_k) - (p_x - x_k)(y_{k+1} - y_k)\right], \tag{4.4}$$

and the point is inside when $\min_k w_k \geq -10^{-6}$; otherwise the walk crosses the edge with the most negative w_k into the neighbouring element. An acceptance within 10^{-2} of an edge is re-checked with the exact containment predicate of Section 4.5. Every outcome that is not a clear containment — a boundary edge, a two-element cycle, a degenerate element, or 256 steps — falls back to the ring search of Section 4.5, so the walk returns the same element as the ring search wherever that search is exhaustive and finds elements beyond the second ring where it is not. The carried element is written to checkpoints. On a 28 000-element mesh the walk costs 35 ns per particle against 126 ns for the ring search; it runs on the unfused paths only (the fused kernels keep their own search), which is why it pays off for element-layout barycentric and edge least-squares interpolation and is left at `local` on node-layout runs whose fused kernels would otherwise be switched off.

Source. `_walk_containing_element` and `find_containing_elements_walk` in `core/mesh.py`; the element neighbours come from `build_elem_neighbors`; the tracking layer seeds and updates the carried elements (`_walk_update_elems` in `numerics/integrators.py`).

Part III

Interpolation in Space

Chapter 5

Horizontal Velocity Interpolation

Once a particle has been located (Chapter 4), the horizontal velocity must be interpolated from the surrounding mesh values to the particle position. KELP provides three schemes, selected by the configuration key `uv_interp`: a *barycentric* scheme, which blends the values at the vertices of the containing element; a *least-squares* scheme (the default), which fits a plane to the velocities at the element centres around the nearest node; and an *edge least-squares* scheme, which fits the plane to the containing element and its edge neighbours, with a wall condition at the coast. The schemes are matched to the storage layout of the velocity (Section 3.1 and Table 5.1): the two least-squares schemes read element-centre values, so supplying node-layout velocity to either is refused at startup. This chapter gives the three schemes explicitly, including the local planar frames in which they compute, and states the accuracy each achieves.

5.1 Projection to a local plane

All three schemes compute in a local planar frame rather than directly in longitude and latitude, so that the spherical geometry is treated correctly away from the equator and across the dateline. The two families use different frames.

Least-squares family. The frame is centred on the particle (λ_p, ϕ_p) and scaled by the metric at the particle's latitude,

$$z_0 = \cos\phi_p \cdot M, \qquad x_k = (\lambda_k - \lambda_p)\, z_0, \qquad y_k = (\phi_k - \phi_p)\, M, \tag{5.1}$$

with $M = 111\,120$ m per degree of latitude. The factor $\cos\phi_p$ is the essential spherical correction: it shrinks the longitudinal axis so that a degree of longitude and a degree of latitude map to their physical lengths. Across the dateline the plain least-squares scheme shifts an element centre by one full turn only when the raw difference $\lambda_k - \lambda_p$ exceeds $\pm 180°$, which leaves every stencil away from the seam bit-identical to the original arithmetic; the edge least-squares scheme wraps every difference to $[-180°, 180°)$.

Barycentric scheme. The frame is centred on the element: its centre is the mean of the vertices, with the longitudes accumulated as wrapped differences from the first vertex so that an element straddling the dateline keeps its shape, and a point maps to

$$x_k = \Delta\lambda_k \cdot \frac{\pi}{180}\, R\cos\phi_0, \qquad y_k = (\phi_k - \phi_0) \cdot \frac{\pi}{180}\, R, \tag{5.2}$$

where (λ_0, ϕ_0) is the frame centre, $\Delta\lambda_k$ the wrapped longitude difference and R the mesh's Earth radius. The weights computed below are ratios of areas in this plane and are therefore independent of R; the radius serves only to keep the coordinates dimensionally consistent.

velocity source	layout	admissible `uv_interp`
FESOM2 `u`/`v`; FESOM-C element fields; HEALPix pixel geometry	element	`leastsq`, `edge_leastsq`, `barycentric`
FESOM2 `unod`/`vnod`; FESOM 1.4 `uo`/`vo`; FESOM-C node fields; HEALPix dual mesh; NEMO (C-grid faces averaged to T-points)	node	`barycentric`

Table 5.1: Velocity layouts by source and the interpolation schemes each admits. The layout is detected from the field's dimensions; a metadata-only probe before the output file is created and a backstop on the first step reject a mismatch with a message naming the fix.

Source. The projections are performed inline in `_interp_leastsq_core`, `_interp_edge_leastsq_core` and `_interp_barycentric_core` (`numerics/interpolate.py`).

5.2 Barycentric interpolation

The barycentric scheme is the one scheme that serves node-layout velocity. The value at the particle is the weighted average of the velocities at the vertices of the containing element, with weights equal to the barycentric coordinates of the particle within it (Figure 5.1); we give the triangle here and the quadrilateral in Section 5.5. Let the projected triangle have vertices $1, 2, 3$ at (x_i, y_i) carrying velocities (u_i, v_i), and let the particle project to (x_p, y_p). The barycentric coordinate w_i is the area of the sub-triangle *opposite* vertex i — the sub-triangle formed by the particle and the other two vertices — divided by the area of the whole triangle:

$$w_i = \frac{\text{area(sub-triangle opposite } i)}{\text{area(triangle)}}, \qquad w_1 + w_2 + w_3 = 1. \tag{5.3}$$

A particle coinciding with vertex i therefore has $w_i = 1$ and the other weights zero, and a particle on an edge has zero weight for the opposite vertex; the weights vary continuously across element boundaries. In implemented form the weights are the signed-area ratios

$$\begin{aligned} D &= (y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3), \\ w_1 &= \frac{(y_2 - y_3)(x_p - x_3) + (x_3 - x_2)(y_p - y_3)}{D}, \\ w_2 &= \frac{(y_3 - y_1)(x_p - x_3) + (x_1 - x_3)(y_p - y_3)}{D}, \qquad w_3 = 1 - w_1 - w_2, \end{aligned} \tag{5.4}$$

in which D is twice the signed area of the triangle; a triangle whose $|D|$ falls below a small tolerance is treated as degenerate and the interpolation reports failure rather than dividing by it.

The interpolated velocity is then the weighted sum of the three vertex velocities, multiplied by the integration-direction factor of Section 2.5:

$$(U, V) = s_{\mathrm{fb}} \sum_{i=1}^{3} w_i \, (u_i, v_i). \tag{5.5}$$

For node-layout fields the vertex velocity (u_i, v_i) is read directly from the node. For element-layout fields handled through the same routine, each vertex value is first formed as the area-weighted average of the velocities of the elements sharing that vertex, $u_n = \sum_{e \ni n} A_e u_e \,/\, \sum_{e \ni n} A_e$ (a vertex whose areas sum to zero fails the interpolation), and the blend Equation (5.5) is applied to those vertex values. The scheme reproduces a constant field exactly in both layouts and a linearly varying field exactly in the node layout; in the element layout the vertex-averaging

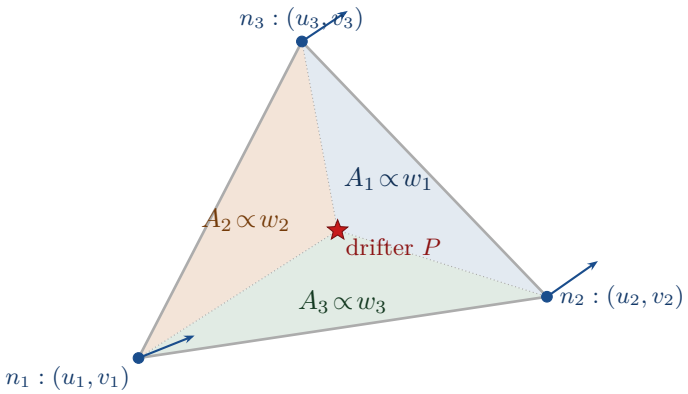


Figure 5.1: Barycentric interpolation of node-layout velocity. The three triangle vertices n_1, n_2, n_3 carry the node velocities (u_i, v_i). The drifter P splits the triangle into three sub-triangles; the weight w_i is the area of the sub-triangle *opposite* vertex i divided by the total area, so the nearer the particle lies to a vertex, the larger that vertex's weight. The interpolated velocity is $w_1(u_1, v_1) + w_2(u_2, v_2) + w_3(u_3, v_3)$.

step makes it smoothing rather than linearly exact. The interpolated field is continuous across element edges. A quadrilateral element is handled by the same blend with Wachspress coordinates (Warren, 1996; Floater, 2003) in place of the triangular weights of Equation (5.3) (Section 5.5).

Source. The barycentric blend is `_interp_barycentric_core` (`numerics/interpolate.py`); the triangle weights Equation (5.4) are `_barycentric_weights_tri` and the Wachspress generalization `_wachspress_weights_quad` (`core/mesh.py`).

5.3 Least-squares interpolation (element layout)

For element-layout velocity, the default scheme fits a plane to the velocities of the element centres surrounding the nearest node and evaluates that plane at the particle (Figure 5.2). The stencil is the one-ring of elements about the nearest node; their centres, carrying the element velocities (u_e, v_e), are the data. With the plane written $f(x, y) = a_0 + a_1 x + a_2 y$ in the particle-centred frame of Equation (5.1) (so the particle sits at the origin), the coefficients minimize the residual

$$\min_{a_0, a_1, a_2} \sum_e (a_0 + a_1 x_e + a_2 y_e - f_e)^2, \tag{5.6}$$

and because the frame is centred on the particle the interpolated value is simply the constant term, $U = s_{\mathrm{fb}}\, a_0^{(u)}$ and $V = s_{\mathrm{fb}}\, a_0^{(v)}$. The minimization reduces to a symmetric 3×3 system of normal equations, solved directly by Cramer's rule. Centring the coordinates on the particle is not cosmetic: it avoids the catastrophic cancellation that would arise from forming the normal equations with raw longitude and latitude values. The system is declared singular, and the interpolation fails, when

$$|\det A| < 10^{-30} \quad \text{or} \quad |\det A| < 10^{-12}\, |a_{11} a_{22} a_{33}|, \tag{5.7}$$

the second test being relative to the scale of the metric entries: a stencil of fewer than three non-collinear centres — a coastal node with only two elements, for instance — has a determinant of pure rounding noise, which an absolute test alone would pass. The same guard protects the plane fits of the vertical velocity, the column height and the diffusivity; the scalar-sampling fit uses an absolute threshold of 10^{-12} with the same relative test. The scheme is exact for fields varying linearly over the stencil. Its stencil is the one-ring of the *nearest node*, so the interpolated field jumps where the nearest node changes, and the anchor must be exact for two runs to agree (Section 4.3).

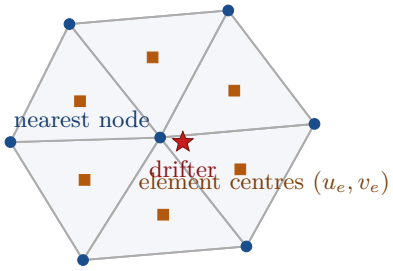


Figure 5.2: Least-squares interpolation of element-layout velocity. The data are the velocities at the centres (orange squares) of the one-ring of elements about the nearest node. A plane $f = a_0 + a_1 x + a_2 y$ is fitted to them in a frame centred on the drifter, so the interpolated velocity is the fitted constant a_0. The scheme is exact for linearly varying fields over this stencil.

The schemes are tied to the storage layout, and the engine enforces the match rather than silently producing a wrong answer. Because both least-squares schemes index velocities at element centres, supplying them with node-layout velocities such as `unod`/`vnod` is a category error; the engine detects the mismatch from the field's dimensions and raises a configuration error at startup, with the message that node-based velocities require `uv_interp` = `barycentric`. The check is performed both during a metadata-only probe before any data is read and as a backstop on the first step, so the run fails fast and informatively rather than midway through.

Source. The plane fit is `_interp_leastsq_core` (`numerics/interpolate.py`); the layout-compatibility check that rejects node velocities under both least-squares schemes is `_validate_uv_compat` (`tracking.py`), fed by each reader's `probe_uv_layout`.

5.4 Edge least-squares interpolation

The edge least-squares scheme (`uv_interp: edge_leastsq`) fits the same plane as Equation (5.6), in the particle-centred frame, but over a different stencil: the centre of the element that contains the particle and the centres of the elements across each of its edges — four points for a triangle, five for a quadrilateral (Figure 5.3). The stencil follows the containing element rather than the nearest node, as in the finite-volume discretisation of the FESOM models, and it needs the containing element, so a particle outside every element fails the interpolation. The plane is exact for a linear field in the interior.

At the coast an edge has no neighbour. It contributes a *ghost* point instead: the own centre $\mathbf{c}$ mirrored across the edge, $\mathbf{g} = 2\mathbf{f} - \mathbf{c}$, with $\mathbf{f}$ the foot of $\mathbf{c}$ on the edge. With $\hat{\mathbf{t}}$ the unit tangent of the edge and $\mathbf{u}$ the own velocity, the ghost carries

$$\mathbf{u}_g = \begin{cases} 2(\mathbf{u}\cdot\hat{\mathbf{t}})\,\hat{\mathbf{t}} - \mathbf{u} & \text{free slip (default)}, \\ -\mathbf{u} & \text{no slip}, \end{cases} \tag{5.8}$$

selected by `edge_leastsq_bc`. The free-slip ghost is the mirror image of the own velocity — its tangential component kept and its normal component reversed — so the fitted plane carries a flow parallel to the wall exactly and damps the flow through it. The no-slip ghost makes the plane vanish near the wall; being a single plane fitted to both sides, it overshoots on the interior side of a wall element and can reverse the flow near a corner between two walls, a property of the fit rather than a defect. A boundary edge of zero length contributes no point. The stencil stays well conditioned even on very thin elements, because the mirrored ghosts span both directions.

Source. `_interp_edge_leastsq_core` (`numerics/interpolate.py`); the edge neighbours are `build_elem_neighbors` (`core/mesh.py`), built once when the run starts.

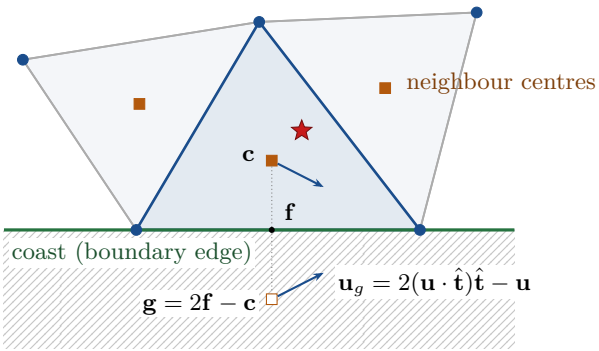


Figure 5.3: The edge least-squares stencil. The drifter lies in the shaded triangle; the plane is fitted to its centre **c** and the centres of the elements across its two interior edges. Its third edge is on the coast and contributes a ghost at the mirror image **g** of **c**, carrying the mirrored velocity: the component along the wall is kept and the component into the wall reversed (free slip), or the whole velocity reversed (no slip).

5.5 Interpolation on mixed meshes: quadrilateral elements

The barycentric scheme of Section 5.2 generalizes from the triangular meshes of FESOM2 to meshes with quadrilaterals: the mixed meshes of FESOM-C, the dual mesh built from a HEALPix grid (mostly quadrilaterals; an all-wet sphere has exactly eight triangles), the pixel geometry of a HEALPix grid and the meshes built from structured NEMO grids (quadrilaterals only). Every horizontal interpolator is split into a *geometric weight stage*, which depends on the number of vertices of the containing element, and a *value stage*, which depends only on the field layout. The plain least-squares scheme bypasses the geometric stage entirely — it reads one velocity per element centre — and is identical on triangles and quadrilaterals; the edge least-squares scheme differs only in the number of edges.

Before the weight stage, the element's vertices are compacted: the -1 padding is removed, and a triangle that FESOM-C has stored as a quadrilateral with a repeated leading vertex is reduced to its three distinct vertices. A triangle on a mixed mesh is therefore interpolated by the triangular weights of Equation (5.4), bit-for-bit as on a pure FESOM2 mesh; only a genuine four-vertex element takes the quadrilateral path.

For a quadrilateral the weights are the Wachspress generalized barycentric coordinates (Warren, 1996; Floater, 2003), the natural analogue of Equation (5.3) for a convex polygon (Figure 5.4). With the four vertices $u_1, \ldots, u_4$ in cyclic order, the particle p, and the doubled signed areas

$$A_k = 2\,\mathrm{area}(p, u_k, u_{k+1}), \qquad C_k = 2\,\mathrm{area}(u_{k-1}, u_k, u_{k+1}), \tag{5.9}$$

the classical weight is $C_k/(A_{k-1}A_k)$, normalized over the vertices. The engine uses its *product form*, the classical weight multiplied by $A_1A_2A_3A_4$,

$$w_k = \frac{C_k\, A_{k+1}\, A_{k+2}}{\sum_j C_j\, A_{j+1}\, A_{j+2}}, \tag{5.10}$$

which contains no division by an area that vanishes on an edge: on an edge it reduces exactly to linear interpolation along that edge and at a vertex to that vertex's indicator. Each term carries three signed areas, so a clockwise element negates every term and the normalization cancels it; no orientation handling is needed. The coordinates reproduce a linear field exactly, are smooth in the interior and give every vertex a non-zero weight.

The formula needs no guard and has no fall-back because quadrilaterals are required to be convex: the FESOM-C reader, the builder of structured-grid meshes and the HEALPix builders check every quadrilateral when the mesh is loaded and name the offending elements if one is not convex.

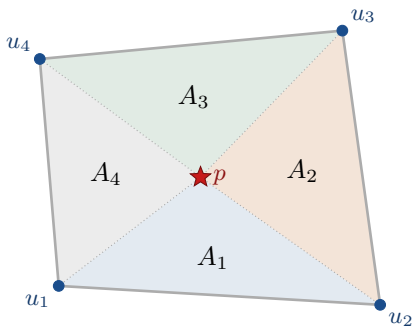


Figure 5.4: Wachspress interpolation on a quadrilateral element. The drifter p splits the element into four fan triangles of doubled area $A_1, \ldots, A_4$. The weight at vertex k is proportional to $C_k A_{k+1} A_{k+2}$, where C_k is the doubled area of the corner triangle (u_{k-1}, u_k, u_{k+1}): the two fan triangles *not* adjacent to vertex k grow as p approaches it.

The *containment* test of Section 4.5, which decides whether a particle lies inside a quadrilateral, uses a fan of triangles — never the Wachspress construction, which enters only the weight stage after containment has been established. The same vertex-count branch governs the `barycentric` option of the vertical-velocity combine (Section 6.6) and of scalar sampling (Chapter 7); the `nearest`, `idw` and `leastsq` options of those routines are independent of element shape. Element areas enter only when an element-layout vertex value is formed as an area-weighted average (Section 5.2). They are read from the model's mesh file when it provides them (FESOM2 mesh diagnostics do; ASCII meshes and regional extracts do not); otherwise they are computed on the sphere as the spherical excess of the element's fan of triangles times the square of the mesh radius. Because they appear only as ratios, their absolute scale is immaterial.

5.6 Common properties

Three dimensions. In a three-dimensional run the horizontal velocity is, by default, piecewise constant in the vertical: each particle reads the layer found in Section 6.3, and the scheme is applied to that layer's field without vertical interpolation. With a depth-linear field — `mode.uv_vertical: linear`, or FESOM 1.4 velocities on the levels — the order is reversed for each member of the stencil: every vertex or element is first evaluated in depth, in its own column, at the particle depth (Equation (6.4)), and the scheme then combines these values horizontally with its unchanged weights, plane fit or ghosts. A member whose sea floor lies above the particle is treated as a wall, the way each scheme treats the coast: the barycentric schemes leave it out and renormalise the remaining weights, the least-squares scheme leaves it out of the plane fit, and the edge least-squares scheme replaces it by the ghost of Equation (5.8) (Section 6.4, Table 6.3). With every member wet the arithmetic is exactly that of the layer scheme. The depth-linear rule is served by `barycentric` on node layout and by all three schemes on element layout.

Time. Between two snapshots the time-aware variants blend the two fields in time for each element or vertex they read, before the unchanged spatial formula (Chapters 10 and 11); with a depth-linear field the order is time, then depth, then the horizontal combination.

Failure. A scheme that cannot interpolate — a singular fit, no containing element, a degenerate triangle — reports failure; the particle's step is undone, its status set to `OUTSIDE` and its failure reason to an interpolation failure (Chapter 13).

Element tracking. With `elem_search: walk` (Section 4.6) the barycentric and edge least-squares schemes receive the containing element from the walk and skip their own search; the plain least-squares scheme does not use it.

Source. The weight dispatcher is `_polygon_weights`; the triangular and Wachspress weights are `_barycentric_weights_tri` and `_wachspress_weights_quad`; the convexity check is `validate_quad_convexity` with `find_nonconvex_quads`; element areas are `build_elem_areas`; all in `core/mesh.py`. The vertex compaction is in `_interp_barycentric_core` (`numerics/interpolate.py`). The public interpolators `interp_barycentric`, `interp_leastsq` and `interp_edge_leastsq` hand a depth-linear field to their twins `_interp_barycentric_dl` (both layouts), `_interp_leastsq_dl` and `_interp_edge_leastsq_dl` in the same file, which read every particle at its own depth and ignore the layer argument.

Code vs. documentation. The projections use three radii: the barycentric horizontal-velocity kernel uses the mesh radius, the containment test and the vertical-velocity and scalar barycentric kernels use 6 371 000 m, and the least-squares family uses 111 120 m per degree. The weights are ratios of areas in the projected plane and the least-squares value is the fitted constant, so the interpolated values do not depend on the radius; we record the difference only for fidelity to the source.

Chapter 6

Vertical Structure and Vertical-Velocity Interpolation

In a three-dimensional run the particle also moves vertically, driven by the model vertical velocity W. This chapter describes the sign convention of W and how each reader establishes it, how the depth coordinate is organized for the two vertical-coordinate families, how the layer containing a particle is found, how the horizontal velocity is read in depth — per layer or linearly between its data points — how W is interpolated to the particle in the vertical and then in the horizontal, how a step advances the depth on a terrain-following mesh, and how a particle is held within the water column. The two families are the fixed depth levels (z-levels) of FESOM2, FESOM 1.4 and the HEALPix products, and the terrain-following sigma coordinate of FESOM-C; they are implemented by *separate* routines throughout, and we give each explicitly. Depth z and vertical velocity W are positive downward (Section 1.5). On a z-level mesh the vertical position update is

$$z_{n+1} = z_n + W\,\Delta t, \tag{6.1}$$

which deepens a particle wherever the flow is downwelling; on a sigma mesh the step advances the particle's fraction of the water column instead (Section 6.7).

6.1 The sign of the vertical velocity

Inside the engine the vertical velocity is the positive-down rate of the tracked depth. Ocean models write the opposite sign, and every reader converts once, when a snapshot is read (Table 6.1). The convention of a file is resolved in this order: the configuration key `mode.w_positive` when it is `up` or `down`; otherwise the variable's CF `positive` attribute; otherwise a CF `standard_name` beginning with `upward` or `downward`; otherwise the default of the reader's producer. The default, `auto`, therefore needs no setting for model output. For a FESOM2 dataset without a vertical velocity, the tool `utils/reconstruct_w.py` reconstructs one offline from u, v and the sea-surface height, by integrating the horizontal divergence down the column; it writes the result positive down, with the attribute that says so.

> *Code vs. documentation.* Engine versions before September 2026 applied no conversion, so their three-dimensional FESOM2 and HEALPix runs moved particles vertically in the wrong direction. The sign is now tested against the equatorial upwelling in real model output. A vertical-velocity file written by the offline reconstruction before that date carries no attribute and is read with `w_positive: down`.

The zstar coordinate of FESOM2. FESOM2's interfaces move with the sea surface, and its `w` is the flow through them. The engine integrates it on the fixed reference levels, where

producer (reader)	variable	file convention	conversion
FESOM2 (`fesom2`)	`w`	positive up; in the zstar coordinate the velocity *through* the moving interfaces	negated
HEALPix products (`healpix`)	`wo`, `avg_wo`	positive up ("upward sea water velocity")	negated
FESOM-C (`fesomc`)	`w`	positive up; the flux through the sigma interfaces per unit area, zero at the surface and the bottom	negated
offline reconstruction	`w`	positive down, attribute `positive = "down"`	none

Table 6.1: Vertical-velocity conventions of the supported producers and the conversion each reader applies. The FESOM-C diagnostic `w_cart` is refused: its flux term lacks the dia-sigma part of the Cartesian vertical velocity.

the horizontal velocities also live, which drops the factor $H/(H+\eta)$ of the zstar equation: a relative error η/H in the vertical rate, about 10^{-4} in the deep ocean and 1 % to 2 % on a shelf 50 m to 100 m deep. The recorded depth is the zstar (reference) depth, which differs from the physical depth by $\eta\,(1 - z/H)$, at most $|\eta|$.

Source. `resolve_w_sign` in `readers/__init__.py`, applied by the FESOM2, HEALPix and FESOM-C readers; the key `mode.w_positive` is validated in `core/config.py`.

6.2 Vertical coordinate: z-levels and sigma

The two families are distinguished only by how an interface depth is obtained, but the difference propagates through every routine below. For the *z-level* family the interface depths are a fixed array $z_0 = 0 < z_1 < \cdots < z_{K-1}$, the same at every horizontal location; the per-node sea floor is the separate array `depth`. FESOM2 meshes hold the levels in the mesh file. FESOM 1.4 meshes take them from `nod3d.out` or from the depth coordinate of level-wise data, and their per-node floor is the depth of each column's last wet level (Section 3.7). A HEALPix product takes them from `mesh.levels` — the path of a level table or an inline list of depths, required to be strictly increasing — or from the table a regional extract embeds; its per-node floor comes from a bathymetry variable when the store has one, otherwise from the deepest level with finite data. For the terrain-following *sigma* family of FESOM-C the interface depths are reconstructed per water column from a fixed set of fractions σ_k and the local column thickness H,

$$H = \max(d + \eta,\, D_{\min}), \qquad z_k = (1 - \sigma_k)\, H, \tag{6.2}$$

where d is the still-water depth, η the sea-surface height, and $D_{\min}$ a configurable minimum thickness (default 0.5 m). With this convention $\sigma = 1$ maps to $z = 0$ (the surface) and $\sigma = 0$ maps to $z = H$ (the floor). The key `mode.max_levels` keeps the top `max_levels` interfaces of either family.

Code vs. documentation. The FESOM2 NetCDF and Zarr loaders store the z-levels positive down; the `mesh.z_levels` property takes the absolute value once more. Three-dimensional runs on FESOM2's legacy ASCII mesh format are unsupported: its level reader takes the whole `aux3d.out` file, node depths included, as the level table. (The FESOM 1.4 reader reads its own, different `aux3d.out` format correctly, Section 3.7.) For sigma meshes the orientation $\sigma = 1$ (surface) to $\sigma = 0$ (floor) is assumed by Equation (6.2) but is *not* validated when the mesh is read: a file written with the opposite ordering would silently invert the column.

On z-level meshes η does not enter the vertical path at all: the layer search, the vertical-velocity profile and the depth clamp depend only on the fixed levels and the per-node floor. On sigma meshes η sets the column thickness everywhere, and the column that matters is the one *at the particle*: every routine below uses H_p, the column thickness interpolated to the particle position by the same horizontal method as the vertical velocity (Section 6.6).

placement	rows of u, v	data points p_k (positive down)	set by
`"layer"`	$K-1$	none: one value per layer	every reader; the default
`"layer_linear"`	$K-1$, node or element layout	the layer centres $\frac{1}{2}(z_k + z_{k+1})$; on a sigma mesh $1 - \frac{1}{2}(\sigma_k + \sigma_{k+1})$, in the sigma fraction	`mode.uv_vertical: linear`; the tracking layer marks every field it reads
`"level"`	K, node layout	the levels z_k	the FESOM 1.4 reader; the option is refused there

Table 6.2: The three placements of the horizontal velocity in depth. Only `"layer"` is piecewise constant; the other two are read by the depth-linear rule of this section.

6.3 Finding the layer

Given a particle depth z, the engine identifies the layer that contains it, and the two families do this differently. On a z-level mesh the search is over the fixed interfaces: a particle at or above z_0 is assigned to the top layer, and otherwise the layer is the index k with $z_k \le z < z_{k+1}$. The search is bounded below by the local floor through a specific rule — the deepest admissible layer is $\max(k^* - 1, 0)$, where k^* is the first index whose lower interface z_{k+1} exceeds the per-node bottom depth — so a particle is never placed in a layer that lies below its local sea floor. On a sigma mesh the search is over the interfaces $z_k = (1 - \sigma_k)H_p$ of the particle's own column; its deepest layer is simply $K - 2$, bounded by the bottom interface, with no analogue of the z-level step-back. This asymmetry is a genuine difference in behaviour between the two families at the deepest layer, not merely a difference in formulae.

Source. Layer location is `find_layer`/`find_layers` (`numerics/vertical.py`, z-level) and `find_layer_sigma`/`find_layers_sigma` (`numerics/vertical_sigma.py`, sigma, with the particle column passed as `H_particle`); the per-particle layer index is assembled by `_compute_layers` (`numerics/integrators.py`).

6.4 Horizontal velocity in depth: layers or linear

The horizontal velocity of a three-dimensional run is stored at a few discrete depths per column, and the engine offers two rules for reading it in between, selected by `mode.uv_vertical`. Most producers write u and v once per layer, at the layer centre: the FESOM2 node fields `unod`/`vnod` and element fields `u`/`v`, the HEALPix `uo`/`vo`, and the FESOM-C `u`/`v` on its sigma layers, $K - 1$ rows each. Under the default, `layer`, the velocity is *piecewise constant in depth*: every member of the horizontal stencil of Chapter 5 — a vertex or an element — reads the row of the particle's layer, found by the rule of Section 6.3 in the nearest node's column (on a sigma mesh, in the particle's own column), so a particle that crosses a level sees the velocity jump. Under `linear` the same data are read *linearly in depth* between the layer centres, and every member is read in its own column. FESOM 1.4 writes u and v on the levels themselves, and its fields are always read linearly in depth. Inside the engine the rule is a property of the field, its *placement* `VelocityField.uv_vertical`, with the three values of Table 6.2. This section gives the rule, the treatment of a stencil member whose sea floor lies above the particle, its form on sigma meshes and for FESOM 1.4, and the combinations the engine refuses.

The data-point table. For a depth-linear field the engine builds, once per mesh, placement and layout, a table of the data points $p_0 \le p_1 \le \cdots \le p_{M-1}$ ($M = K$ for levels, $M = K - 1$ for layer centres) and, for every *column* c — a node for node-layout fields, an element for element-layout fields — two numbers: the index ℓ_c of its *last usable point*, the deepest point it

may be read at, and its *bed* d_c, the physical depth of its sea floor. For a node i,

$$\ell_i = \max\Big(0,\ \min\big(M-1,\ \max(\{k : l_k \le d_i\} \cup \{0\}),\ D_i\big)\Big), \tag{6.3}$$

where $l_k = p_k$ for level data and $l_k = z_{k+1}$, the layer's *lower* level, for layer data; d_i is the node's bed `mesh.depth`, and D_i an optional declaration by the reader of the last finite point of its own data column (`mesh.metadata["uv_data_last"]`). For layer data this is the rule of the layer search (Section 6.3): a *partial bottom layer*, one whose lower level lies below the bed, is never read. The HEALPix reader declares where each column's u data end when the mesh is loaded for a run with the option, and the FESOM 1.4 reader declares each column's last wet level; $D_i = -1$ means "the surface only", and the surface value must then be finite. A NaN bed matches no level, which gives $\ell_i = 0$; an infinite bed follows the formula as written ($+\infty$ keeps every point, $-\infty$ the surface point only). For an element e the node depths cannot tell where its water ends — FESOM2 fixes the number of levels of each element first and gives a node the deepest of its elements — so the element table takes both numbers from the reader: $\ell_e = \max(0, \min(M-1, D_e))$ with D_e the element's last wet layer (`uv_data_last_elem`), and d_e its bed (`elem_bed`), which the FESOM2 reader derives from the mesh diagnostic (Section 3.5). A `mode.max_levels` cut caps every ℓ_c at the last kept point and leaves the bed unchanged. Points below ℓ_c are *never read*, not even multiplied by zero, so dry or fill values stored there are harmless.

Source. The table is `depth_points` with the frozen dataclass `DepthPoints` (fields `p`, `last`, `bed`) and the builder `_build_depth_points`, in `numerics/vertical_uv.py`. It is cached on the mesh per placement and layout and rebuilt whenever `mesh.sigma`, the bed array or the declaration is replaced; it is built from their values and keeps its own arrays, so a write into one of them in place after the first use is not seen (assign a new array).

Bracket and blend. At a column c and particle depth z the depth is first clipped into the column's usable range, the bracketing points $p_k \le z_c < p_{k+1}$ (with $k+1 \le \ell_c$) are found, and the stored values f (u or v) are blended:

$$z_c = \max(p_0,\ \min(z,\ p_{\ell_c})), \qquad f_c(z) = (1-\alpha)\, f_{k,c} + \alpha\, f_{k+1,c}, \qquad \alpha = \frac{z_c - p_k}{p_{k+1} - p_k}. \tag{6.4}$$

Above the first point and below the last usable point the value of that point is used: the ends are constant, and nothing is extrapolated. A depth exactly on a point, at either end, or in a degenerate interval (a spacing below 10^{-12}) reads the point directly, without touching its neighbour; of two coincident points the deeper one is read, except at the surface end, where a depth at or above p_0 reads point 0. Single-precision data are widened to double precision before the blend. Because the bracket depends only on the column's own ℓ_c, a member whose last usable point lies above the particle — a partial bottom layer, a `max_levels` cut, a declared prefix that ends above the bed — contributes the value of that point, provided it holds water at z (Figure 6.1).

Source. `uv_bracket` returns (k, α), with $\alpha < 0$ meaning "read point k directly", and `uv_at` applies Equation (6.4); both are compiled kernels in `numerics/vertical_uv.py`.

Walls at depth. A column is *wet* at the particle depth when $z \le d_c$, with d_c the node's `mesh.depth` — the bed the end-of-step clamp of Section 6.8 uses — or the element's `elem_bed`. A NaN particle depth is read as $+\infty$, the layer path's reading of a NaN depth, and is therefore dry wherever the bed is finite. A member whose bed lies above the particle is *dry*, and it is treated as a wall, like the coast: it is left out of the interpolation. We emphasize that this is neither a zero velocity, which would brake every particle travelling along the wall, nor the dry member's bottom value read below its bed, which could accelerate it. Each interpolator applies

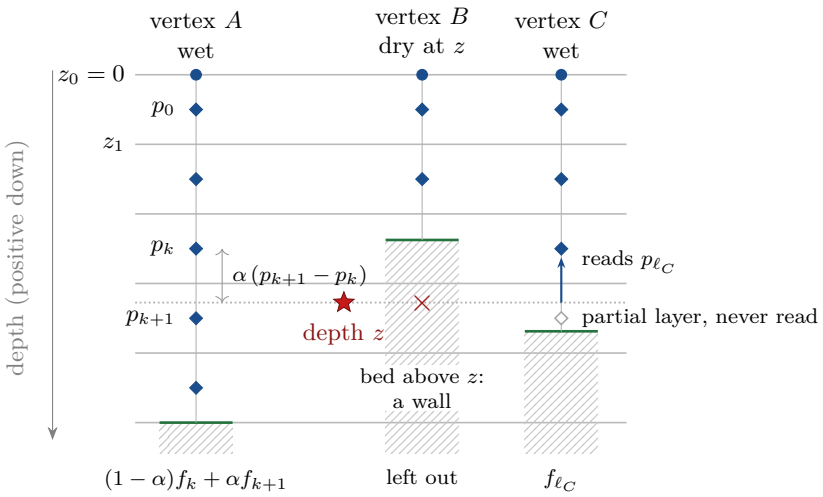


Figure 6.1: The depth-linear rule at one particle (star, depth z) whose containing triangle has the vertices A, B, C, drawn as three columns side by side. Diamonds are the data points — here layer centres between the levels (grey lines) — down to each column's last usable point p_{ℓ_c}; green segments are the beds. Vertex A is wet at z and blends its two bracketing points. The bed of B lies above z, so B is dry there: a wall, left out of the sum. Vertex C is wet, but its bottom layer is cut by its bed (a partial layer, open diamond, never read), so it contributes the value of its last usable point. The node-layout result is $U = s_{\mathrm{fb}}(w_A^+ u_A + w_C^+ u_C)/(w_A^+ + w_C^+)$ (Equation (6.5)); for element layout the columns are the elements of the stencil, under the same rule.

the rule the way it treats the coast (Table 6.3). The node-layout barycentric scheme leaves a dry vertex out of the sum of Equation (5.5) and renormalises the weights of the wet vertices,

$$U = s_{\mathrm{fb}} \frac{\sum_{i \text{ wet}} w_i^+ \, u_i(z)}{\sum_{i \text{ wet}} w_i^+}, \qquad w_i^+ = \max(w_i, 0), \tag{6.5}$$

where a negative weight — a particle just outside an edge, which the containment test accepts down to -10^{-6} — is counted as zero, so the result stays a convex blend of the wet vertex values; V is formed alike. The least-squares scheme leaves a dry element out of the plane fit Equation (5.6) over the nearest node's ring, as the elements beyond a coast are absent from a coastal node's ring. The edge least-squares scheme makes the edge to a dry neighbour a boundary edge, whose ghost Equation (5.8) is built from the own element's value at z, mirrored (free slip) or negated (no slip); with no slip a wall at depth therefore brakes the flow as the coast does. The element-layout barycentric scheme leaves a dry element out of each vertex's area-weighted mean, and a vertex without a wet element out of the sum, the weights of the others renormalised as in Equation (6.5).

When every member is wet, the result is exactly the layer interpolator's arithmetic — metric, summation order and singular guard — applied to the depth-linear values $f_c(z)$; a fit that is singular with every member wet, such as a coastal ring, still fails the particle as in layer mode. Fallbacks apply only when at least one member is dry, and only in three situations. First, when the wet members cannot support the interpolator, the least-squares scheme (fewer than three wet elements, or a singular fit over them — one or two wet elements in a narrow trench, where the fit would otherwise fail and freeze the particle) and the edge least-squares scheme (a singular fit over the own element and its wet neighbours) take the area-weighted mean of the wet members, $\sum_e A_e f_e(z) / \sum_e A_e$, with the element areas of Section 5.5; the barycentric scheme, in either layout, takes the equal-weight mean of the remaining vertex values when their weights sum below 10^{-12} (the particle on a left-out vertex). Second, when no member is wet — possible, because the end-of-step clamp uses the nearest node's bed only — the node-layout scheme reads the vertex with the largest bed (the first on ties) and the element schemes read

interpolator	a dry member	wet members insufficient	no member wet
`barycentric`, node layout	vertex left out; wet weights renormalised, negative ones as zero	weights sum $< 10^{-12}$: equal-weight mean of the wet vertices	the vertex with the largest bed, at p_ℓ
`leastsq`	element left out of the ring fit	fewer than three wet, or a singular fit: area-weighted mean of the wet elements	the containing element (searched), at p_ℓ
`edge_leastsq`	its edge a boundary edge with the `edge_leastsq_bc` ghost	a singular fit: area-weighted mean of the wet members; the own element dry: of its wet neighbours	the containing element, at p_ℓ
`barycentric`, element layout	element left out of each vertex mean; a vertex without one left out	weights sum $< 10^{-12}$: equal-weight mean	the containing element, at p_ℓ

Table 6.3: The wall rule and its fallbacks per interpolator. With every member wet none of the three right-hand columns applies and the result is the layer interpolator's arithmetic on the depth-linear values.

the containing element, each at its last usable point; the least-squares scheme, whose ring does not need the containing element, searches for it in this case only — a particle outside the mesh fails, and a containing element that the search finds outside the ring and wet at z is read at z. Third, when the containing element of the edge least-squares scheme is itself dry, the scheme takes the area-weighted mean of its wet edge neighbours, without ghosts, and with none wet the containing element at its last usable point.

The velocity therefore jumps where a particle crosses the depth of a neighbour's bed, because that member leaves or enters the stencil — the analogue of the jump at a layer interface in layer mode. Near the bed the two rules differ by more than the vertical interpolation. Layer mode reads every member in the row of the nearest node's layer, a vertex or an element shallower than that layer included; FESOM2 files store zeros there, so a shallow neighbour pulls the velocity — for the least-squares scheme, the fitted plane — towards zero. Linear mode leaves that member out. For instance, a uniform flow along a trench, below the bed of the shelf beside it, keeps its speed in linear mode and is slowed in layer mode (Section 16.3).

Time and stages. In a time-interpolated run every stored value is first blended in time (linear or PCHIP, Chapter 10), then read in depth by Equation (6.4), and then combined horizontally. PCHIP is not linear, so this order is part of the contract: the fused kernels follow it and agree bit for bit with the unfused path, which materializes the time blend first. A dry member is never read, not even blended in time. Every Runge–Kutta stage reads the horizontal velocity at its own stage depth — Euler at the step-entry depth, RK2 at the entry and the midpoint depth, RK4 at its four stage depths — before any clamp: a stage below a member's bed meets that wall, a stage below every bed of its stencil takes the no-member-wet fallback, and a stage above the surface reads p_0 everywhere.

Source. The node-layout core is `_interp_barycentric_core_dl` with the wall rule `_dl_node_wall`; the element-layout cores are `_interp_leastsq_core_dl`, `_interp_edge_leastsq_core_dl` and `_interp_bary_elem_core_dl`, which share the fallback `_dl_elem_wet_mean`; the wet test is `uv_wet` (`numerics/vertical_uv.py`). The fused twins are `_fused_euler_step_3d_dl` (element layout, least squares, Euler) and `_fused_rk2_step_3d_bary_tinterp_dl` and `_fused_rk2_step_3d_bary_pchip_dl` (node layout, RK2, linear and PCHIP in time), in `numerics/integrators.py`, with the time-blend vertex cores `_interp_barycentric_core_tinterp_dl` and `_interp_barycentric_core_pchip4_dl`. Element-layout RK2 and RK4 run unfused, as in layer mode.

Sigma meshes. A sigma column has the same layers everywhere, stretched to its own height, so on a FESOM-C mesh the coordinate of the rule is the particle's *sigma fraction* rather than its depth — the fraction of Equation (6.9), written s here as in the code — and the data points are the layer centres in that fraction:

$$s = \min\big(\max(z/H_p,\, 0),\, 1\big), \qquad p_k = 1 - \tfrac{1}{2}(\sigma_k + \sigma_{k+1}), \quad k = 0, \ldots, K-2, \tag{6.6}$$

with the interfaces at $s_k = 1 - \sigma_k$ (0 at the surface, 1 at the bed). Only `"layer_linear"` data are served, on either layout. Every layer of every column holds water, so the table reads no bathymetry, no declaration and no element bottom: $\ell_c = M - 1$ and $d_c = +\infty$ for every column. Every member is therefore wet at every s; there are no walls at depth and none of their fallbacks, and the record check below requires every layer of every column to be finite. Every member is read at the particle's s, that is, along the sigma surface through the particle — as the vertical velocity is (Figure 6.3) and as layer mode reads every member in the row of the particle's sigma layer. Each Runge–Kutta stage takes its s from its own position, time and column: the integrators hand the interpolator $s = \min(\max(z_{\text{stage}}/H_p(\mathbf{x}_{\text{stage}}, t_{\text{stage}}), 0), 1)$, with H_p interpolated at the stage position from the stage's surface elevation (Euler at the entry, RK2 at the entry and the midpoint, RK4 at all four stages), and keep metres for the vertical velocity, its layers and the depth clamp. A NaN fraction is read as $+\infty$, that is, the bottom centre, and a `max_levels` cut keeps the first interfaces from the surface, so every column reads down to the last kept centre. On a sigma mesh "exact" means exact for a field linear in s. On a flat bed that is also linear in z; on a sloping bed, however, a field linear in z is not claimed, since sigma data cannot represent it exactly under either rule.

Source. The sigma branch of `_build_depth_points` (`numerics/vertical_uv.py`); the stage fraction is `_sigma_frac_for_uv` with the stage column `_sigma_col_H`, in the sigma branches of the unfused steps of `numerics/integrators.py`. The sigma path stays unfused, as in layer mode (Chapter 11).

FESOM 1.4: velocities on the levels. FESOM 1.4 is a P1 finite-element model: u, v, W, temperature and salinity share the same three-dimensional nodes, which sit on the levels, and vary linearly over the tetrahedra of its three-dimensional mesh. Its reader returns u and v with K rows, on the levels, in node layout and with the placement `"level"`, so every three-dimensional run on FESOM 1.4 output is depth-linear between the levels, down to each column's last wet level, which the reader declares as D_i in Equation (6.3). The option `mode.uv_vertical: linear` is refused for it, with the advice to remove the option. The engine does not interpolate on the model's tetrahedra: it combines the barycentric weights of the surface triangle with the linear profile in depth of each vertex's own column. The two agree on the levels and along each node's column. Inside the prism between two levels they differ: the model's field is linear within each tetrahedron, whereas the engine's is barycentric at every depth and linear in depth at every position, so it holds products of the two. We therefore call it a *prism-wise approximation* of FESOM 1.4's own interpolation. The vertical velocity of FESOM 1.4 is also stored on the levels, which serve as the interfaces of Section 6.5.

The record check. Every record a run reads with a depth-linear field is checked once, when the run stores it: a non-finite u or v at a point $k \le \ell_c$ of any column stops the run, and the message names the component, the node or element, the point and the record. The time blends of Chapter 10, the interpolators and the step functions use the checked records and do not check again. Points below ℓ_c may hold anything. With the option the HEALPix reader checks the raw record before it zeroes missing values, so a surface mask that drifts between records is refused rather than zeroed.

Refusals. The option `linear` is refused for two-dimensional tracking (`Fdrift3D: false`); for a mesh that is neither z-level nor sigma; for an interpolator outside the layout's row — node layout takes `barycentric` only, element layout `leastsq`, `edge_leastsq` or `barycentric`; for the `rk45` integrator, which is two-dimensional only (the step's entry gate refuses a depth-linear field for it as well); for a reader whose u and v sit on the levels (FESOM 1.4); for element-layout data on a z-level mesh without element bottoms (a mesh diagnostic with neither `nlevels` nor `zbar_e_bottom`, an ASCII mesh directory, or a region extracted before the extraction tool kept them, Section 3.5); and for element-layout data on a mesh with ice-shelf cavities (`ulevels` > 1 anywhere). Ice-shelf cavities are not supported in the code yet and are planned. Every one of these refusals comes before the output file exists: the value of the key and the two-dimensional case when the configuration is loaded; the mesh type, the interpolator and the reader right after the mesh is loaded; the layout and the element bottoms at the metadata-only probe that precedes the writer (Section 5.3). A bad record is refused when it is read. A field already marked depth-linear that reaches a run with the option off is refused too, and so are the fields of one step whose placements differ. A run with the option on a HEALPix mesh loaded without it (no declaration) is refused at its first record: the mesh must be loaded with the run's mode configuration, as `run_tracking` and the command line do.

Source. The key is validated in `core/config.py`; `_check_uv_vertical_option` (after the mesh load), `_validate_uv_compat` and `_check_linear_element_mesh` (the pre-writer probe), and `_mark_depth_linear`, which marks every field the velocity cache stores, in `tracking.py`; `refuse_elem_cavities` and `check_depth_linear_record` in `numerics/vertical_uv.py`; the step-entry gate `_check_depth_linear_entry` in `numerics/integrators.py`.

Exactness. Where every member is wet the rule reproduces the layer answer where it should: data uniform in depth give the layer result bit for bit wherever every member reads a data point directly, and between points within a few units in the last place of the largest member value (2 for node-layout barycentric, 4 for element-layout barycentric, 8 for the two least-squares schemes; absolute bounds, since member values of opposite sign may cancel). A field linear in depth is reproduced exactly between the first and the last centre of the columns read, and a smooth profile converges at second order in the layer thickness, layer mode at first order (Section 16.3).

6.5 The vertical velocity profile

Within the bracketing interfaces the vertical velocity is interpolated linearly in *physical depth* (Figure 6.2), for both families: on a sigma mesh the bracketing interface depths are first reconstructed from Equation (6.2) and the interpolation fraction is then formed in metres. With

$$\alpha = \frac{z - z_k}{z_{k+1} - z_k} \in [0, 1], \tag{6.7}$$

the profile value at a single node is

$$W(z) = s_{\mathrm{fb}}\big[(1 - \alpha)\, W_k + \alpha\, W_{k+1}\big], \tag{6.8}$$

where W_k, W_{k+1} are the interface values and s_{fb} is the integration-direction factor. A particle at or above the surface interface takes the surface value and a particle at or below the deepest interface of the column takes the deepest value; no vertical extrapolation is performed, and a degenerate layer (thickness below 10^{-12} m) returns the upper-interface value. The local floor enters through the horizontal combination, which evaluates each neighbour no deeper than its own bottom (Section 6.6).

On z-level meshes the bracket (k, α) depends only on the depth, so the least-squares and inverse-distance combinations compute it once for the particle and reuse it at every neighbour

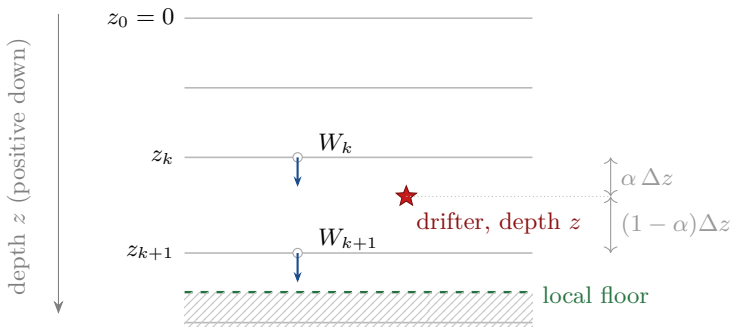


Figure 6.2: Vertical interpolation of the vertical velocity, identical in form for the z-level and sigma families. The interface values W_k, W_{k+1} (open circles; downward arrows mark the positive-down sign) bracket the drifter at depth z. The fraction $\alpha = (z - z_k)/(z_{k+1} - z_k)$ is formed in physical depth, and the profile value is $(1 - \alpha)W_k + \alpha W_{k+1}$. No extrapolation is performed beyond the ends of the column.

deep enough to contain the particle's depth; only shallower neighbours are bracketed again at their own bottom. The result is bit-identical to bracketing each neighbour separately and makes the vertical-velocity interpolation 1.1 to 1.4 times faster, the more so the more levels a column has.

Source. The single-node profiles are `_interp_w_linear_single` (z-level; the reusable bracket is `_w_bracket` with `_w_at_bracket`) and `_interp_w_sigma_linear_single` (sigma), in `numerics/vertical.py` and `numerics/vertical_sigma.py`.

6.6 Horizontal combination of the vertical velocity

The profile of Equation (6.8) gives the vertical velocity at a single node; to obtain it at the particle, the per-node profiles are combined horizontally over the one-ring of nodes surrounding the particle. The combination method is selected by `w_interp`: `nearest` (the profile at the nearest node alone), `leastsq` (a plane fit to the ring-node profiles, the default), `barycentric` (the vertex blend of Chapter 5 applied to the profiles), and `idw` (inverse-distance weighting). This least-squares stencil is over the surrounding *nodes*, in contrast to the horizontal velocity of Section 5.3, whose stencil is over the surrounding *element centres*; the two are distinct constructions that share a name. Only `barycentric` depends on the element type: on a quadrilateral it uses the Wachspress weights of Section 5.5, on a triangle the triangular weights. The choice also decides whether a three-dimensional run can use a fused kernel (Chapter 11).

z-level meshes. When a profile is sampled at a neighbouring node, the evaluation depth is capped at that neighbour's floor, $z_{\text{eval}} = \min(z, d_j)$, so that a shallow neighbour cannot contribute a spurious below-floor velocity. The `nearest` method evaluates the nearest node's own profile without a cap: an RK stage position below that node's floor reads the profile below the bottom, where the other methods' fallbacks cap the depth at the floor.

Sigma meshes. On a sloping bottom a fixed depth cuts across sigma surfaces, and sampling the neighbours at the particle's depth would mix the flow of different sigma levels. The neighbours are therefore evaluated *along the particle's sigma surface* (Figure 6.3): with the particle's fraction of its column

$$\xi = \min\big(\max(z/H_p,\, 0),\, 1\big), \tag{6.9}$$

neighbour j is evaluated at $z_j = \xi\, H_j$, $H_j = \max(d_j + \eta_j, D_{\min})$. The particle column H_p is interpolated by the same method — a least-squares plane (floored at $D_{\min}$), the barycentric

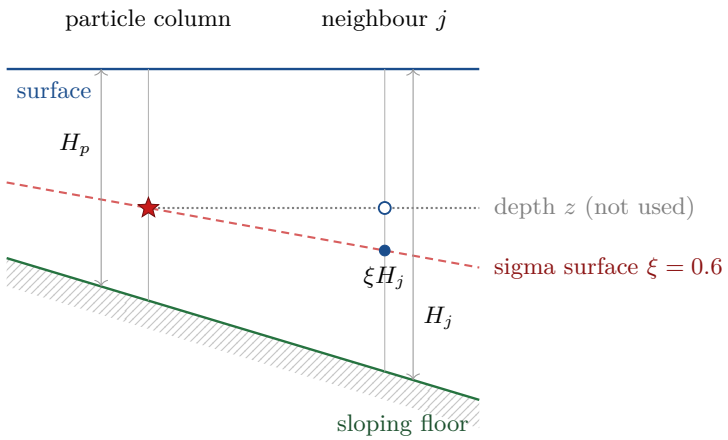


Figure 6.3: Sigma-following evaluation of the vertical velocity. The particle sits at the fraction $\xi = z/H_p$ of its column; the neighbouring column is evaluated at ξH_j on the same sigma surface (filled node), not at the particle's depth (open node), which on a sloping floor lies on a different sigma level.

weights of the containing element, or inverse-distance weights — which makes it exact for a linearly sloping bottom under `leastsq` and `barycentric`; `nearest` uses the nearest node's column.

Every method falls back to the nearest-node profile when its stencil is unusable, so the combined vertical velocity is never undefined.

Source. The horizontal combinations are `interp_w_spatial` (z-level) and `interp_w_sigma_spatial` (sigma), in `numerics/vertical.py` and `numerics/vertical_sigma.py`; the particle column is `sigma_column_height` with the per-method helpers `_H_leastsq_single`, `_H_bary_single` and `_H_idw_single`.

6.7 The step on a sigma mesh

FESOM-C's vertical velocity is a flux through the sigma interfaces, not the rate of the particle's depth: with the surface rising and falling and the floor sloping, a particle at rest on its sigma surface still changes depth. The step therefore integrates the particle's fraction of the water column,

$$\xi = \frac{z}{H_p}, \qquad \frac{d\xi}{dt} = \frac{W}{H_p}, \tag{6.10}$$

with W the negated file value of Table 6.1 (including the direction factor). With $\xi_0 = z_n/H_p(\mathbf{x}_0, t_0)$ and $f_k = W_k/H_{p,k}$ evaluated at stage k,

$$\text{Euler:} \quad \xi_{n+1} = \xi_0 + \Delta t\, f_1, \tag{6.11}$$

$$\text{RK2:} \quad \xi_{1/2} = \xi_0 + \tfrac{\Delta t}{2} f_1, \qquad \xi_{n+1} = \xi_0 + \Delta t\, f_2, \tag{6.12}$$

$$\text{RK4:} \quad \xi_{n+1} = \xi_0 + \tfrac{\Delta t}{6}\,(f_1 + 2f_2 + 2f_3 + f_4). \tag{6.13}$$

Every stage places the particle at $z = \xi\, H_p(\mathbf{x}_k, t_k)$ in the column at the stage position and time, and the end of the step at $z_{n+1} = \xi_{n+1}\, H_p(\mathbf{x}_{n+1}, t_{n+1})$. The surface elevation at the start and at the end of every step comes from one function of time — the PCHIP value where a four-snapshot stencil exists and PCHIP is enabled, otherwise the linear blend, and in discrete mode the next record — so the end of one step and the start of the next see the same surface bit for bit, ξ is continuous across steps, and a resumed run reproduces the uninterrupted one. With a linear time interpolant, a linear bottom and a uniform dia-sigma velocity the discrete solution is exact; with a dia-sigma velocity linear in ξ the three schemes converge at orders one, two and four. The layer of the horizontal velocity, the depth clamp and the diffusion kicks

(Chapter 12) use the same column H_p. The recorded vertical velocity on a sigma mesh is W itself, the dia-sigma flux, not the rate of the particle's depth.

Source. The sigma branches of `_euler_step_unfused`, `_rk2_step_unfused` and `_rk4_step_unfused` with the helpers `_sigma_col_H` and `_sigma_end_eta` (`numerics/integrators.py`); the surface at the step boundaries is `_eta_at_time` (`tracking.py`). Sigma meshes always take the unfused path (Chapter 11).

6.8 Depth clamping at the surface and the floor

After the vertical position is advanced, the depth is clamped into the admissible range, and the limits differ by family. On a z-level mesh the depth is clamped into $[\, z_0,\ d_{\mathrm{nn}} \,]$ — the surface interface and the per-node sea floor of the nearest node — and neither η nor any minimum-thickness parameter enters. On a sigma mesh the depth is clamped into $[\, 0,\ \max(H_p - z_{0b},\, 0) \,]$, with H_p the end-of-step column at the particle and z_{0b} (default 0.1 m) holding the particle a fixed distance above the bed. The clamp is applied at every step, so a particle that reaches the floor is not removed but tracks along it, in contrast to a particle that leaves the horizontal domain, which is flagged and removed (Chapter 13).

Code vs. documentation. The keys `Dmin` ($D_{\min}$) and `z0b_min` (z_{0b}) are consumed only on sigma meshes; the z-level clamp uses neither (nor η). z_{0b} is a geometric clamp offset, not a roughness length in any physics. The sigma surface clamp holds the particle at a literal $z = 0$ rather than at $(1 - \sigma_0)H$, so it coincides with the top interface only when $\sigma_0 = 1$ exactly.

Source. Depth clamping is `clamp_depth` (z-level, limits $[z_0, d_{\mathrm{nn}}]$) and `clamp_depth_sigma` (sigma, limits $[0, \max(H_p - z_{0b}, 0)]$) in the respective vertical modules, invoked by `_clamp_z` (`numerics/integrators.py`); the defaults $D_{\min} = 0.5\,\mathrm{m}$, $z_{0b} = 0.1\,\mathrm{m}$ are set in `core/config.py`.

Chapter 7

Scalar Field Sampling Along Trajectories

Beyond advecting particles, the engine can record the value of model scalar fields — temperature and salinity, for example — at each particle position as the trajectory unfolds. This sampling is a diagnostic: it does not affect the motion, but it attaches a time series of water properties to every trajectory. The fields to sample are listed in the configuration key `mode.sample_vars` (a single name is accepted as a one-element list; an empty list switches sampling off and leaves the output byte-identical), and the horizontal method in `numerics.sample_interp`. This chapter describes what is sampled, where in the column the value is taken, how the horizontal combination reuses the interpolation machinery of Chapters 5 and 6, and why sampling is deliberately independent of the integration direction.

7.1 What is sampled, and when

The sampled fields are node-based scalars, read from the same forcing dataset as the velocity. Each requested field is probe-read once at startup, before the output file is created; a field that is absent, or that is stored at element centres rather than nodes, raises a configuration error immediately rather than failing midway through a run. Each reader prepares the fields in its own way: the FESOM2 reader recognises mid-layer and interface fields by their vertical dimension and picks the tracked level in a two-dimensional run; the FESOM-C reader reduces sigma-layered fields to two dimensions with a node-aware reduction in a two-dimensional run; the HEALPix reader compacts pixels to nodes and fills each column downward from its deepest valid value, so that the bottom clamp reads bottom water, never a missing value (pixel geometry does not support sampling); the NEMO reader samples surface T-point fields only. Values are recorded at the trajectory output cadence (every `output_freq` steps, Chapter 15), not at every integration step. A record is labelled with the time at the end of its step (Equation (15.2)), and the scalar is taken at that instant as far as the forcing allows. When the velocity is reconstructed in time (Chapter 10), the scalar is re-interpolated to the exact record time, so the sampled value is consistent with the recorded position; this re-interpolation is linear between the bracketing snapshots even when the velocity uses the monotone-cubic reconstruction. In the discrete-time mode the scalar is read from the snapshot at the end of the step — the next snapshot, which the step already holds as the target of its Runge–Kutta stages — so its instant is the record time as well. Only when no next snapshot exists, on the last step of a run that stops at the end of the forcing, is the snapshot at the start of the step used, one snapshot behind the record time; the variable's comment attribute says so. (Before output format 2 the discrete mode always sampled the start-of-step snapshot.)

Each sampled field becomes a variable of the trajectory output with the dimensions of the

positions, filled with missing values before its first record and in any record that does not carry it, and carrying the units and long name of the source variable, the positions as its coordinates, and a comment on the sampling instant; names that collide with the output's own variables, the release variables included, are refused. Checkpoints do not store sampled values. A field read internally for beaching (Chapter 13) is written only when it is also listed in `sample_vars`. On a rotated mesh (Chapter 8) scalars are sampled in the rotated frame and written as they are, which is correct for scalars; a vector component listed as a sample stays in the rotated frame.

7.2 Vertical placement by field shape

The vertical treatment is selected by the shape of the field array, which encodes where the field lives in the column. A two-dimensional field, one value per node, is sampled horizontally only. A mid-layer field, with one value per layer per node, is interpolated linearly between the two bracketing mid-layer depths — the midpoints of the layers, computed from the fixed interface depths on a z-level mesh or from the column geometry of Equation (6.2) on a sigma mesh, and clamped to the first and last midpoint. On a z-level mesh each neighbour is evaluated no deeper than its own floor, as for the vertical velocity (Section 6.6). On a sigma mesh each neighbour column is evaluated at the particle's *depth*, capped at the column thickness, and not along the particle's sigma surface as the vertical velocity is; on a sloping bottom the sample therefore mixes neighbouring sigma levels, a limitation that remains to be removed. An interface field, with one value per interface per node, is handled by delegating to the vertical-velocity machinery of Sections 6.5 and 6.6, including its sigma-following evaluation. In that delegation the integration-direction factor is fixed to +1, for the reason given in Section 7.4.

7.3 Horizontal combination and element type

The horizontal combination offers the same four methods as the vertical velocity — `nearest`, `leastsq`, `barycentric`, and `idw` — and shares their behaviour with respect to the mesh. As with the vertical velocity, only the `barycentric` method depends on the element type, using the Wachspress weights of Section 5.5 on a quadrilateral and the triangular weights on a triangle; the other three combine surrounding node values irrespective of element shape. Each method falls back to the nearest node's value when its stencil is unusable. A particle that is not alive (including a beached or a not-yet-released one) and a particle without a nearest node are recorded as missing.

Model fields often carry fill values at coastal or below-bottom nodes, and a linear combination would propagate them. The fitting methods therefore skip non-finite neighbours: the least-squares fit uses the finite ones and needs at least three; inverse-distance weighting drops them; a barycentric element with a non-finite vertex falls back to the nearest node's value. Under `nearest`, and under `idw` for a particle that coincides with a node, the sample is that node's raw value, a fill value included, because the sample point then *is* that node; the least-squares fit has no such case and returns the plane through the finite neighbours even at a node. Interface fields, which go through the vertical-velocity machinery, are not filtered.

7.4 Independence from the integration direction

Sampling is a measurement of the instantaneous field and must not change sign when the run is integrated backward in time. The engine enforces this by fixing the direction factor to +1 in the sampling path, in contrast to the advection path, where the factor s_{fb} carries the forward or backward direction into the interpolated velocity (Section 2.5). The distinction matters specifically for an interface-located scalar, which is sampled through the vertical-velocity

machinery: without the override it would inherit the velocity's direction sign and report a sign-flipped value on a backward run. A sampled temperature is a temperature regardless of the direction of integration.

Source. Sampling is implemented in `sample_field` and the direction-free combine `_combine_horizontal` (`numerics/sampling.py`); the per-record sampling is driven by `_sample_record_fields` (`tracking.py`); each reader's `read_scalar_fields` prepares the fields and serves the startup probe.

Chapter 8

Coordinate Frame and Rotation

A global ocean mesh must contend with the convergence of the meridians at the geographic poles, where the longitude metric $z_0 = \cos\phi\, M_\phi$ of Section 1.5 vanishes and longitude becomes ill-conditioned. FESOM2 meshes commonly avoid this by rotating the computational frame so that its poles lie over land — conventionally over Greenland and Antarctica — leaving the entire ocean far from the coordinate singularity. This chapter describes how the engine supports such a rotated frame: the rotation itself, the distinction between rotating points and rotating vectors, what is and is not affected, the contract that keeps the rotation invisible to the user, and the accuracy it achieves. The facility is available on FESOM2 and HEALPix meshes; FESOM-C meshes and NEMO-derived meshes reject a non-null `mesh.rotation` with a configuration error.

8.1 The rotated frame

The rotation is configured by the key `mesh.rotation`. Its default is `null`, a geographic frame in which no rotation is applied and the trajectories are computed directly in longitude and latitude; a run with this default is byte-identical to one on an engine without the facility. Setting `mesh.rotation` to `fesom2` selects FESOM2's own Euler angles $(\alpha, \beta, \gamma) = (50^\circ, 15^\circ, -90^\circ)$, which place the computational poles in Greenland and in East Antarctica; an explicit triple of angles may also be given. The rotation is a rigid map of the sphere,

$$\mathbf{r}' = \mathbf{R}(\alpha, \beta, \gamma)\, \mathbf{r}, \qquad \mathbf{r} = (\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi), \tag{8.1}$$

with $\mathbf{R}$ the Euler rotation matrix and $\mathbf{R}^{\mathsf{T}}$ its inverse; rotated longitudes are wrapped to $[-180^\circ, 180^\circ)$. It is applied once when the mesh and each velocity snapshot are read, never inside the per-particle loop. With the `fesom2` angles the rotated FESOM2 mesh reproduces the model's own element areas to machine precision.

8.2 Rotating points and rotating vectors

The rotation acts differently on positions and on velocities, and the distinction is essential to correctness (Figure 8.1). Node coordinates and element centres are *points*: they are mapped by Equation (8.1). The element centres are rotated as points and not recomputed from the rotated nodes, because a centroid computed on the sphere does not commute with the rotation. The horizontal velocity components, by contrast, are the components of a *tangent vector* expressed in the local east–north frame, and that local frame itself turns from place to place; the velocity must therefore be re-expressed in the rotated frame's local east–north basis. With c and s the cosine and sine of the local angle through which the unit-east direction turns, the transformed components are

$$u' = c\,u - s\,v, \qquad v' = s\,u + c\,v, \tag{8.2}$$

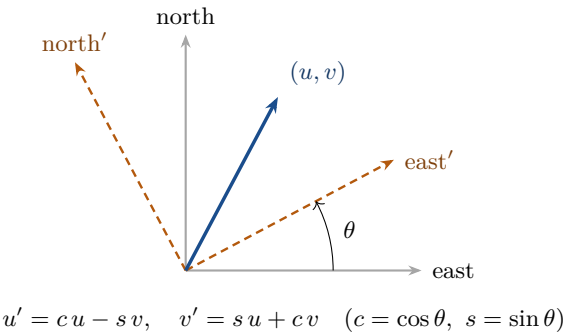


Figure 8.1: Rotation of a horizontal velocity into the rotated frame. The velocity (u, v) is a tangent vector; its components are re-expressed in the rotated local east–north basis (dashed), turned through the local angle θ, by Equation (8.2). The map preserves the speed. Node coordinates and element centres, being points rather than vectors, are instead mapped directly by the rotation.

a rotation that preserves the speed, $u'^2 + v'^2 = u^2 + v^2$. The factors (c, s) are the rotated-frame components of $\mathbf{R}\,\hat{\mathbf{e}}$, with $\hat{\mathbf{e}} = (-\sin\lambda, \cos\lambda, 0)$ the geographic unit east vector, computed once per velocity carrier — per node, per element centre, or both, as the mesh's velocity layout requires — from the *geographic* coordinates, before the points are rotated. At an exact pole the east direction is taken as the limit at the stored longitude.

Scalar and vertical quantities have no horizontal direction and pass through unchanged: the vertical velocity, the sea-surface height, the diffusivity and the sampled scalars. For the diffusivity one qualification applies: a diffusivity whose zonal and meridional values differ acts along the rotated axes rather than along east and north, and the engine warns once when it meets such a pair. The stochastic kicks move along great circles (Chapter 12), so an isotropic diffusivity gives the same statistics in either frame.

8.3 The user-facing contract

The rotation is arranged to be invisible outside the engine. Release positions are always supplied in geographic coordinates and are rotated into the computational frame when the run starts. Trajectory output is always geographic: at every record the positions and the horizontal velocity diagnostics are un-rotated on a copy of the state, while the internal state stays in the rotated frame; the vertical velocity and the sampled scalars are written as they are. The release positions written with the trajectories (Section 15.2) are geographic too: the particle state carries them in the rotated frame, checkpoints store them in that frame, and they are un-rotated by the same rule when they are written. A restart checkpoint records the three angles of the frame in which it was written, and a resuming driver validates them against the mesh (`_validate_restart_frame`, Chapter 15), so a run is not continued in a different frame by accident; the output file carries the angles as metadata whenever the frame is rotated. Within this contract the rotation is purely an internal device for conditioning the arithmetic near the poles. Periodic fold windows (Chapter 13) are defined in the mesh's coordinates and cannot be combined with a rotation; the combination is refused.

8.4 Rotation on HEALPix meshes

A FESOM2 model builds its mesh in its own rotated frame; a HEALPix product carries geographic east and north components on a global pixel grid. Rotation of a HEALPix mesh is therefore a pure runtime transform: node coordinates are rotated as points, the velocities receive tangent factors per node (per pixel centre on the pixel mesh), the search structures are

rebuilt on the rotated coordinates, and the tracking layer handles the rest as for FESOM2. The rotated frame has two poles of its own, and the polar pathologies move there, so the loader guards against them: a wet element that straddles a rotated pole is refused with an error that names where the poles land, and wet nodes poleward of 88° rotated latitude produce a warning. With the `fesom2` angles both poles fall on land, and on the global HEALPix products tested the wet nodes stay below about 85° rotated latitude. In the geographic frame, by contrast, the containment test rejects points poleward of the first ring of pixel centres, $\arcsin(1 - 1/(3\,n_{\mathrm{side}}^2))$ — 84.15° at $n_{\mathrm{side}} = 8$, 89.91° at $n_{\mathrm{side}} = 512$ — so a particle there dies as `OUTSIDE`; for global HEALPix runs the rotation is the recommended setting.

8.5 Accuracy and when rotation is needed

Rotation is a change of chart, not of physics, and away from the poles a correctly implemented run differs between the geographic and rotated frames only at the level of the discretization. In the engine's verification the two frames agree to about 10^{-6} degrees at mid-latitudes, and the difference shrinks as the step is refined: at second order in the step for element-layout velocity and at first order for node-layout velocity. Across a pole only the rotated frame can track: a meridional flow over the pole follows the analytic great circle to within 7×10^{-8} m, whereas the geographic frame breaks down there. A genuine framing error would produce discrepancies of tens of degrees, independent of the step. Over long integrations the two frames diverge like any two slightly different discretizations of a chaotic flow; their statistics, not their individual paths, are what agree.

Rotation becomes necessary only when the mesh carries wet points close to a geographic pole. The FESOM2 reader warns, and suggests the `fesom2` setting, when a mesh extends beyond about ±88° without a rotation, and warns about a node exactly at a pole when rotated; the FESOM-C reader warns without the suggestion; the HEALPix reader warns as described in Section 8.4. For regional domains away from the poles the geographic frame is appropriate and is the default.

Source. The maps are `euler_matrix`, `rotate_lonlat`/`unrotate_lonlat` and `rotate_vectors`/`unrotate_vectors` in `core/rotation.py`; the angles and vector factors are `rotation_angles` and `_vec_rot_factors` (`core/mesh.py`); the readers apply them in `_apply_mesh_rotation` (`readers/fesom2.py`) and `_apply_rotation` (`readers/healpix.py`); the tracking layer rotates seeds in `_maybe_rotate_seeds`, unrotates records in `_geo_view_for_record` and checks restarts in `_validate_restart_frame` (`tracking.py`).

Part IV

Time Integration

Chapter 9

Time-Integration Schemes

With the velocity available at an arbitrary position and time (Chapters 5 to 10), the engine advances the trajectory equation Equation (1.1) by an explicit time-integration scheme. Four are provided, selected by `numerics.integrator`: the first-order Euler method, the second-order midpoint Runge–Kutta method (RK2), the classical fourth-order Runge–Kutta method (RK4) (Butcher, 2016), and an adaptive Dormand–Prince 5(4) method (RK45) (Dormand and Prince, 1980) that subdivides each step to an error tolerance. This chapter gives the discrete position update common to all of them, the per-scheme stage formulae as implemented, what happens when a stage fails, and the way the integration direction enters. The interaction of scheme order with the temporal reconstruction — which can cap the achievable order — is the subject of Chapter 10.

9.1 The displacement metric

The velocity is in metres per second but the horizontal position is in degrees, so each update converts through the spherical metric of Section 1.5. A horizontal velocity (U, V) over a step Δt displaces the particle by

$$\Delta\lambda = \frac{\Delta t\, U}{z_0}, \qquad \Delta\phi = \frac{\Delta t\, V}{M_\phi}, \qquad \Delta z = W\,\Delta t, \tag{9.1}$$

with $M_\phi = 111\,120\,\mathrm{m}\,^{\circ -1}$ the metres per degree of latitude and $z_0 = \max(\cos\phi\, M_\phi, 1)$ the metres per degree of longitude. The lower bound of $1\,\mathrm{m}\,^{\circ -1}$ on z_0 is a guard against the cosine vanishing at the poles; longitude is wrapped to $[-180^\circ, 180^\circ)$ and latitude is held within $[-90^\circ, 90^\circ]$ after each update, for the stage positions and for the particles that moved — the positions of dead and not-yet-released particles are returned unchanged. The interpolated velocity already carries the integration-direction factor s_{fb} (Section 2.5), so the time step Δt in Equation (9.1) is always positive and the same formulae serve forward and backward integration.

9.2 The three schemes

Euler. The Euler step evaluates the velocity once, at the current position, and applies Equation (9.1) directly:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta\mathbf{x}\big(\mathbf{u}(\mathbf{x}_n, t_n)\big). \tag{9.2}$$

It is first-order accurate: the error per step is of order Δt^2.

RK2 (midpoint). The midpoint method evaluates the velocity twice. A first evaluation at the current position gives a provisional half-step displacement to a midpoint; the velocity is

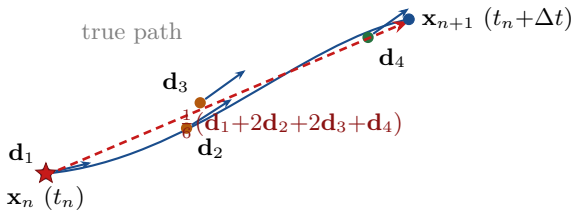


Figure 9.1: The four stages of RK4. The slope $\mathbf{d}_1$ is sampled at the start (t_n); $\mathbf{d}_2$ and $\mathbf{d}_3$ at provisional midpoints (time $t_n + \frac{1}{2}\Delta t$); $\mathbf{d}_4$ at a provisional endpoint $(t_n + \Delta t)$. The step is their weighted average Equation (9.5) (dashed), a fourth-order approximation to the true path (blue). Each stage requires the velocity at a different position *and* time, which is why the temporal reconstruction of Chapter 10 matters.

then evaluated at that midpoint, at the half-step time, and the full step uses this midpoint velocity:

$$\begin{aligned} (U_1, V_1) &= \mathbf{u}(\mathbf{x}_n, t_n), & \mathbf{x}_{\text{mid}} &= \mathbf{x}_n + \tfrac{1}{2}\Delta\mathbf{x}(U_1, V_1), \\ (U_2, V_2) &= \mathbf{u}(\mathbf{x}_{\text{mid}},\, t_n + \tfrac{1}{2}\Delta t), & \mathbf{x}_{n+1} &= \mathbf{x}_n + \Delta\mathbf{x}(U_2, V_2). \end{aligned} \tag{9.3}$$

The longitude metric of each displacement is taken at the latitude where its velocity was evaluated: $z_0(\phi_n)$ for the half step and $z_0(\phi_{\text{mid}})$ for the full step, $\Delta\lambda = \Delta t\, U_2 / z_0(\phi_{\text{mid}})$. With the metric of the starting latitude the longitude update would fall to first order; the scheme is second-order accurate.

RK4 (classical). The fourth-order method evaluates the velocity at four stages and combines their displacements with the classical weights $\frac{1}{6}(1, 2, 2, 1)$ (Figure 9.1). Writing the per-stage degree increment as $\mathbf{d}_k = \Delta\mathbf{x}(U_k, V_k)$, the stages are

$$\begin{aligned} \mathbf{d}_1 &= \Delta\mathbf{x}\big(\mathbf{u}(\mathbf{x}_n,\, t_n)\big), & \mathbf{d}_2 &= \Delta\mathbf{x}\big(\mathbf{u}(\mathbf{x}_n + \tfrac{1}{2}\mathbf{d}_1,\, t_n + \tfrac{1}{2}\Delta t)\big), \\ \mathbf{d}_3 &= \Delta\mathbf{x}\big(\mathbf{u}(\mathbf{x}_n + \tfrac{1}{2}\mathbf{d}_2,\, t_n + \tfrac{1}{2}\Delta t)\big), & \mathbf{d}_4 &= \Delta\mathbf{x}\big(\mathbf{u}(\mathbf{x}_n + \mathbf{d}_3,\, t_n + \Delta t)\big), \end{aligned} \tag{9.4}$$

and the position advances by

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \tfrac{1}{6}(\mathbf{d}_1 + 2\mathbf{d}_2 + 2\mathbf{d}_3 + \mathbf{d}_4). \tag{9.5}$$

The longitude metric z_0 is recomputed at each stage's latitude. RK4 is fourth-order accurate in space.

In three dimensions on a z-level mesh the depth is advanced by the same stage structure, with $\Delta z_k = W_k\, \Delta t$ (for RK2, $z_{\text{mid}} = z_n + W_1 \Delta t/2$ and $z_{n+1} = z_n + W_2 \Delta t$), without clamping between stages, and the depth is clamped to the column after the step (Section 6.8). On a sigma mesh the stages advance the particle's fraction of the water column instead (Section 6.7).

9.3 The adaptive scheme

With `integrator: rk45` each particle subdivides the tracking step Δt into substeps h chosen by an error estimate, using the Dormand–Prince 5(4) pair with the coefficients of MATLAB's `ode45`. The right-hand side is $(\dot\lambda, \dot\phi) = (U/z_0,\, V/M_\phi)$; each stage samples the velocity at its own time through the same nested temporal blend as the fused kernels (Section 11.3). The difference between the fifth- and fourth-order solutions is converted to metres,

$$e = \sqrt{(e_\lambda\, z_0)^2 + (e_\phi\, M_\phi)^2}, \tag{9.6}$$

and a substep is accepted when e does not exceed `rk45_tol_m` (default 1 m). After every attempt, accepted or rejected, the substep becomes

$$h \leftarrow h \cdot \min\big(\max\big(0.9\,(\mathrm{tol}/e)^{1/5},\, 0.2\big),\, 5\big), \tag{9.7}$$

the first attempt is the whole step, the last substep is clipped to end exactly at Δt, and the last stage of an accepted substep is reused as the first of the next. Apart from the last one, clipped to the end of the step, a substep is never smaller than `rk45_h_min` (default $10^{-4}\Delta t$): a substep at that floor is accepted even above the tolerance, and the run warns with the number of such substeps. A failed stage interpolation counts as a rejected attempt and retries at a fifth of the substep; only a failure at the floor, or more attempts than $\max(10^4,\ 2\,\Delta t/h_{\min} + 16)$, ends the particle.

The adaptive scheme has a single fused kernel and no unfused twin, and its first version is limited to two-dimensional runs with element-layout least-squares interpolation, without the monotone-cubic time reconstruction, element tracking, or spatially varying diffusivity; each excluded combination is refused before the output file is created. The tolerance, and the floor when it is set, are written to the output metadata.

9.4 Failure within a step

A stage whose interpolation fails — no fit, no containing element — ends the step for that particle: it is returned to its position at the start of the step, with its starting nearest node, its status is set to `OUTSIDE` and its failure reason to an interpolation failure. Stage positions are not tested for containment; only the final position is, and a final position outside every element fails the same way with the reason out of bounds (Chapter 13). The velocity recorded with the particle is that of the first stage — the velocity at the start of the step — for Euler, RK2 and RK4, and that at the end of the last substep for RK45.

9.5 Order, and what limits it

The nominal orders — one, two, and four — are spatial: they describe how the error falls as the step shrinks, provided the velocity is known exactly as a function of time. In practice the velocity is reconstructed between stored snapshots, and the accuracy of that reconstruction in time can become the binding constraint. As we show in Chapter 10, a linear-in-time reconstruction caps the time-truncation error at second order regardless of the spatial scheme, so that RK4 delivers no advantage over RK2 unless either the step is much smaller than the snapshot interval or a higher-order temporal reconstruction is used. The adaptive scheme controls the local error of the trajectory for the velocity field as reconstructed; it cannot recover accuracy that the reconstruction in time does not carry. We therefore defer any claim about realized accuracy to that chapter.

Source. The schemes are `euler_step`, `rk2_step`, `rk4_step` and `rk45_step` (kernel `_rk45_kernel_2d`, stage evaluation `_rk45_eval`), dispatched by `get_integrator`; the update equations Equations (9.1) to (9.5) are implemented in the kernels of `numerics/integrators.py`, with the metric in `core/geometry.py`.

Chapter 10

Temporal Reconstruction of the Velocity Field

The integration schemes of Chapter 9 require the velocity not only at arbitrary positions but at arbitrary times — at the half-step and full-step instants of the Runge–Kutta stages — whereas the model stores it only at discrete snapshots. The engine bridges this gap by reconstructing the velocity in time between snapshots. This chapter describes the two modes of operation (one snapshot per step, or sub-stepping with interpolation), the linear and monotone-cubic reconstructions, and the central consequence that the temporal reconstruction can cap the order of the spatial scheme. The handling of the snapshot times themselves — including the shift applied to time-averaged forcing — belongs to the forcing reader and is described in Chapter 14.

10.1 Discrete and interpolated modes

The key `temporal_interp` selects between two modes. In the *discrete* mode (the default), each integration step advances by one stored snapshot; the displacement uses the step `numerics.dt`, which must therefore equal the record spacing (taken from the file's time axis when it is not set; a different step, or irregular record times such as calendar months, are refused when the run starts), and the Runge–Kutta stages are still ordered in time, the next snapshot being supplied so that the stages see the correct time progression across the step. In the *interpolated* mode, the time step may be smaller than the snapshot interval, and the velocity at each stage time is reconstructed from the bracketing snapshots. Sub-stepping in the interpolated mode is the means by which a fine step can be taken against coarsely sampled forcing, for example a sub-daily step against daily-mean fields. The interpolated mode needs a physical time axis: a file without a decodable time coordinate is refused unless the snapshot spacing is given as `numerics.file_dt`.

10.2 Linear reconstruction

The default reconstruction is linear in time. For a time t falling between snapshots at t_a and t_b carrying velocity fields $\mathbf{u}_a$ and $\mathbf{u}_b$, the reconstructed field is

$$\mathbf{u}(t) = (1-\alpha)\,\mathbf{u}_a + \alpha\,\mathbf{u}_b, \qquad \alpha = \frac{t-t_a}{t_b-t_a}. \tag{10.1}$$

A step starting at t between two snapshots receives the field at its start, with $\alpha_0 = (t-t_a)/(t_b-t_a)$, and a *target* snapshot $\mathbf{u}_T$ toward which its stages move: the next snapshot, t_b, when integrating forward, and the previous one, t_a, when integrating backward; a backward step that starts

exactly on a snapshot brackets downward, so it has a real target rather than none. The stage with time fraction c_k (Section 9.2) samples

$$\mathbf{u}_k = (1-\alpha_k)\,\mathbf{u}(t) + \alpha_k\,\mathbf{u}_T, \qquad \alpha_k = \frac{c_k\,\Delta t}{\Delta t_T}, \tag{10.2}$$

with Δt_T the time from the step's start to the target snapshot; Equation (10.2) is Equation (10.1) evaluated at $t \pm c_k \Delta t$. The engine never forms $\mathbf{u}(t)$ as a whole field unless a consumer asks for it: the velocity handed to a step between two snapshots is a lazy view that blends a field on its first use, and the fused kernels blend each element or vertex they read (Equation (11.1)).

A step longer than the time to its target snapshot, $\Delta t > \Delta t_T$, would place its later stages beyond the bracketing pair. Such a step runs on a steady velocity — the field at its start — and the engine warns once per realization and reports the number of such steps at the end of a verbose run. This happens at the end of the record, on a start between snapshots, and whenever Δt does not divide the snapshot interval: every step that straddles a snapshot then runs steady.

10.3 The order cap, and monotone-cubic reconstruction

A linear reconstruction in time is itself only second-order accurate, and this caps the *time*-truncation error of the whole integration at order Δt^2 irrespective of the spatial scheme. The practical consequence is important: under linear reconstruction, RK4 collapses onto RK2 — the fourth-order spatial scheme delivers no benefit — unless the step is taken much smaller than the snapshot interval. The remedy, short of finer sub-stepping, is a higher-order reconstruction in time, selected by `temporal_reconstruction = pchip`.

The PCHIP reconstruction (Figure 10.1) fits a monotone piecewise-cubic Hermite interpolant (Fritsch and Carlson, 1980) through four consecutive snapshots $t_{n-1}, t_n, t_{n+1}, t_{n+2}$, evaluating it within the central interval $[t_n, t_{n+1}]$. On that interval, with local coordinate $s = (t - t_n)/h_1$ and $h_1 = t_{n+1} - t_n$, the value is the Hermite combination

$$f(t) = h_{00}(s)\,f_n + h_{10}(s)\,h_1 m_n + h_{01}(s)\,f_{n+1} + h_{11}(s)\,h_1 m_{n+1}, \tag{10.3}$$

with the standard cubic basis $h_{00} = 2s^3 - 3s^2 + 1$, $h_{10} = s^3 - 2s^2 + s$, $h_{01} = -2s^3 + 3s^2$, $h_{11} = s^3 - s^2$. The slope at an interior snapshot is the weighted harmonic mean of the two adjacent secants d_- and d_+ over intervals h_- and h_+ (Fritsch and Butland, 1984),

$$m = \frac{w_1 + w_2}{w_1/d_- + w_2/d_+}, \qquad w_1 = 2h_+ + h_-, \quad w_2 = h_+ + 2h_-, \tag{10.4}$$

and zero wherever the two secants differ in sign or either vanishes. This makes the interpolant monotone between samples and prevents the spurious overshoots that an unconstrained cubic would introduce. The property holds for each component separately: the cubic does not preserve linear relations between components or between snapshots — a vanishing divergence, rotation equivariance, a window mean — which the linear reconstruction does preserve. Because the reconstruction is now cubic in time, the fourth-order spatial scheme is no longer masked: RK4 separates from RK2 at coarse snapshot cadence. The PCHIP stencil requires four consecutive snapshots, so on the first and last interval of the record, and across a cyclic seam, the engine falls back to the linear reconstruction, with one warning per realization. The reconstruction applies to the horizontal and vertical velocity and the sea-surface height; the diffusivities stay linear in time, and sampled scalars are re-interpolated linearly to the record time (Section 7.1). PCHIP is active only with `temporal_interp: true` (otherwise it warns that it has no effect) and only for the multistage schemes RK2 and RK4; the Euler step has no stage times to reconstruct.

The two-dimensional least-squares kernels and the three-dimensional barycentric RK2 kernel evaluate the cubic inline for each element or vertex they read (Chapter 11). Every other

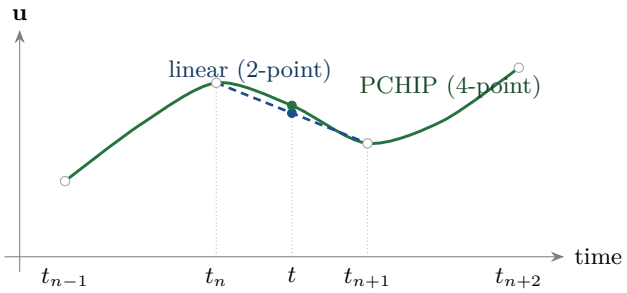


Figure 10.1: Temporal reconstruction within the interval $[t_n, t_{n+1}]$. The linear reconstruction (dashed) joins the two bracketing snapshots; the monotone-cubic PCHIP reconstruction (solid) uses four consecutive snapshots and follows the curvature of the sampled series. At a sub-step time t the two differ, and only the cubic lets a fourth-order spatial scheme realize its order. PCHIP is monotone, so it introduces no overshoot between samples.

configuration reconstructs whole fields at each distinct stage time — the two midpoint stages of RK4 share one — with a parallel loop over the field that is bit-identical to the plain array expression; one such blend of a global field takes about 46 ms at eight threads.

10.4 Read-ahead

Reconstruction in time requires several snapshots to be in memory at once — two for the linear scheme, four for PCHIP. Snapshots stay in a small cache as long as the current bracket or stencil needs them, so a PCHIP run reads one new snapshot each time its bracket advances. The key `prefetch_size` controls how many consecutive snapshots are read in a single bulk operation: a value above one reads a window ahead in the direction of travel, reducing the number of separate reads on large meshes where input dominates the cost (Chapter 17), while a value of one reads one snapshot at a time; the trajectories are bit-identical either way. Read-ahead is disabled under cyclic forcing, where the wrap-around would otherwise thrash the cache, and when snapshots are shared between processes through `shared_fields_dir` (Chapter 14).

Source. The reconstructions are in `numerics/temporal_reconstruct.py` (the slopes `_fc_slope`, the Hermite evaluation and the parallel whole-field blend `_pchip_blend4`); the lazy linear view `LazyLerpField` and its blend are in `core/io.py`; the stage fields of the unfused paths are built by `_make_stage_vels` and `make_stage_vels_recon`; the bracket and stencil resolution (`_resolve_time_bracket`, `_resolve_recon_stencil`), the step payload (`_build_vel_payload`) and the read-ahead cache (`_VelCache`) are in `tracking.py`.

Chapter 11

Fused and Unfused Integration Kernels

The per-particle operations of a step — locate, interpolate, integrate, apply the boundary condition (Figure 2.1) — are realized by two interchangeable implementations. This chapter describes the two paths, which configurations take which, and the guarantee that relates them. The distinction is an implementation matter and does not change the numerics of the preceding chapters; we document it because it decides the cost of a step (Chapter 17).

11.1 Two implementations of one step

In the *fused* path the entire step for a particle — the nearest-node search at every evaluation site, the spatial and temporal interpolation, the integrator update and the containment check — is carried out within one just-in-time-compiled kernel that loops over the particles in parallel, with no intermediate arrays (Figure 11.1); two exceptions are the adaptive RK45 step, which tests containment after its kernel, and the three-dimensional RK2 kernel, which receives its start anchor from the batch search. In the *unfused* path the same operations are separate routines, called in turn from Python, with the intermediate quantities passed between them as whole arrays: all stage-one positions, then all stage-one velocities, and so on. Each of these passes is itself a parallel compiled loop — the stage interpolations of RK2 and RK4 feed the stage positions to the vectorised interpolators — so the unfused path also uses every core; it pays for the extra passes over memory and, on a time-dependent flow, for materialising the velocity field at each stage time. The fused path exists for speed; the unfused path exists for generality, covering every combination of options for which no fused kernel is provided. Without Numba every step is unfused.

11.2 Which path applies

Table 11.1 lists the fused kernel families and the conditions under which the step functions choose them; every other configuration runs on the unfused path. In short: the two-dimensional least-squares runs are fused for every integrator and every temporal treatment, except Euler with `em`/`m1` diffusion (whose combined stochastic step replaces it) and, with the exact guard off, HEALPix pixel meshes that fail the density certificate; two-dimensional barycentric runs on node-layout velocities are fused when the flow is time-dependent, reconstructed linearly and without element tracking; three-dimensional runs are fused for Euler with least-squares velocities and for RK2 with node-layout barycentric velocities on a z-level mesh. Among the paths that stay unfused are three-dimensional RK4, every three-dimensional sigma-mesh run, element-layout barycentric and edge least-squares interpolation, steady barycentric runs, and

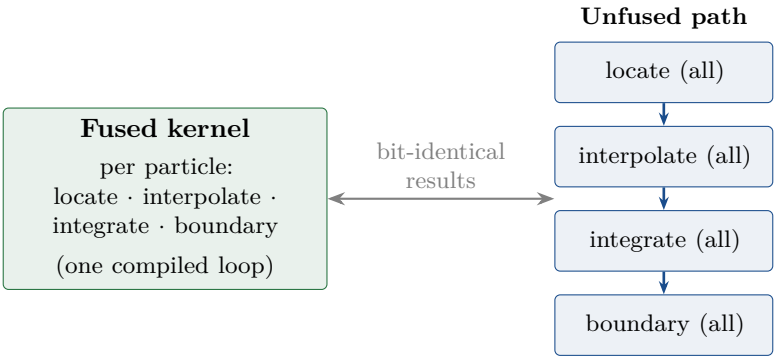


Figure 11.1: The two implementations of a single step. The fused kernel (left) performs all per-particle operations inside one compiled loop; the unfused path (right) runs the same operations as separate parallel passes over all particles. Where both exist they produce bit-identical trajectories.

kernel family	integrators	conditions
2-D least squares, steady	Euler, RK2, RK4	`uv_interp: leastsq` (element layout)
2-D least squares, time-aware	Euler, RK2, RK4	as above; RK2 and RK4 with the next snapshot, Euler with a velocity between two snapshots, blended per element
2-D least squares, PCHIP	RK2, RK4	as above, with `temporal_reconstruction: pchip`
2-D barycentric, time-aware	RK2, RK4	`uv_interp: barycentric`, node layout, next snapshot present, linear in time, no carried element
3-D Euler	Euler	z-level mesh, `uv_interp: leastsq`, `w_interp nearest` or `leastsq`
3-D RK2 barycentric	RK2	z-level mesh, node layout, `uv_interp: barycentric`, `w_interp nearest` or `barycentric`, no carried element; steady, linear or PCHIP
adaptive	RK45	two-dimensional, element-layout least squares; this scheme has no unfused twin

Table 11.1: The fused kernel families and the configurations that select them. "Carried element" refers to `elem_search: walk` (Section 4.6): the barycentric kernels keep their own search, so a walk run takes the unfused path, while the least-squares kernels run and ignore the carried element.

the `idw` and `leastsq` vertical combines of the three-dimensional RK2 kernel. The last one matters in practice: the default `w_interp: leastsq` keeps a node-layout three-dimensional RK2 run unfused, and setting `w_interp` to `nearest` or `barycentric` moves it onto the fused kernel.

Both three-dimensional families have depth-linear twins, selected under the same conditions by the placement of the field: the Euler twin for element-layout least squares and the RK2 twins for node-layout barycentric interpolation, steady, linear or PCHIP in time. They reproduce the unfused depth-linear path bit for bit (Section 6.4).

The three-dimensional RK2 kernel evaluates per particle: the stage-one anchor and layer, the horizontal velocity by barycentric interpolation with the temporal blend applied at each vertex, and the vertical velocity W_1 — computed before the horizontal verdict, so that a particle that dies on the horizontal interpolation reports the same vertical velocity on both paths; the midpoint $\mathbf{x}_{1/2}$ and $z_{1/2} = z + W_1\,\Delta t/2$; the stage-two anchor, layer and velocities; the end position with $z_{\text{new}} = z + W_2\,\Delta t$; the containment test; and the clamp of the depth to the water column. At one million particles on a mesh of 196 000 nodes and 70 levels it was measured 1.30 times faster than the unfused path at 16 threads (a development measurement not kept in the repository); the gain grows with the mesh and field size, while scattered per-particle

reads dominate both paths (Section 17.2). The two-dimensional barycentric kernels remove the per-stage materialisation of the whole node field: on a four-million-node mesh an RK4 step fell from 83 ms to 9 ms at eight threads, independent of the mesh size.

11.3 Bit-identity

Where a fused and an unfused implementation of the same configuration both exist, they produce bit-identical trajectories, statuses and failure reasons, which the test suite checks (Chapter 16). Three things make it hold.

The temporal blend. Rather than materialising the field at each stage time, the fused kernels apply a nested two-level blend inline,

$$\mathbf{u}_k = (1-\alpha_k)\big[(1-\alpha_0)\,\mathbf{u}_A + \alpha_0\,\mathbf{u}_B\big] + \alpha_k\,\mathbf{u}_T, \tag{11.1}$$

where A and B are the snapshots bracketing the step's start, α_0 is the start's position in that bracket, and T is the stage *target* snapshot the step moves toward — the later bracket snapshot when integrating forward, the earlier one when integrating backward, which is why the target is passed separately from the bracket. Equation (11.1) performs the same floating-point operations as stepping on the materialised field of Equation (10.1); with $\alpha_0 = 0$ it collapses to a single blend, and the barycentric kernels short-circuit the end-point fractions exactly as the unfused stage construction does, so the identity holds for non-finite values too.

The anchors. Every evaluation site of a fused kernel — the start, each stage and the final position — finds its nearest node through the exact search of Section 4.3, the same function the unfused batch query calls, so the two paths anchor every particle to the same node, ties included. This is the default (`mesh.exact_nearest: true`). With the guard off, the fused kernels widen an empty window ring by ring — an approximate anchor — and identity rests on gates instead: the two-dimensional barycentric kernels (and the least-squares kernels on HEALPix pixel meshes) run only on meshes that pass the density certificate of Section 4.3, with a warning and a fall-back to the unfused path otherwise; the three-dimensional RK2 kernel flags a particle whose stage window is empty, repairs the flagged positions with the batch search and re-runs only those particles through the kernel with the repaired anchor, for up to two rounds, before a last-resort unfused re-run of the remainder. The least-squares kernels on FESOM-family meshes and the fused three-dimensional Euler kernel have no such gate: with the guard off they agree with the unfused path only where no three-by-three window is empty, which holds in the domain interior of quasi-uniform meshes.

The diagnostics. Each fused kernel records why a particle failed — interpolation failure or out of bounds — in a per-particle scratch array at its failure branches, and the caller merges it into the sticky failure-reason field (`mode.track_failure_reason`, Chapter 13) exactly as the unfused path does. A failed particle keeps its start position and anchor on both paths, and frozen particles are copied verbatim.

Source. The fused kernels are the `_fused_*` functions of `numerics/integrators.py`; their runners are `_run_fused_2d`, `_run_fused_euler_2d_pair`, `_run_fused_2d_tinterp`, `_run_fused_2d_bary_tinterp`, `_run_fused_2d_pchip`, `_run_fused_euler_3d` and `_run_fused_rk2_3d_bary_tinterp/_pchip`; the adaptive scheme is `_rk45_kernel_2d`. The dispatch lives in `euler_step`, `rk2_step` and `rk4_step`; the anchor of each site is `_fused_site_anchor`; the guard-off gates are `_bary_fused_anchor_exact` and `_leastsq_fused_anchor_exact`, and the repair loop `_run_fused_3d_with_heal`; failure reasons are merged by `_merge_fused_failure`. The unfused stage interpolation goes through `_interp_uv_at`. The inline blends are the `_interp_*_tinterp` and `_interp_*_pchip4` cores of `numerics/interpolate.py` and, for the vertical velocity, of `numerics/vertical.py`.

Part V

Stochastic Transport and Boundaries

Chapter 12

Stochastic Diffusion

The resolved velocity field carries particles by advection, but it omits the mixing done by motions below the model's resolution. The engine can represent this unresolved mixing as a random walk superimposed on the advective step, turning the trajectory equation into a stochastic differential equation. This chapter sets out the random-walk model and its treatment of the integration direction, how a horizontal kick moves a particle on the sphere, the three horizontal schemes, the way the diffusivity and its gradient are sampled from the mesh, the vertical random walk, the counter-based random-number generator that makes the result reproducible and restart-safe, and how the kicks compose with the integrators. Diffusion is off by default; when disabled it is byte-for-byte identical to a purely advective run.

12.1 The random-walk model

A diffusivity K_h spreads an ensemble of particles so that its variance grows linearly in time, $\sigma^2 = 2K_h t$. The corresponding increment added to a particle over a step combines the deterministic advective displacement with a random kick (Figure 12.1):

$$\Delta x = u\,\Delta t \;+\; \xi\sqrt{2K_h\,|\Delta t|}, \tag{12.1}$$

where ξ is a standard normal random variate drawn independently per component. With a spatially varying diffusivity a drift term $\partial_x K_h\,|\Delta t|$ joins them (Section 12.3). The signs follow three rules. The advective term carries the signed step, so it reverses under backward integration. The random term uses $\sqrt{|\Delta t|}$ and spreads the ensemble in *either* direction. The drift uses $|\Delta t|$ too: it belongs to the diffusion, not to the advection, so a backward run integrates the adjoint process, $-u + \nabla K_h$ plus the same noise. (A drift that reversed with the step would leave a backward ensemble in a closed column with a stationary density proportional to K^{-2} instead of a uniform one.) Diffusion is intrinsically irreversible; a backward run does not un-mix what a forward run mixed, and the backward cloud is not a validated probability distribution of source positions. The diffusivity may be anisotropic, with independent zonal and meridional components K_x, K_y.

12.2 Kicks on the sphere

A kick is a displacement (d_x, d_y) in metres, east and north. Added to longitude and latitude through the metric of Equation (9.1), it would be a random walk on the longitude–latitude chart, which lacks the drift $-K\tan\phi/R_M$ that a random walk on the sphere has: its particles would spread uniformly in latitude rather than in area and would accumulate toward the poles. The engine therefore moves every horizontal kick along the great circle. With $d = \sqrt{d_x^2 + d_y^2}$,

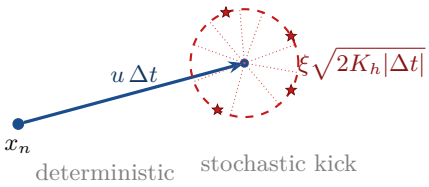


Figure 12.1: One diffusive step. The deterministic advective displacement $u\,\Delta t$ (blue) carries the particle from x_n; a random kick of typical size $\sqrt{2K_h|\Delta t|}$ (dashed circle) is then added, scattering the realized endpoint (stars). The advective term reverses under backward integration; the kick, scaling with $\sqrt{|\Delta t|}$, spreads the ensemble either way.

$\delta = d/R_M$ and the metric radius $R_M = M_\phi \cdot 180/\pi$, the position $\mathbf{r} = (\cos\phi\cos\lambda, \cos\phi\sin\lambda, \sin\phi)$ and the tangent displacement

$$\mathbf{t} = d_x\,(-\sin\lambda,\ \cos\lambda,\ 0) + d_y\,(-\sin\phi\cos\lambda,\ -\sin\phi\sin\lambda,\ \cos\phi), \tag{12.2}$$

the kicked position is

$$\mathbf{r}' = \mathbf{r}\cos\delta + \frac{\mathbf{t}}{d}\sin\delta, \tag{12.3}$$

converted back to longitude and latitude. The east and north directions are taken at the stored longitude, so the map is regular through a pole. For an anisotropic diffusivity the sphere adds one more term, a northward drift $(K_x - K_y)\tan\phi\,|\Delta t|/R_M$ (with the latitude capped at $\pm 89.9°$); with $K_x = K_y$ it vanishes and the uniform scheme is free of any drift.

Source. `sphere_displace`, `sphere_displace_arrays` and `anisotropic_metric_drift_m` in `numerics/diffusion.py`.

12.3 The three horizontal schemes

The key `diffusion.scheme` selects among three realizations of increasing fidelity to a spatially varying diffusivity. The `uniform` scheme uses a constant diffusivity taken from the configuration (`kh_zonal`, `kh_meridional`) and adds the kick of Equation (12.1) with no gradient term. The `em` scheme (Euler–Maruyama) reads a spatially varying diffusivity from the dataset and adds the Itô drift, per component,

$$\Delta x = u\,\Delta t + \partial_x K_x\,|\Delta t| + \sqrt{2K_x|\Delta t|}\;\xi_1, \tag{12.4}$$

the drift being what prevents particles from accumulating spuriously in regions of low diffusivity and so keeps the scheme consistent with the diffusion equation. The `m1` scheme (Milstein) adds the next term, $\frac{1}{2}\partial_x K_x\,|\Delta t|\,(\xi_1^2 - 1)$; it is rigorous only for a separable diagonal diffusivity and warns once whenever the field varies in space, for which `em` is the safer choice. With Euler (the combined composition) the diffusive part — drift, Milstein term and noise — moves the particle along the great circle from the start of the step (Section 12.2) and the advective part is added on the chart; with RK2 and RK4 (the split composition) the kick is applied at the position the deterministic step reached. Where the diffusivity is constant, `em` and `m1` draw the same normals as `uniform`; only where the kick is applied differs.

12.4 Sampling the diffusivity

The spatially varying schemes require both the diffusivity and its spatial gradient at the particle. The diffusivity is read from node or element variables, their layout detected from the dimensions, and it must not be negative. The value is obtained by the least-squares plane fit of Section 5.3, gated on containment. The gradient is obtained by central differencing of that value over an

offset `dres_m` — by default the median edge of up to 5000 sampled elements, clipped to 100 m to 50 000 m — with a fallback chain to a one-sided difference and then to the analytic gradient of the fitted plane should the central difference be unavailable near a boundary.

12.5 Vertical diffusion

In three dimensions on a z-level or sigma mesh, a vertical diffusivity A_z adds a vertical random walk (`az_enabled`, with a constant `az_const` or, on FESOM2 z-level output, a field `var_az` on the interfaces). Its increment in positive-down depth is

$$\Delta z = \frac{\partial A_z}{\partial z}\,|\Delta t| + \sqrt{2A_z|\Delta t|}\;\xi, \tag{12.5}$$

with the drift mandatory: without it a walk in a non-uniform A_z would collect particles where A_z is small (Visser, 1997). The field is taken from the nearest node's column, linear between interfaces, with the depth clamped to the wet part of the column; the gradient is the slope of the same linear segment — on an interface, the segment above — so that the value that sets the noise and the gradient that sets the drift belong to one interpolant. A kick that leaves the column is reflected, not clamped: with the column $[a, b]$, $L = b - a$ and $y = (z - a) \bmod 2L$, the new depth is $a + y$ for $y \le L$ and $a + 2L - y$ otherwise, which handles any number of bounces. A clamp would pile particles up at the boundaries; the reflection keeps a well-mixed column well mixed, which the tests check in a diffusivity that varies tenfold over the column, in both directions of integration. The column is the surface to the nearest node's floor on a z-level mesh and the particle's end-of-step column less z_{0b} on a sigma mesh, the same bounds as the depth clamp of Section 6.8. The vertical kick composes only with the `uniform` horizontal scheme, runs after the deterministic step and the horizontal kick, and with no source configured the facility is dormant.

Source. `diffusion_az_step_meters`, `az_column_value_gradient`, `reflect_depth` and the loops `_loop_az_kick_zlevel` and `_loop_az_kick_sigma` (`numerics/diffusion.py`); applied by `_apply_split_az_kick` (`numerics/integrators.py`).

12.6 Reproducibility: counter-based random numbers

The random kicks are generated not by a sequential pseudo-random stream but by a *counter-based* generator (Salmon et al., 2011), in which each draw is a stateless hash of an integer key. The key is the tuple of the configuration seed, the realization index, the *original* particle index, the global step index, and the component — zero for the horizontal pair, one for the vertical kick — from which a 64-bit hash produces a uniform pair that is converted to a normal pair. Because each draw depends only on its key and not on the order in which draws are issued, the result is bit-for-bit identical regardless of the number of threads or the order of execution, and it is identical across a restart — the same step index regenerates the same kick. The particle index in the key must be the original, fixed index, not a compacted index that shifts as particles die, or the stream would change when the live set changes. A run over one shard of a larger release (Section 17.9) adds the global index of its first particle, `data.particle_offset` or the header that the splitting tool writes into each shard file, so a particle draws the same numbers in the shard as in the unsharded run.

12.7 Composition with the integrator

How the stochastic kick combines with the deterministic step depends on the scheme and the integrator (`diffusion.composition`, resolved when the run starts). The `uniform` scheme

composes by *splitting*: the deterministic step (of any integrator, including RK45) is taken first and the kick added afterward. The `em` and `m1` schemes with the Euler integrator *combine*: the stochastic differential equation replaces the Euler step as a whole; this mode needs element-layout least-squares velocities. The same schemes with RK2 or RK4 use an opt-in split (`composition: split_rk`), taking the Runge–Kutta step and then the kick, with the diffusivity sampled at the start or, by default, at the end of the step (`kh_eval_position`) and the kick applied at the post-step position; the engine warns once that this is a pragmatic extension rather than a rigorous scheme. The spatially varying schemes do not combine with RK45.

The kicked position is checked like any other: a kick that leaves the mesh returns the particle to its position at the *start* of the step with status `OUTSIDE`. In three dimensions only the `uniform`/split combination is available for the horizontal kick; the depth is clamped again at the kicked position, and on a sigma mesh the particle keeps its fraction of the column. The combined and split-with-Runge–Kutta modes raise an explicit error in three dimensions rather than apply an unvalidated scheme. In time, the diffusivities are interpolated linearly with the velocity payload and are not included in the monotone-cubic reconstruction.

Source. The schemes, the drift, and the Milstein term are `diffusion_uniform_step_meters`, `diffusion_em_step_meters` and `diffusion_m1_step_meters`; the counter-based generator — a keyed SplitMix64 hash in the counter-based spirit of Salmon et al. (2011) — and the Box–Muller pair are in the same module, `numerics/diffusion.py`. The composition is resolved by `_resolve_composition` (`tracking.py`) and applied by `_apply_split_diffusion_kick`, `_apply_combined_em_step` and `_apply_split_rk_em_kick` (`numerics/integrators.py`); the default offset is `_compute_default_dres_m` (`tracking.py`).

Chapter 13

Particle Lifecycle and Boundary Conditions

A particle is not simply a position that updates each step: it carries a status that records whether it is being advected, has been removed, has stranded, or has not yet been released. This chapter describes the four statuses and the transitions between them (Figure 13.1), the record of why a particle died, the treatments of the domain's boundaries — removal at the lateral boundary, clamping at the surface and floor, and the optional periodic fold — the optional beaching model, and the delayed-release mechanism by which particles enter the simulation on a schedule.

13.1 The four statuses

Each particle holds one of four statuses. An `ALIVE` particle is advected normally and is the only kind included in the working set that the integrator processes. An `OUTSIDE` particle has left the domain; the status is terminal, and the particle is thereafter inert, its position frozen where it last was valid. A `BEACHED` particle has stranded in shallow water; uniquely among the non-alive statuses it is reversible, as a beached particle may refloat (Section 13.5). A `NOT_RELEASED` particle has not yet reached its scheduled release time; it is excluded from the working set, frozen in place at no computational cost, until it is activated (Section 13.6). Frozen positions are copied from step to step unchanged, not re-wrapped, so a dead particle at longitude 180° stays there.

13.2 Why a particle died

With `mode.track_failure_reason: true` each particle also carries a failure reason: none, *interpolation failed* (a stage found no fit or no containing element) or *out of bounds* (the final position lies in no element). Two further codes, for surface and bottom contact, are reserved and never set, because those limits never remove a particle (Section 13.3). The reason is set once, when the particle dies, and never overwritten; every integration path records it, fused and unfused alike (Chapter 11). A beaching remap or a resuspension clears it, so the field keeps the meaning "cause of death". It is written to the trajectory output and to checkpoints.

13.3 Lateral removal versus vertical clamping

The engine treats the lateral boundary and the vertical limits in qualitatively different ways, and the distinction is deliberate. The containing-element test of Section 4.5 is the lateral boundary condition: a particle whose advected position is contained by no element has left the domain, and the engine *removes* it — it sets the status to `OUTSIDE` and reverts the position to the start

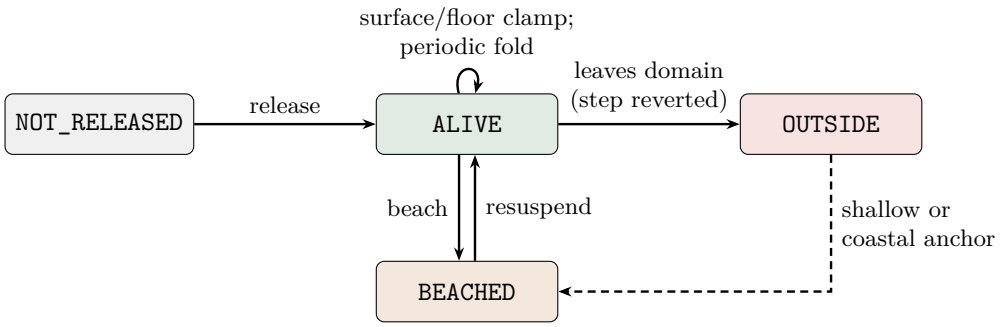


Figure 13.1: The particle status state machine. A particle is released from `NOT_RELEASED` into `ALIVE` at its scheduled time; it is removed to `OUTSIDE` if it leaves the domain (its step undone). With beaching enabled, `ALIVE` and `BEACHED` interconvert, and a particle removed in the same step with a shallow or coastal anchor is reclassified as `BEACHED` (dashed). Reaching the surface or floor does not change the status: the depth is clamped and the particle remains `ALIVE`; nor does a fold back into a periodic window (the self-loop).

of the step, so the particle does not record a spurious move across the boundary. A failed interpolation removes a particle in the same way, and so does a diffusive kick that ends outside the mesh (Section 12.7). Because the test is geometric, an open boundary and a coastline are indistinguishable to it; both produce an `OUTSIDE` result unless the beaching model says otherwise. The surface and the sea floor, by contrast, are enforced by *clamping* after the advective step (Section 6.8): a particle that would cross either limit is held at it and continues to be advected. The vertical diffusive kick is the exception: it *reflects* at the limits, because a clamp would pile particles up there (Section 12.5).

13.4 Periodic windows

An idealized channel domain is made periodic in longitude, latitude or both by giving a window $[x_0, x_1)$ in `numerics.periodic_lon` or `numerics.periodic_lat`. After every step, before the beaching hook and the record, an alive particle outside the window is folded back,

$$x' = x_0 + ((x - x_0) \bmod (x_1 - x_0)), \tag{13.1}$$

and receives a fresh nearest node; particles inside the window are left bit for bit untouched. Seeds are folded once at the start, including those not yet released. The fold adds no connectivity across the seam, so the mesh must extend beyond each periodic side by a *ghost band* at least one step's displacement wide: a particle must still be inside the mesh at the end of the step in which it crosses. The configuration is checked before the output file is created: the windows must lie strictly inside $(-180°, 180°)$ and $(-90°, 90°)$ and overlap the mesh, a periodic-longitude mesh must not straddle the $\pm 180°$ meridian, and a missing ghost band on either side produces a warning. Periodic windows cannot be combined with a rotated frame (Chapter 8). The windows are written to the output metadata when set.

13.5 Beaching and resuspension

When beaching is disabled (the default), every lateral exit becomes `OUTSIDE`. When the key `numerics.beaching.enabled` is set, a post-step hook strands particles by up to four criteria, combined inclusively. The water column at the particle's nearest node is taken raw, $H = d + \eta$ when `use_eta` is set and $H = d$ otherwise, without the minimum-thickness floor, which would hide tidal drying; η comes from the step's own velocity payload.

`depth`
: $H <$ `depth_min`, or the node is marked dry by `dry_mask_var`.

`coastal_element`
: the particle's containing element has an edge on the coast (an edge owned by one element only).

`on_failure`
: a particle the integrator has just removed is reclassified as `BEACHED` if its reverted anchor has H below the zone depth or lies in a coastal element (the coastal test needs the coastal flags, built only when `coastal_element` or `stochastic` is also a criterion); a removal in deep water at an open boundary stays `OUTSIDE`.

`stochastic`
: within a near-shore zone — $H <$ `zone_depth` (default twice `depth_min`), or coastal contact whenever the coastal flags exist — a particle strands with probability $1 - \exp(-|\Delta t|/t_{\mathrm{beach}})$ per step.

A beached particle counts as re-wetted when $H >$ `depth_min` + `rewet_hysteresis` and its node is not marked dry; the hysteresis suppresses flicker between the two states. The resuspension policy is one of `never`; `auto`, which refloats a particle after `min_wet_steps` consecutive re-wetted steps; `stochastic`, which refloats a re-wetted particle with probability $1 - \exp(-|\Delta t|/t_{\mathrm{resuspend}})$; and `auto_or_stochastic`. The random draws use the counter-based generator of Section 12.6 with their own components, keyed by the original particle index and the global step, and the count of re-wetted steps is written to checkpoints, so a resumed run reproduces the uninterrupted one bit for bit.

The hook runs in two phases after the step: first the reclassification of just-removed particles, then the criteria and the resuspension. Its containment test runs only for alive, beached and just-removed particles. The coastal flags are built only when a criterion needs them. A configuration asking for η without a sea-surface height in the dataset, or naming a dry-mask variable that is absent, is refused at startup. On a HEALPix mesh with neither a bathymetry variable nor `mesh.levels` the node depths are zero, and the depth criterion would strand every particle.

13.6 Delayed and pulsed release

Release positions are read from the seed file. Two library helpers can write one: a regular grid over the mesh extent, shrunk inward by a margin and filtered to points inside the mesh, and points scattered uniformly in a circle of given radius around a centre, scaled by $\cos\phi$ so that the circle is round in metres. Both work in the mesh's own coordinates: on a rotated mesh the caller rotates the centre into the mesh frame and the kept points back to geographic before saving them as seeds.

Particles need not all enter at the start of the run. A per-particle release offset (a fourth column of the seed file, in seconds from the start) and a pulse schedule that re-releases the whole seed set at a fixed interval (`time.n_pulses`, `time.pulse_interval`) are both supported, and both are implemented by pre-allocation and freezing rather than by growing the particle array. At setup the seed set is replicated to its full final size — the number of pulses times the number of seeds, pulse by pulse — and each copy is given a release step

$$k_{\mathrm{rel}} = \mathrm{round}\Big(\frac{t_{\mathrm{rel}} + p\,\Delta t_{\mathrm{pulse}}}{\Delta t}\Big) \tag{13.2}$$

for pulse p, with a warning when the offset is not a whole number of steps. Particles whose release step lies in the future begin as `NOT_RELEASED`. At the top of each step the engine activates

those whose step has come, $k \geq k_{\mathrm{rel}}$ in the global step count (so the rule holds for backward runs and across restarts), flipping them to `ALIVE` so they are advected during that same step. A frozen particle costs nothing in the kernel, which skips it, and the fixed array size is what allows the streaming output writer to use a constant trajectory dimension. Dormant rows are written as missing values with their status; a particle whose release step lies beyond the end of the run produces a warning and stays missing throughout. The output records each particle's release position and release time, the start of the step in which it is activated (Equation (15.3)), so the rounded schedule can be read back from the file. The diffusive noise is keyed by the global step, so a particle released later draws a different sequence from one released at the start, by design. A run with a single release and no offsets reduces exactly to the unscheduled case.

Source. The statuses and failure reasons are the enumerations `ParticleStatus` and `FailureReason` in `core/particles.py`; lateral removal and the revert are in the integrator kernels of `numerics/integrators.py`; the periodic fold and its checks are `_apply_periodic_wrap` and `_validate_periodic_windows`, the beaching hook is `_apply_beaching_hook` (`tracking.py`) over `evaluate_beaching` (`numerics/beaching.py`); the release schedule is `build_release_state`, `activate_released` and `mask_dormant_for_record` (`core/release.py`).

Part VI

Data, Configuration, and Validation

Chapter 14

Forcing Input and Time Control

The trajectories are driven by the model's stored velocity fields, which the engine reads on demand from disk and places on a time axis. This chapter describes the reader layer and its lazy access to the forcing, the requirement that the selected variables share a single time cadence — the most common configuration error on FESOM output — the decoding of the time axis, the resolution of the integration window, the handling of time-averaged forcing, the velocity handed to each step, and the cache through which several processes on a node can share one copy of each snapshot. The in-memory read-ahead that serves the reconstruction of Chapter 10 is part of this layer.

14.1 The reader layer

A reader is selected by `mesh.type`: `fesom2`, `fesom14`, `fesomc`, `nemo` or `healpix`. Each provides the mesh, opens the forcing, and reads one snapshot at a time; the NEMO reader builds its mesh with the engine's generic builder for structured grids, which turns the wet cell centres of a logically rectangular grid into nodes and every four adjacent wet centres into a quadrilateral element. The forcing is opened *lazily*: the dataset is mapped through `xarray` without loading the field arrays, and a snapshot is materialized only when a step requires it.

FESOM2
: The mesh comes from a mesh-diagnostics file (three naming conventions are recognised), an ASCII mesh directory or a Zarr store. For the multi-year layout — one file per variable per year, `<var>.fesom.<year>.nc` — the reader assembles exactly the variables the configuration names (velocities, surface height, sampled scalars, vertical diffusivity) and concatenates the years along time with `dask`, so a run may span decades without the files being merged on disk and naming `unod` does not pull in fields that merely share a prefix. A directory in any other layout is assembled from the first matching file per velocity component and the other readable files on the mesh's dimensions.

FESOM 1.4
: The mesh comes from the model's ASCII mesh directory or from the cell polygons of level-wise data (Section 3.7). The forcing, `data.ncinput` (the files of `ncinput_v` and `ncinput_w` simply join it), is a file, a directory (its `*.nc` files in name order, hidden files skipped) or a glob pattern, for instance over the member directory of a CMIP6 tree. Every file's header is read, and the files are grouped by the variables they hold, never by their names. Only the variables the run *consumes* are opened: u and v (`mode.var_u`, `var_v`, default `uo`, `vo`), in three dimensions W (`wo`), the sea-surface height (`zos`) when a file holds it, every `mode.sample_vars` entry and, with beaching enabled, `beaching.dry_mask_var`; a required variable that no file holds is refused, and the message lists what the files do hold. Two data

forms are recognised from the dimensions: level-wise files (time, depth, node), such as the published CMIP6 output, and the raw writer's vectors over the three-dimensional nodes (time, nodes_3d), which are mapped onto the levels through the index table of the ASCII mesh. Model restart files, the mean files of an older model branch whose frame and time axis cannot be checked, and files whose layouts, levels or node coordinates disagree are refused. Every record of every variable is compared with the static wet mask of the mesh on every level before anything is changed; only then are u, v and W set to zero below the bed and sampled scalars continued downward from the column's last wet value, so that no fill value reaches a particle. The velocities come back on the levels, in node layout, with the placement `"level"` (Section 6.4); a two-dimensional run reads one level (`mode.sigma_layer`), the depth mean (`mode.sigma_mean`) or the surface. Without `zos` the surface height is zero. The reader resolves the time each record represents itself (Section 14.5). Refused before the output file exists: the `em` and `m1` diffusion schemes, since FESOM 1.4 output holds no diffusivity fields, and `beaching.use_eta` without a sea-surface height; the offline reconstruction of W refuses FESOM 1.4 meshes, whose output holds `wo` itself.

FESOM-C
One NetCDF file carries the mesh and the forcing; separate files for v and w may be merged in.

HEALPix
The mesh is derived from the data store itself: its nodes are the wet pixel centres (`mesh.healpix_geometry: dual`, the default) or, with `pixel`, the pixels are the elements and their corners the nodes (two-dimensional only); wet means finite velocity at the first record on the tracked level. Plain Zarr and NetCDF stores and reference stores (`.parq`, `.json`) are supported; the sea-surface height may come from a separate store with an identical time axis (`data.ncinput_eta`), and the vertical levels from `mesh.levels` (Section 6.2).

NEMO
The grid-T file is the input; the grid-U and grid-V files are found by substituting the grid token in its name and must share its time axis exactly. The first version is two-dimensional and requires a grid aligned with east and north.

A reader may also provide a hook `prepare`, which checks the opened data against the mesh. It runs at every entry: right after the data are opened, before any output file exists, and again at the entry of each realization, after the `max_levels` cut, so it always sees the levels in use. The FESOM 1.4 hook only checks — the horizontal sizes, the full level axis, the node coordinates and the wet mask of the first record of every variable — and changes nothing. `kelp info` opens the data as a run does, the hook included.

Every reader that reads a vertical velocity converts it to the engine's positive-down convention (Section 6.1; FESOM 1.4 writes `wo` positive up), and with `velocity_float32` stores the velocity fields in single precision to halve their memory, the interpolation arithmetic staying in double precision.

14.2 One cadence at a time

FESOM output stores variables at several temporal cadences, set per variable in the model's configuration, that nonetheless share a time-units string. The variables selected for a run must share a single cadence (Figure 14.1): Lagrangian advection typically uses the daily node velocities and vertical velocity together with the daily sea-surface height and any daily scalars, and must not mix them with monthly element velocities. On the multi-year path the reader merges the selected variables onto a common time axis with an exact join; a set that cannot

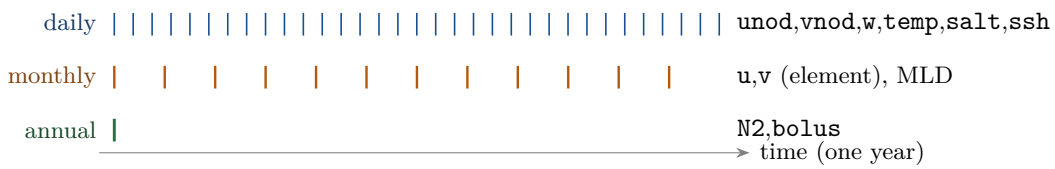


Figure 14.1: An example of the cadences of one FESOM2 output configuration: the variables share one time-units string but are sampled at different rates. A run must select variables of a single cadence; Lagrangian advection uses the daily node-based fields (top). Mixing cadences — for example the daily `unod/vnod` with the monthly element `u/v` — cannot be aligned on one time axis and is rejected when the forcing is opened (a finer attached file that contains every velocity stamp is thinned with a warning instead).

be aligned raises an error with the hint that a mixed cadence is the likely cause, the merged axis is checked to be strictly increasing, and a gap between years produces a warning. On the other layouts every file attached to the velocity file (a separate v or w, the remaining files of a directory) is matched to the velocity records by time stamp: a file on another axis is refused, and a finer one that contains every velocity stamp is thinned to them with a warning. A mixed-cadence selection is easy to write and is caught only when the forcing is opened, not when the configuration is read. The FESOM 1.4 reader applies the rule to the records it resolves: every consumed variable must have exactly the records of u (Section 14.5).

14.3 The time axis

The snapshot times are converted to seconds from the first record. A datetime axis and a calendar-aware (`cftime`) axis are converted directly; a numeric axis with units of the form "*unit* since *epoch*" is decoded with its calendar; a numeric axis with a bare unit (days, hours, minutes, seconds and their abbreviations) is scaled by that unit. An index axis $0, 1, \ldots, N-1$ without units is *not* taken as time; any other numeric axis without units is read as seconds. The snapshot spacing is the common difference when all differences agree to 10^{-3}, and `numerics.dt` defaults to it. In the discrete mode `numerics.dt` must equal that spacing, and an irregular axis is refused; the interpolated mode accepts both. When the axis cannot be decoded, `numerics.file_dt` supplies the spacing and the axis becomes $t_i = i\,\Delta t_{\text{file}}$; without either, the interpolated mode of Section 10.1 is refused before the output file is created.

14.4 The integration window

The window in time is specified flexibly. The start is an index into the forcing record (`time.start_time_idx`) or a calendar date (`time.start_date`), parsed in the calendar of the axis. In the interpolated mode the run starts at that instant, whether or not it coincides with a stored time; in the discrete mode the date must coincide with a stored time, and the run starts at that record. The length is a number of integration steps (`time.n_steps`, the preferred form), an explicit end index, or a duration in days (`time.duration_days`, only together with `time.start_date`, converted to $\lfloor \text{days} \cdot 86400/\Delta t \rfloor$ steps). When the run reaches the end of the available forcing it either stops or wraps to the beginning, according to `on_exhausted`, the latter permitting a periodic forcing to be cycled. Multiple independent realizations may be requested (`num_runs`), each offset in start time by `run_interval` snapshots, which is the basis for ensembles with the stochastic schemes of Chapter 12; each realization writes its own output file.

14.5 Time-averaged forcing

Forcing is often stored as time averages — daily or monthly means — and the engine must know this to place the snapshots correctly on the time axis. The key `time_averaging` declares whether the fields are averages (`mean`) or instantaneous samples (`instant`). For averaged fields, whose file timestamps conventionally mark the *end* of each averaging window, the engine shifts each timestamp to the window *midpoint*, $\tilde{t}_i = (t_{i-1} + t_i)/2$ and $\tilde{t}_0 = t_0 - (t_1 - t_0)/2$, before it is used for temporal reconstruction, so that a daily-mean field is treated as representative of the middle of its day. Instantaneous fields are used unshifted. We recommend setting this key explicitly: when it is omitted the engine warns and guesses `mean` for FESOM2 and HEALPix and `instant` otherwise, and the guess, though usually correct, is not guaranteed. A start date is matched against the *unshifted* file times. In the interpolated mode the run then starts at the named instant (until 2026-10-04 a date equal to a stored stamp started half a window earlier, at the shifted midpoint, while a date one second later started on time); a date in the last half window, past the last midpoint, starts at the last record with a warning. In the discrete mode the run starts at the matched record.

Record times resolved by the reader (FESOM 1.4). The FESOM 1.4 reader does not hand on the stamps as written. It decodes every file with its own epoch and calendar — the published CMIP6 files start a new epoch every decade, raw output one per job segment — and resolves, once per variable, the instant each record represents: the *centre of its averaging window* for a time mean, the stamp itself for a snapshot. The window comes from exactly one source, in this order. First, the explicit keys `data.time_stamp` (where a stamp sits in its window: `start`, `centre` or `end`, or `instant` for snapshots) and `data.time_window` (`calendar_month`, `calendar_year`, `calendar_day` or a fixed duration such as `"5 d"`), given together; they decide for every file, and when a file also has time bounds or an output schedule a warning names what was overridden and the first record where the two disagree. Second, the files' time bounds, as in the CMIP6 files: a January record bounded by 0 and 31 days represents day 15.5, that is, 12:00 on the 16th. Third, the raw writer's `output_schedule` attribute, whose stamps are the window ends in seconds since the job's start date; the reader enumerates the output events the schedule implies and requires the stamps to be exactly those events. Otherwise the run is refused with a message naming the two keys. The key `data.time_offset` (`"<±number> <s|min|h|d>"`) is added to the stamps, the bounds and the epoch of every file before any of this; it serves raw output of a job that started at a time of day other than midnight, whose stamps all sit early by that time of day, and for raw-writer files it must lie in $[0, 24\,\mathrm{h})$. Calendar windows are computed exactly in the files' calendar, and with a calendar window a stamp that does not sit where `data.time_stamp` puts it — on a boundary of the window, or at its centre — is refused. A window is never estimated from the spacing of neighbouring stamps, every record the source says must exist has to be present — a gap, a missing month or file, refuses the run — and every consumed variable must resolve to exactly the records of u, the same instants and windows to within 1 ms. The last rule is the reader's form of the single-cadence requirement of Section 14.2: the default raw output, which writes `zos` daily and the velocities monthly, is refused, and the message suggests leaving `zos` out of the input.

The resolved instants become the dataset's time axis, and nothing downstream moves them again. For this reader the engine gives no warning, applies no midpoint shift, refuses a set `data.time_averaging`, and writes `time_averaging = "resolved_by_reader"` to the output. The refusal is deliberate: the shift of this section assumes stamps at the window ends, whereas the reader's instants are already window centres — a CMIP6 monthly stamp, for instance, already sits in the middle of its month — so a second shift would move every record by half a window. Daily means therefore sit at 12:00, and a `start_date` is matched against these centres: a discrete-mode start at midnight matches no record and needs `temporal_interp`. Monthly

means lie a calendar month apart, an irregular spacing that the discrete mode refuses, so they need the interpolated mode. The three keys are refused for every other mesh type, and the run summary and `kelp info` print one line saying what resolved the times, with the first and the last instant.

Source. The record times are `resolve_axis` in `readers/fesom14/_timeaxis.py`, called per consumed variable by `Fesom14Reader.open_dataset` (`readers/fesom14/reader.py`); the keys reach it through `_time_option_kwargs`, which also refuses `data.time_averaging` for the mesh types in `_READER_RESOLVES_TIME`, and `_describe_record_times` prints the summary line (`readers/__init__.py`); `run_tracking` (`tracking.py`) skips the midpoint shift and records `RESOLVED_BY_READER`.

14.6 The velocity handed to a step

Snapshots are read through a small cache keyed by record index, which keeps the bracketing pair, the PCHIP stencil and the read-ahead window and releases the rest (Section 10.4). Between two snapshots at fraction α the step receives a lazy view of the pair rather than an interpolated field: each field is blended, $(1-\alpha)\,a+\alpha\,b$, the first time a consumer reads it and is then cached, and a field that no consumer reads is never blended — the two-dimensional fused kernels and the three-dimensional barycentric RK2 kernel read none, the fused three-dimensional Euler kernel reads the blended velocities, and the beaching hook the surface height and a dry mask. At a snapshot, to within 10^{-12} in α, the snapshot itself is passed. With the view come the target snapshot and the time to it (Section 10.2). Large blends run as a parallel loop that is bit-identical to the array expression; fields stored in single precision are blended serially.

Source. The readers live in `readers/` (one module per model; the registry and `extract_time_info` in `readers/__init__.py`); the multi-year assembly and its cadence check are in `readers/fesom2.py`; the structured-grid builder is `readers/structured.py`; the FESOM 1.4 reader is the package `readers/fesom14/` (`Fesom14Reader` in `reader.py`, the two data forms and the validity predicate in `_layout.py`), and the `prepare` hook is run by `_prepare_dataset` (`readers/__init__.py`); the midpoint shift (`_shift_to_midpoints`), the start date (`_resolve_start_date`), the payload (`_build_vel_payload`) and the cache `_VelCache` are in `tracking.py`; the lazy view `LazyLerpField` and its blend are in `core/io.py`.

14.7 Sharing snapshots between processes

A large release is often split into several processes on one node (Section 17.9), and each would otherwise read and hold its own copy of every snapshot. With `numerics.shared_fields_dir` set, one process reads and publishes each processed snapshot and the others map the same bytes read-only (Figure 14.2). A process that finds a snapshot missing takes a file lock; the one that wins builds the arrays, writes them under a temporary name and renames the directory into place, which is atomic, so a sibling never sees a partial snapshot. The writer keeps a window of three snapshots on either side of the current one and removes the rest.

The snapshots live under a namespace derived from everything that determines their content: the configuration keys that change what a snapshot read returns (mesh, mode, the input paths, the velocity precision, the diffusivity and dry-mask variables), the size and modification time of the mesh and data files, the engine version, and a *generation*. The generation is the key `numerics.shared_fields_generation` when it is set; otherwise it is taken from the environment variable `KELP_SHARED_FIELDS_GENERATION`, and failing that from the batch job and job step. Shards of one job therefore share a namespace, while another job, or a rewritten data file, never reuses a snapshot; without a generation the run falls back to local reads. Each process registers itself on start and deregisters when it ends, and a new cache removes namespaces whose processes have all ended. Every error of the shared layer falls back to local reads with one warning, read-ahead is disabled, and the trajectories are identical with and without the

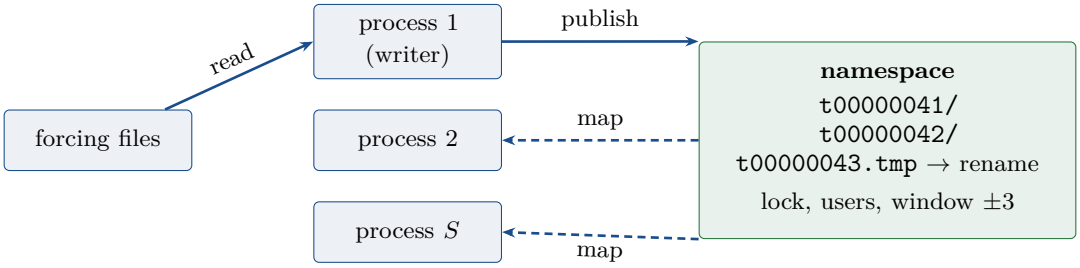


Figure 14.2: The node-shared snapshot cache. The process that wins the lock reads a snapshot from the forcing files, writes it under a temporary name and renames it into place; the other processes map the published arrays read-only. The namespace is derived from the configuration, the input files, the engine version and the job, so no other run reuses it.

cache. Placed on the parallel file system and shared within one node, the cache costs almost nothing; shared across nodes it slows every step (Section 17.8). Node-local memory (`/dev/shm`) has not been measured.

Source. `SharedFieldCache` in `core/shared_fields.py`; the namespace is `_shared_fields_tag` with `_path_fingerprint` and `_shared_fields_generation` (`tracking.py`).

Chapter 15

Trajectory Output and Restart

The product of a run is a set of trajectories: the recorded position, and any sampled properties, of every particle through time. This chapter describes the two output backends and how to choose between them, the variables written and their arrangement (output format 2, the format of release 1.0), the record time, the provenance recorded with the trajectories, the checkpoint mechanism that allows a long run to be resumed and continued in a chain of jobs, and the merging of a release split across processes. The authoritative variable-by-variable description is `docs/io_and_time.md`, section 5.

15.1 Two backends

The output format is chosen by the file extension of the output path. A `.zarr` path selects a chunked Zarr store; any other selects a compressed NetCDF4 file (an unlimited record dimension, compression level 4). The distinction is operationally significant for long runs. A NetCDF4 file is held under an exclusive write lock for as long as it is open, so it cannot be inspected while the run proceeds; the key `output_flush_every` (command line `-flush-every`) mitigates this by buffering that many records and then opening, appending, and closing the file, so that completed records become readable between flushes and the file stays byte-identical. A Zarr store, by contrast, is lock-free and can be read while the run is still writing. It buffers records in memory and writes one chunk of records at a time, so readers see it grow in chunk-sized increments and a killed run loses at most one chunk of records; a checkpoint and the end of the run flush the buffer. For multi-hour runs that one may wish to monitor, Zarr is the more convenient choice.

The Zarr layout. Each variable is chunked in blocks of n_t particles by n_o records,

$$n_t = \min(P,\ 500\,000), \qquad n_o = \max\Big(1,\ \min\big(256,\ \lfloor 4 \cdot 10^6/(8\,n_t)\rfloor\big)\Big), \tag{15.1}$$

so a chunk holds about 4 MB. Up to half a million particles the particle axis is one block and a chunk is one or more complete time columns; above that, a subset of particles reads only the blocks it touches. Outputs written before this layout (August 2026) keep a single particle block.

15.2 What is written

The output follows the CF conventions (the Climate and Forecast metadata conventions) for trajectories and carries the global attribute `kelp_output_format` with the value 2; a file without it was written by an earlier engine (Section 15.5). It is organized as a set of two-dimensional arrays indexed by particle and by record, the *trajectory* and *time* dimensions (Figure 15.1), and

both dimensions have a coordinate variable of the same name. Every run writes the position — longitude, latitude and depth (positive down) — the particle status as a coded flag (1 alive, 2 beached, 3 outside, 4 not released), the record time, a trajectory identifier, and, once per particle, the release position and release time. The identifier is the particle's index in the whole release, so a shard of a sharded release (Section 15.4) numbers its particles from its first global index. Several further variables are written when they are present: the velocity as `vel_u`, `vel_v` and, in three dimensions only, `vel_w` (described below), the failure reason (Section 13.2), and one array per sampled scalar field (Chapter 7). Positions, release positions and horizontal velocities are always written in geographic coordinates, un-rotated from any computational frame (Chapter 8). Where nothing applies the arrays hold a fill value: a missing value (NaN) for positions, velocities, sampled fields and release variables, and zero (no failure) for the failure reason; the rows of particles not yet released are missing values. Positions carry the CF names and units (`longitude`, `latitude`, and `depth` with `positive = "down"`), the status, failure reason and sampled fields name the positions as their coordinates, and both backends write the same attributes. The one known departure from CF is the order of the dimensions: CF wants the unlimited record dimension to be the outer one, while here it is the inner one, so that the track of one particle is contiguous on disk.

A record is written after every step g for which $g + 1$ is a multiple of `output_freq`, counting steps from the start of the run across any restarts. The release positions are therefore not part of the records — a particle first appears at the end of a step, at least one step after its release — and are kept in the separate release variables instead. The size of the output is proportional to the number of particles times the number of records; for a large pulsed release this product, not the number of live particles, sets the size, because the frozen particles occupy rows in the pre-allocated arrays. A run with several realizations writes one file per realization, with the suffix `_r`N.

The record time. The time is stored in seconds since the first record of the forcing time axis as it appears in the file, and the attributes of the time variable name that record's date as the epoch and the forcing's calendar (for example `noleap` or `360_day`), so that a reader such as xarray decodes the values to dates. The record written after global step g holds

$$\tau_g = \tau_{\mathrm{run}} + (g + 1)\,\Delta t\, s_{\mathrm{fb}}, \tag{15.2}$$

the time at the *end* of the step, in both time modes, where τ_{run} is the time at which global step zero started. Let g_{s} be the global index of the first step of the current call (zero for a run that starts afresh). In the interpolated mode $\tau_{\mathrm{run}} = t_{\mathrm{first}} - g_{\mathrm{s}}\,\Delta t\, s_{\mathrm{fb}}$, with t_{first} the time at which the call's first step starts (the requested start date, or the time of the start snapshot); in the discrete mode τ_{run} is the time of the snapshot g_{s} records before the call's start snapshot, the one the run started from. Under time-averaged forcing these are the shifted midpoints (Section 14.5), so the dates are window centres, while the epoch stays the first unshifted record. Because τ_g depends only on the global step and the start of the whole run, the segments of a resumed chain carry exactly the labels of an uninterrupted run. In the discrete mode the label advances by Δt per step; the time-step check of Section 10.1 admits a small difference between Δt and the record spacing, by which the label may drift from the stored stamps. When the forcing is cycled the time does not wrap: past the first period the dates lie beyond the end of the forcing, and an attribute records the period. A forcing without a calendar epoch — a bare unit, or an index axis with a declared spacing — gives plain seconds from its first record, with an attribute saying so.

Release positions and times. For every particle the output holds its release longitude, latitude and depth — the seed position, after the fold into a periodic window when one is set —

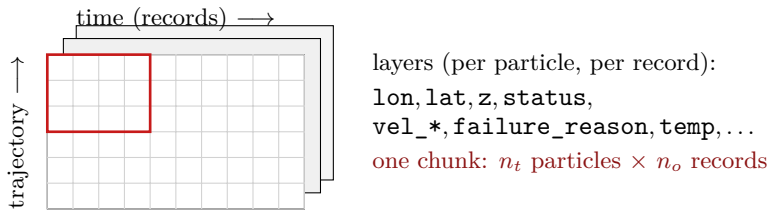


Figure 15.1: The output data model: two-dimensional arrays indexed by particle (trajectory) and by record (time), one such array per stored variable, with the time and the trajectory identifier as one-dimensional coordinate variables and the release position and time as one-dimensional arrays over the particles. The position and status are always present, and so are the velocities of a tracking run (NaN where none was evaluated); failure reasons and sampled scalars are added when available. In a Zarr store each array is split into chunks of Equation (15.1).

and its release time,

$$\tau^{\mathrm{rel}} = \tau_{\mathrm{run}} + \max(k_{\mathrm{rel}}, 0)\,\Delta t\, s_{\mathrm{fb}}, \tag{15.3}$$

the start of the step in which the particle is activated, with k_{rel} the rounded release step of Equation (13.2) (zero without a delayed release). A particle never activated within the run still gets its scheduled time, which then lies after the last record. The release positions travel with the particle state and are stored in every checkpoint (Section 15.3), so each segment of a resumed chain writes the same values; they are missing only for a run resumed from a checkpoint of an earlier engine.

Velocities. The velocity of a record is the one the integrator evaluated for the step that ends at the record, at the step's start position and time (for RK45, at its end); it therefore names no coordinates. `vel_w` is the rate of depth change, positive down, and on a sigma mesh the dia-sigma flux (Chapter 6). All components are flow velocities, with the same sign in forward and backward runs (the integrators carry s_{fb} times the velocity; the record removes the factor). A velocity is written only where the particle was alive through the whole step — alive after the release activation and before the step, and alive after the step and the beaching hook. The record in which a particle dies or strands, every later record of a beached or outside particle, a particle not yet released and a particle resuspended during the step hold missing values. A two-dimensional run writes no `vel_w`.

Provenance. Every output carries global attributes that record how it was produced: the integrator, the step, the mesh type, the direction, the start index and step count, whether the run is three-dimensional and time-interpolated, the temporal reconstruction and time averaging, the engine version, the exact-anchor flag and the resolved grid-hash cell with the rule that chose it (Section 4.2); in three dimensions the vertical-velocity combine and sign convention. Options that are off by default add their parameters only when they are on — pulses, diffusion, beaching, the adaptive scheme's tolerance, periodic windows, sampling, a non-default horizontal scheme or element search, single-precision fields, read-ahead and the rotation angles — so that an output written with the defaults carries none of them. A run is repeated from these attributes; the recorded grid-hash cell, for instance, can be pinned to reproduce the anchors of an earlier run. Two of them concern the additions of Sections 6.4 and 14.5. A run with `mode.uv_vertical: linear` has the global attribute `uv_vertical = "linear"`, and a run in the default layer mode has no such attribute, so its output is unchanged. The attribute is the proof that the option was on: a misspelled key is ignored silently, like every unknown key, and leaves no attribute behind. A FESOM 1.4 run, whose velocities are always depth-linear and which refuses the option, has no `uv_vertical` attribute; its time averaging reads `"resolved_by_reader"`.

15.3 Checkpoints and restart

When `checkpoint_freq` is set, the engine writes a checkpoint after every step g for which $g + 1$ is a multiple of it, overwriting the previous one, and a final checkpoint at the end, stamped with the last step actually executed. The checkpoint lives in the output file itself as arrays named `restart_*`: the positions, statuses and nearest nodes, the release positions, the step and forcing indices, and — when the run uses them — the failure reasons, release steps, carried elements, the counts of re-wetted steps and the frame angles. Positions and release positions are stored in the computational frame. The arrays are created by the first checkpoint, so an output written without checkpoints holds none of them. A flag, `restart_valid`, is created before any other checkpoint array and written as zero, cleared again before every later checkpoint, and set only after all arrays are written; a checkpoint interrupted mid-write, the first one included, is therefore detected rather than trusted. Checkpoints are off by default: a run killed at a wall limit without them leaves nothing to resume from.

Resuming is done by the calling program rather than by a configuration key. It loads the checkpoint (`load_restart`), checks its frame against the mesh, and continues with the next step index, the saved counts of re-wetted steps and the forcing position; in the interpolated mode it also supplies the simulation time, which the checkpoint does not store. For the record times of the continuation to equal those of the uninterrupted run (Equation (15.2)), the program passes the same forcing times as the first call — under time-averaged forcing the shifted midpoints — and, when a cycled forcing is read in the discrete mode, the start index counted on without wrapping (in the discrete mode the forcing index stored in the checkpoint is that unwrapped index, so the index after it continues the run). The release positions need no step of their own: they come back with the particle state. A program that splits a resumed state into parts slices it with `ParticleState.subset`, which slices every per-particle array, the release positions included. With the exact-anchor guard the nearest nodes of the resumed state are recomputed once.

One vertical rule per trajectory. The rule of `mode.uv_vertical` cannot change inside one trajectory, since a resumed segment under the other rule would join piecewise-constant and depth-linear velocities in one path. A checkpoint therefore records its rule: the embedded checkpoint through the output's `uv_vertical` attribute, a standalone restart file through its attribute `restart_uv_vertical`, written only for `linear`. `load_restart` returns the rule as `uv_vertical`, with `None` meaning `layer` (a layer run, or a file written before the option existed), and the resume recipe checks it next to the frame: a resume with the other value is refused, and the message names both rules.

Resumable chains. A run longer than one allocation of a batch system is split into a chain of jobs. Each job resumes from the newest standalone restart file — written with `write_restart_file`, which writes the same arrays to a separate NetCDF file under a temporary name and moves it into place only after a clean close, so a failed write keeps the previous restart — writes its records into its own output segment, and drops a numbered restart at every checkpoint (Figure 15.2). At the end the segments are concatenated along time. The concatenated chain equals one uninterrupted run bit for bit, because everything that depends on time — the random draws of diffusion and beaching, the release activation, the record and checkpoint cadence — is keyed to the global step index, never to the step within a job. The library supplies these pieces; the driver that submits the chain belongs to the experiment.

15.4 Merging a sharded release

A large release split into contiguous blocks of seeds and run as parallel processes (Section 17.9) produces one Zarr output per shard. The merge tool concatenates them along the trajectory

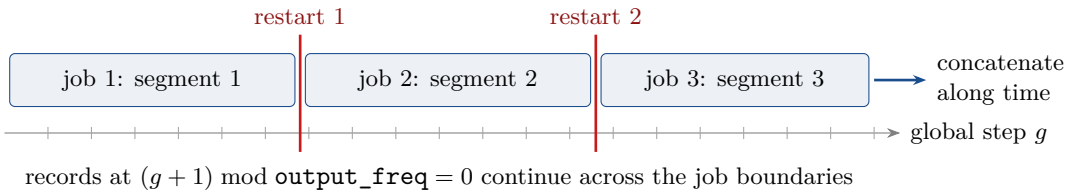


Figure 15.2: A resumable chain. Each job resumes from the newest numbered restart, writes its own segment and drops restarts as it goes; the segments are concatenated at the end. Because every cadence and every random draw is keyed to the global step, the result equals one uninterrupted run.

axis in shard order, after checking that the shards share their time axis, their variables and every attribute that defines the run. A run with a single release merges into an output identical in its data to the unsharded run, stochastic diffusion and beaching included, because each shard keys its random draws by the particle's index in the full release; the merge refuses shards passed out of that order whenever the run draws random numbers (and warns for a deterministic run). A pulsed release is refused. Shards that are independent by design — a pulsed run split through its restart, one seed per shard — merge on an explicit option that lifts the offset order, the pulsed refusal and the seed equality, refuses shards that share any seed of a random feature they use over the same particle indices, and records each shard's particle count, offset and seeds in the merged store. Checkpoint arrays are not merged. The merge keeps the time attributes and the release variables, and refuses shards that differ in the output format, in the name of the record dimension, or in the units or calendar of the time; shards of the earlier format merge as before.

15.5 Outputs of earlier engines

Outputs written before release 1.0 are in output format 1. Such a file has no `kelp_output_format` attribute, and its record dimension is called `obs`. In format 1 the record time has no epoch: in the discrete mode it is the index of the snapshot the step started from, one step behind the position it labels, and in the interpolated mode the seconds from the first record. Format 1 has no release variables; a two-dimensional run writes a vertical velocity of zeros; a particle's velocity is written in the record in which it dies or strands and is zero afterwards; the discrete mode samples scalar fields at the start of the step; and the checkpoint arrays exist in every file, holding fill values when no checkpoint was written. These files remain readable by `load_trajectories`, `load_restart`, the merge tool and the visualization helpers; there is no converter.

Source. The writer `TrajectoryWriter` and its NetCDF and Zarr backends, the embedded checkpoint, `write_restart_file`, `load_restart` and `load_trajectories` are in `writers.py`; the record cadence and checkpoint scheduling are in the time loop of `tracking.py`, the record time and the release variables in `_record_clock_origin` and `_release_arrays` there; the epoch and calendar of the time come from `time_axis_epoch` (`readers/__init__.py`); the velocity mask is `mask_inactive_velocity` (`core/release.py`); the frame check on resume is `_validate_restart_frame` and the rule check `_validate_restart_uv_vertical` (`tracking.py`); sharding and merging are `utils/sort_seeds.py`, `utils/split_seeds.py` and `utils/concat_trajectories.py`.

Chapter 16

Verification and Validation

The numerical methods of the preceding chapters are exercised by an automated test suite of about 4700 tests, plus 47 slow ones that need external data and run separately. The large majority guard low-level invariants — array shapes, configuration parsing, byte-for-byte reproducibility — and are not individually informative. This chapter does not enumerate them. Instead it describes the tests that verify the numerical *physics* against independent references, organized by the kind of evidence they provide, and names each so that a reader may locate it. The goal of these tests, and of this chapter, is to characterize whether the engine computes what it is specified to compute, not to assess the physical realism of any simulation. Validation suites with pinned tolerances are themselves tested: the code or the data is perturbed on purpose in a number of ways, and each pin must fail; where a result has a natural ordering — a finer resolution closer to the reference — the ordering is pinned as well.

16.1 Closed-form analytic flows

The most direct evidence comes from flows whose trajectories are known in closed form, for which the computed path can be compared against the exact solution. The suite includes uniform flow, helical rotation (with a test that the orbital radius is conserved), a Burgers vortex, inertial oscillations, an Ekman spiral, and incompressible linear strain, whose exponential trajectory is matched to $2 \cdot 10^{-6}$ degrees horizontally and 8 mm vertically. Linear strain, solid-body rotation and downwelling are repeated on quadrilateral meshes. In each case the particle is integrated through the analytic field and its position checked against the formula, to tolerances far below the grid scale. The edge least-squares scheme is checked to be exact for linear fields inside triangles and quadrilaterals, to carry a flow parallel to a wall exactly under free slip, and to give the same result on either side of the dateline. The adaptive scheme's coefficients are checked against the Dormand–Prince order conditions, and its accuracy on a vortex against the tolerance and against RK4.

16.2 Conservation and statistics

A second class of tests checks conserved quantities and statistical laws that the numerics must respect. For the three-dimensional vertical advection, the key tests exploit continuity: in a steady, divergence-free flow a streamfunction exists and must be conserved along trajectories. Integrated through an analytic downwelling cell, particles keep the streamfunction to 0.15 %; through an Ekman-pumping cell, to 3 % over 200 steps with the default vertical combine and 5 % over 100 steps with every combine. These are what verify that the vertical-velocity interpolation neither creates nor destroys volume.

For the stochastic schemes, the ensemble variance is checked to grow as $\sigma^2 = 2K_h t$ (Chapter 12). On the sphere, the Monte-Carlo mean change of $\sin\phi$ under the horizontal kicks is checked against the generator of the spherical random walk, $-2K_y\, h \sin\phi / R_M^2$ — the walk on the longitude–latitude chart gives half of it — and particles are checked to cross a pole cleanly. In the vertical, a uniform column in a diffusivity that varies tenfold must stay uniform: after 5000 steps each of eight depth bins stays within 20 % of its share, forward and backward; without the drift term the density would follow $1/A_z$ instead.

16.3 The depth-linear horizontal velocity

The depth-linear rule of Section 6.4 is checked against answers known in closed form, through the public run path: synthetic FESOM2 files are read by the FESOM2 reader and tracked by the ordinary run entry, on node layout (barycentric) and on element layout (least squares, edge least squares, element barycentric), with element beds that include partial bottom cells. Five properties are pinned. First, a velocity independent of depth — per-column constants that vary from column to column and in time, with NaN stored below each column's last usable point, so that a read there would show — gives the same trajectory in linear and in layer mode at every record: bit for bit wherever every member reads a stored value (above the first centre, at a centre, below the last one, a particle exactly at its members' bed), and within a few units in the last place elsewhere. Second, a velocity linear in depth, $u = u_0 + g\,(\phi - \phi_c) + b\,z$, is followed exactly: the linear run moves by the closed form, also with a uniform vertical velocity that moves particles across layers (exact for RK2 and RK4), while the layer run is off by $b\,(p_k - z)\,T$, with p_k the centre of the particle's layer. Third, for the smooth profile $u = G(x)\cos(\pi z/H)$ on 10, 20 and 40 layers the largest reconstruction error falls by a factor of four per halving of the layer thickness in linear mode and by two in layer mode — second order against first. Fourth, the same field tracked with RK4 shows the same orders in the end positions of fifty particles, with the time-step error below 1 % of either mode's error. Fifth, a uniform flow along a straight trench, below the bed of the shelf beside it, keeps its speed in linear mode, where the shelf is a wall, and is slowed in layer mode, which reads the shelf's zeros; with the no-slip ghost the edge least-squares scheme brakes it as at a coast.

The sigma counterparts use synthetic FESOM-C files read by the FESOM-C reader, with a bed sloping in both directions and a rising, tilted surface. Per-column constants give the layer run bit for bit beyond the outer centres and within a few units in the last place between them; the profile $u = G(x)\cos(\pi s)$ converges at second order in the sigma layer thickness on uniform and on stretched sigma levels, layer mode at first order. A separate suite checks the sigma step by hand: with a bed linear in longitude and latitude, a dia-sigma velocity linear in s and u linear in s, every interpolator is exact, so each expected position is a hand-written combination of stages, and a probe on the interpolators records the fraction s that every stage hands them — its own position, time and column height (Section 6.4). The fused depth-linear kernels reproduce the unfused path bit for bit, with walls at depth included.

These suites received a mutation pass. Eighteen deliberate defects were put into a copy of the code one at a time — among them the two blend weights swapped, the wet test made strict, a dry element kept in the least-squares fit or in the wet-member mean, the too-few fallback returning zero, the RK2 second stage read at the entry depth, the fused Euler twin reading the layer row, the bracket reading the nearest point, every vertex counted wet, the sigma points placed at the interfaces instead of the centres, and a stage fraction handed in metres — and each turned specific tests red by the expected mechanism, for instance first-order convergence where second order is pinned, or a trench flow 18–50 % too slow.

Source. `tests/test_depth_linear_analytic.py` (the five properties, their sigma counterparts and the mutation record in its docstring); `tests/test_integrators_depth_linear_sigma.py` (the sigma step by hand); `tests/test_interp_depth_linear.py` and `tests/test_interp_depth_linear_elements.py` (the rounding bounds of the interpolators); `tests/test_fused_3d_depth_linear.py` (the fused twins).

16.4 Contracts with model data

Some properties can only be checked against real model output; these tests run when the data are available and are skipped otherwise. The sign of the vertical velocity (Section 6.1) is tested end to end — a file value of $+10^{-4}$ m/s positive upward lifts a particle 8.64 m in one Euler day — and against physics: in the equatorial Pacific at 50 m the mean vertical velocity read from native FESOM2 output, and from its HEALPix remap, must be upward. The HEALPix remaps are validated against the native model: the same seeds run for thirty daily RK2 steps on the native mesh and on three HEALPix resolutions, and the tests pin the ratio of median displacements, the median separation of paired trajectories, the mean drift, the survivors, the ordering — the finest remap closest to the native run — and the equality of the time axes, which catches a uniform one-day offset that the statistics alone would miss.

The sigma step of Section 6.7 has exact solutions: with a tide, a sloping bottom, a horizontal flow and a uniform dia-sigma velocity, the fraction of the column is linear in time, and Euler, RK2 and RK4 reproduce it to a relative 10^{-9}, forward and backward, with uniform and stretched sigma levels; with a dia-sigma velocity linear in the fraction the three schemes converge at orders one, two and four.

The FESOM 1.4 reader is checked against published output: the monthly means of a CMIP6 experiment of the AWI-CM-1-1-LR model, in the directory layout of the CMIP6 archive, read from the directory named by `KELP_TEST_FESOM14_DATA`. The tests pin the mesh recovered from the cell polygons (126 859 nodes and 244 660 triangles, checked independently of the recovery, with `depth` equal to the depth of the last wet level at every node); the absolute time axis, every resolved instant equal to the centre of its time bounds decoded from each file's own epoch; the first-step velocity of probes placed on mesh nodes a quarter of a layer below a level, computed by hand as the lerp in depth between two levels of the records that bracket the start; three-month runs with linear and PCHIP time interpolation, whose first-step velocities follow the order time first, then depth (Section 6.4), including probes released after the PCHIP window has moved; the sign of W against the equatorial upwelling; the separation of level-linear from layer-constant reading; and the equivalence of the two file forms — the published fields and the recovered mesh rewritten as an ASCII mesh and as raw-writer files give bit-identical trajectories. The raw form and the ASCII mesh are thereby checked against the model's description of its formats and on real geometry, not against real raw output. A mutation pass — the level points moved half a layer, the sign of W inverted, the resolved axis shifted by a day, the midpoint shift applied a second time, the depth and time blends in the reversed order, among others — failed the pins named in the test module.

16.5 Reversibility and determinism

A further class checks structural properties of the implementation. Because the deterministic integration direction enters only as a sign on the velocity (Section 2.5), a forward run followed by a backward run should return a particle to its origin; the suite verifies this on analytic fields, including under the monotone-cubic reconstruction, to 10^{-6} degrees in longitude, 10^{-9} in latitude and 1 µm in depth. The same property holds empirically on the *core2* mesh: a clean one-year surface round trip returns one hundred particles to a median of 0.16 km (Figure 16.1).

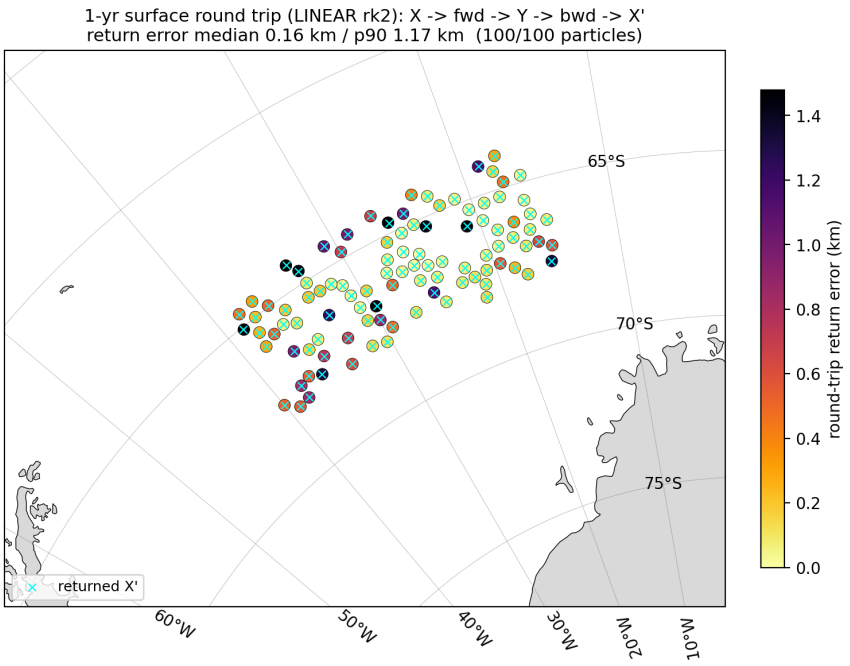


Figure 16.1: A controlled reversibility check on the *core2* mesh, measured in June 2026: one hundred particles are integrated forward for one year and then backward for one year with the same scheme (RK2, linear reconstruction in time), and the return error between the start and end positions is mapped. The median return error is 0.16 km (ninetieth percentile 1.17 km), far below the local grid scale of tens of kilometres (a three-dimensional run at 5 m depth in the Weddell Sea, engine of June 2026; the vertical motion was negligible). Larger separations over long windows reflect Lagrangian chaos and boundary clamping rather than a defect of the scheme.

With diffusion the backward run is the adjoint process, not the reversal of the forward one (Section 12.1), and the tests check that the backward drift is kept.

Determinism is checked separately, as bit-for-bit identity. The fused and unfused kernels must agree for every fused family, including the three-dimensional barycentric RK2 kernel and the monotone-cubic kernels (Chapter 11); the exact nearest-node search must match a brute-force search on random geometries, and the nearest-node kernels must match frozen copies of their earlier versions, ties included; element tracking must give the same verdicts as the ring search. A resumed run must reproduce the uninterrupted one — with beaching, vertical diffusion, element tracking and sampling — and must record the same steps; a checkpoint interrupted mid-write must never load; a sharded release merged back must equal the unsharded run in its data; and a run with the shared snapshot cache must equal one without it. The counter-based random numbers (Section 12.6) are independent of the thread count by construction: each draw depends only on its key.

16.6 Time-dependent benchmark flows

A constant zonal flow, a time-dependent moving eddy and a decaying eddy are integrated and compared with their closed forms. A step-refinement test confirms the second-order ceiling that linear-in-time reconstruction imposes on the whole scheme: halving the step cuts the RK4 error by a factor between 3.5 and 5, not sixteen — the $O(\Delta t^2)$ cap of Chapter 10 that the monotone-cubic reconstruction lifts. Two diffusion benchmarks check the moments of Brownian motion and the skewness that a spatially varying diffusivity must produce. These suites are marked slow and run outside the default test run.

The engine adopts a metre-per-degree constant that differs from that of the original Fortran

code, which intentionally breaks bit-parity with it; the Fortran comparison is therefore not part of the suite, and the Fortran prototype and its reference trajectories are not part of this repository.

Source. The physics-validation tests live in `tests/`: the analytic-flow and stream-function families (`test_analytical_3d_benchmarks.py`, `test_downwelling_psi.py`, `test_3d_analytical.py`, `test_quad_mesh_benchmarks.py`); the random walks (`test_spherical_random_walk.py`, `test_az_diffusion.py`); the data contracts (`test_w_sign_contract.py`, `test_healpix_native_twin.py`, `test_fesomc_w_contract.py`, `test_fesom14_published_lr.py`); the bit-identity pins (`test_fused_3d_bary.py`, `test_exact_nearest.py`, `test_node_xyz.py`, `test_edge_walk.py`, `test_checkpoint_integrity.py`, `test_sharding.py`); and the reconstruction-order tests (`test_temporal_reconstruction.py`); the benchmark flows and diffusion benchmarks are the suites marked slow.

Chapter 17

Computational Cost and Performance

The cost of a run is governed by a small number of terms, and knowing them is what lets a study be sized and configured sensibly. This chapter sets out the per-step budget, the per-particle cost of the schemes and of the nearest-node search, how the work spreads over threads, processes and nodes, what bounds memory, and how the cost is measured. Unless stated, the timings were taken on compute nodes with two 64-core AMD EPYC 7763 processors (128 cores, 256 GB), with the engine's own benchmark scripts and verbose timing, and are collected in `docs/performance.md`; the nearest-node rows (the node coordinate table and Table 17.2) were measured on a login node and are kept in `benchmarks/nearest_scan_table.md`. Absolute values depend on the hardware and the mesh; the structure of the budget does not.

17.1 The per-step budget

A step consists of four phases, and `kelp run -verbose` prints each of them for every step (`io=`, `step=`, `beach=` when beaching is enabled, `rec=`) together with their means at the end of the run:

$$t_{\text{step}} \approx t_{\text{read}} + t_{\text{blend}} + N\, c_{\text{kernel}} + t_{\text{record}}, \tag{17.1}$$

with N the number of live particles. The *read* term loads the snapshots the step needs; it is set by the size of the field and by how often a new snapshot is needed, and it is independent of the particle count and of where the particles are. The *blend* term forms the velocity at the step's time from the bracketing snapshots (Chapter 10); it is paid only for the fields a consumer actually reads, because the payload of a step blends a field on its first access and not before. The *kernel* term is the tracking itself — locate, interpolate, advance — and grows with the number of live particles; dead and not-yet-released particles cost nothing. The *record* term writes a trajectory record every `output_freq` steps. The phases inside the kernel are not separable in the fused kernels (Chapter 11); the verbose split is the finest the engine reports.

Two consequences follow. A step that reads no snapshot and writes no record measures the kernel alone; the *steady step* of a run is therefore defined as the median, over the steps after the first that read no snapshot, of the tracking phase — and, for a run split over processes, of the slowest process, which sets the completion time. And because the kernel cost of a step does not depend on the time step, the cost of a model day grows linearly with the number of sub-steps per snapshot interval: halving `dt` doubles the run.

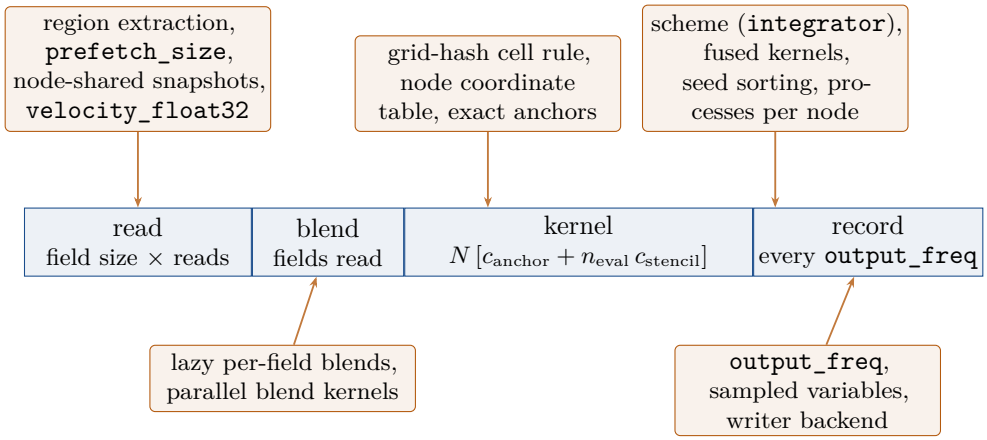


Figure 17.1: The per-step budget of Equation (17.1) and the lever that acts on each term. The kernel term splits into the nearest-node anchor (c_{anchor}, Section 17.3) and the stencil work, repeated once per velocity evaluation of the scheme (n_{eval}).

17.2 The per-particle kernel cost

Schemes. The kernel cost grows with the number of velocity evaluations per step. In two dimensions, per particle and step at sixteen threads, forward Euler costs about 1.1 µs, the midpoint method about 6.6 µs and RK4 about 22 µs (a June 2026 measurement, before the exact-anchor guard and the node coordinate table of Section 17.3; the ladder, not the absolute values, is the point). The monotone-cubic temporal reconstruction is blended inside the two-dimensional least-squares kernels and adds only a small fixed offset, so the midpoint method with PCHIP is the cost-effective way to buy temporal accuracy (Chapter 10).

Three dimensions. The fused three-dimensional midpoint kernel for node-layout barycentric velocities on z-level meshes (Chapter 11) evaluates the whole step, including the four-snapshot PCHIP blend, per particle; on steady steps its cost per particle does not depend on the mesh (Section 17.5). The unfused three-dimensional paths — among them RK4, sigma meshes, and `w_interp` other than nearest or barycentric — blend whole fields per stage. Those blends run in a parallel kernel over contiguous double-precision fields, and RK4 reconstructs its repeated midpoint stage once: a four-snapshot PCHIP blend of a global field of 47 layers by 244 000 elements (an element velocity field of the *core2* mesh) takes 46 ms at eight threads instead of 953 ms serially, and a full RK4 stage set (u, v, w, surface) about 275 ms at eight threads and 136 ms at 32. The whole-field reconstruction is therefore no longer a multi-second floor of three-dimensional PCHIP runs.

Source. Parallel blends: `_pchip_blend4_flat` in `numerics/temporal_reconstruct.py` and `_blend` in `core/io.py`; the lazy payload is `LazyLerpField` in `core/io.py`; the stage dedup is in `make_stage_vels_recon`.

Code vs. documentation. The parallel linear blend requires double-precision inputs: with `velocity_float32` the linear time interpolation falls back to a serial blend (about 0.25 s instead of 0.05 s per global-size field), and the fused three-dimensional step pays 15–20 % for the conversion on gather. The PCHIP blend converts first and keeps its parallel path. Single precision is a memory lever, not a speed lever.

The depth-linear rule. Reading the horizontal velocity linearly in depth (Section 6.4) costs two data points per stencil member instead of one, a search of the member's column for the bracket and a test of the member against its bed. Table 17.1 gives what this adds to one three-dimensional step, timed with the same arrays read in layer and in linear mode on the

step	time blend	layer [ms]	linear [ms]	ratio
node layout, barycentric, fused RK2	linear	9.59	10.52	1.10
	PCHIP	12.51	12.45	1.00
element layout, least squares, fused Euler	linear	7.30	7.75	1.06
element layout, least squares, unfused RK2	linear	24.18	24.79	1.03
	PCHIP	37.21	36.18	0.97
sigma, element least squares, unfused RK2	linear	32.54	31.92	0.98
	PCHIP	36.57	37.69	1.03
sigma, node barycentric, unfused RK2	linear	36.51	37.51	1.03
	PCHIP	34.49	35.71	1.04

Table 17.1: Cost of the depth-linear rule per three-dimensional step: median milliseconds per step in layer and in linear mode and their ratio, with the walls at depth of Section 6.4; 100 000 particles, 128 threads, two AMD EPYC 7763 processors. The Euler step reads its start field only and has no PCHIP row.

compute node above (128 threads, 100 000 particles at random positions, a synthetic HEALPix dual mesh of 49 152 nodes or its triangulated twin of 98 300 elements, 21 layers down to 4000 m with varied bathymetry, a six-hour step; the median of fifteen interleaved steps per mode, measured in October 2026). With a linear blend in time the rule adds about 10 % to the fused node-layout RK2 step, 6 % to the fused element-layout Euler step and 3 % to the unfused element-layout RK2 step; with the four-snapshot PCHIP blend the difference lies within the run-to-run spread, presumably because the time blend then dominates the work per member. On a sigma mesh, where every step is unfused, the rule in the sigma fraction costs between nothing measurable and 4 %. In the element workload about 16 % of the particles had a dry element in their least-squares ring and about 4 % fewer than three wet ones, so the walls and the area-mean fallback were exercised. Reading the records and the once-per-record check are not included.

Source. `benchmarks/single_node/depth_linear_cost.py` and its rows in `benchmarks/single_node/depth_linear_cost.md`; the kernels compared are `_fused_rk2_step_3d_bary_tinterp` and `_fused_rk2_step_3d_bary_pchip` against their `_dl` twins, `_fused_euler_step_3d` against `_fused_euler_step_3d_dl`, and `_rk2_step_unfused` with the layer and the depth-linear interpolation cores.

17.3 The nearest-node anchor

Every evaluation site starts from the nearest mesh node (Section 4.1), found by scanning the nodes of a 3×3 window of grid-hash cells. The scan is linear in the number of nodes in the window, so its cost depends on how many nodes a cell holds, and under the exact-anchor guard (`mesh.exact_nearest`, the default) on how often the window must be widened before the nearest node is proven. Three measures keep this term small.

The cell is chosen by measurement. The automatic cell of the reader is $2\sqrt{A/N}$, with A the bounding-box area of the mesh in square degrees, and at least 0.05°. On a mesh whose resolution varies by orders of magnitude, a cell sized for the coarse part holds thousands of nodes in the refined part. Since September 2026 every run chooses the cell itself (`mesh.grid_hash_cell_rule: occupancy`): it samples up to 200 000 element centres, finds each one's exact nearest node once, and, for candidate cells $\ell_{\mathrm{legacy}}/\sqrt{2}^{\,k}$, evaluates a per-query cost estimate — the nodes of the window plus nine cells plus one certificate for a provable hit, the nodes and cells of the widened rectangle and one certificate per widening otherwise. It leaves the reader's cell only when the best eligible candidate (at most 20 % widened queries, at most 128 MB of hash offsets) promises at least a 2.75-fold gain. Quasi-uniform meshes keep the reader's cell (the

Table 17.2: Cost of exact nearest-node anchors: one fused step, 400 000 particles, eight threads, milliseconds, with the node coordinate table. "Off" is the approximate in-window anchor; "unfused" is the unfused path under the guard.

Configuration	off	on	unfused
structured mid-latitude mesh, automatic cell, RK4	115	157	230
structured mesh at 80°–88° N, automatic cell, RK4	102	201	278
same, cell three times below the node spacing	81	371	455
HEALPix, polar band 89°–89.9° N, RK4 barycentric	531	855	961
HEALPix, global uniform seeds, RK4 barycentric	554	541	1019
HEALPix at 80°–88° N, fused 3-D midpoint barycentric	398	385	406

predicted gain is about two, which buys nothing measurable on a step); structure-resolving meshes take a smaller one: a coastal FESOM-C mesh goes from 0.058° to 0.014° (search 1.7 times faster) and an 11.5-million-node global mesh from 0.145° to 0.051° (1.7–2.6 times). An explicit `mesh.grid_hash_cell_size` always wins; the chosen cell and the rule are written to the output metadata.

The node coordinate table. The scans read unit-sphere coordinates of the nodes from a table filled once when the hash is built, instead of evaluating trigonometric functions per candidate. The pure scan is 6.3 times faster (3.2 µs to 0.5 µs per query on one thread), a two-dimensional midpoint step phase about twice as fast, at 24 bytes per node and process; the trajectories are bit-for-bit unchanged.

The cost of exact anchors. Table 17.2 gives the cost of the guard for 400 000 particles at eight threads. It is a few per cent on HEALPix meshes and on the fused three-dimensional kernel, about 40 % on a structured mid-latitude mesh even with every anchor provable, and grows where many windows must widen — at high latitude, where a degree of longitude shrinks, and with a cell well below the node spacing. In every case the fused kernel with exact anchors stays faster than the unfused path.

Source. Cell rule: `choose_grid_hash_cell`, `apply_grid_hash_cell_rule` and the estimate `_gh_sample_stats`; table: `_node_xyz_fill`; exact search: `_grid_hash_nearest_exact`; all in `core/mesh.py`. Benchmarks: `benchmarks/exact_nearest_cost.py`, `benchmarks/nearest_scan_table.md`, `benchmarks/grid_hash_cell_rule_table.md`.

17.4 Threads and processes on one node

The kernels parallelize over particles. Two-dimensional fused kernels with single-precision velocities scale steeply, about 38-fold for the midpoint method from one to 128 threads on a global mesh. The fused three-dimensional midpoint kernel is not memory-bound: it gains 6.2-fold from 16 to 128 threads. The unfused three-dimensional paths are bound by memory bandwidth and saturate at 16 to 32 threads. The thread count must always be set explicitly (`NUMBA_NUM_THREADS`); left unset, the kernel library occupies every logical core and small kernels slow down markedly.

A node can also be split into several processes, each tracking a contiguous share of the particles and pinned to its own NUMA domain. Table 17.3 shows the effect for six million particles in three dimensions on a zoom-9 HEALPix mesh with node-shared snapshots: the same 128 cores take a 22 % shorter step as sixteen processes of eight threads than as one process of 128 (29 % more particle-steps per second).

Table 17.3: Process layouts on one node: 6.0 million particles, fused three-dimensional midpoint barycentric kernel, zoom-9 HEALPix mesh (2.2 million wet surface nodes), 128 threads in total, node-shared snapshots. Steady tracking step of the slowest process.

Processes × threads	1 × 128	2 × 64	4 × 32	8 × 16	16 × 8
Tracking step (s)	0.576	0.540	0.489	0.478	0.447
Throughput (10^6 particle-steps/s)	10.4	11.1	12.3	12.6	13.4

Table 17.4: Mesh size: 6.0 million particles, one 128-thread process, fused three-dimensional midpoint barycentric kernel, HEALPix meshes of zoom 5 to 11.

Zoom	5	6	7	8	9	10	11
Wet surface nodes (10^6)	0.009	0.036	0.14	0.57	2.2	8.9	35.5
Tracking step (s)	0.742	0.814	0.628	0.803	0.638	0.635	0.908

17.5 Mesh size and particle count

The kernel does not see the size of the mesh: a particle's work is a constant-size stencil around its position. Table 17.4 confirms this for the fused three-dimensional step over a 3765-fold range of HEALPix meshes with the same six million particles — the step varies by at most 1.45 times, and not monotonically; the search window holds a median of 57–65 nodes on all seven meshes, so the variation is not the nearest-node search. The step grows slightly less than linearly with the number of particles (Figure 17.2): from 0.144 µs per particle-step at three million to 0.106 µs at 24 million on 128 threads, a log–log slope of 0.88.

17.6 Many nodes

Particles are independent, so a release splits over nodes as contiguous shares, one or several processes per node, each reading its own snapshots or sharing them within the node (Section 17.8). Figure 17.3 shows weak and strong scaling on the zoom-11 HEALPix mesh (35.5 million wet surface nodes, 69 layers), one 128-thread process per node — a zoom-11 process needs about 146 GiB, so one fits a 256 GB node. With six million particles per node the step stays within 90 % of the one-node value up to sixteen nodes (96 million particles in 0.85 s, 113×10^6 particle-steps per second). A fixed release of 24 million particles runs 12.6 times faster on sixteen nodes; on the zoom-9 mesh, split as eight processes per node, six million particles run 9.7 times faster on sixteen nodes, where each process holds only 47 000 particles.

Very large releases keep the same throughput. Tracked for a few steps without any output, 10^9 particles on sixteen nodes step in 7.6 s and 10^{10} particles on sixteen 512 GB nodes in 73 s — both about 1.3×10^8 particle-steps per second.

17.7 Memory

A process holds the mesh, the node coordinate table (24 bytes per node), the hash offsets (at most 128 MB under the cell rule), the snapshots of the read window, and the particle state. For releases large enough that the particles dominate, the peak resident memory of a three-dimensional midpoint process without output was measured as

$$M_{\text{peak}} \approx 61\,\text{GiB} + 181\,\text{B} \times N_{\text{process}} \qquad \text{(zoom-11 mesh, fused 3-D midpoint, no output)}, \quad (17.2)$$

fitted through one-node releases of 10^9 (230 GiB) and 4×10^9 particles (735 GiB); it predicted 166.7 GiB for 625 million particles per process where 166.2 GiB was measured. Below about

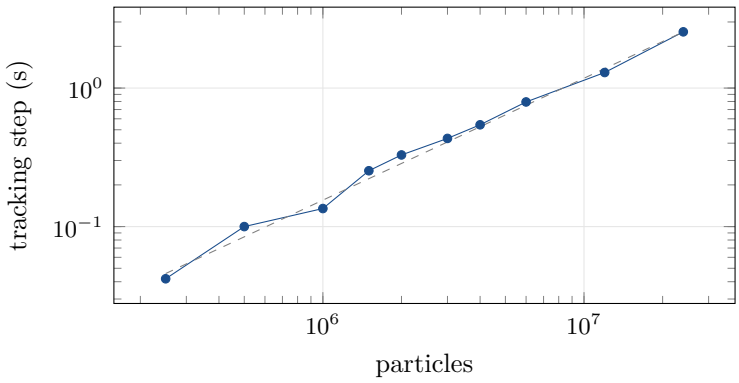


Figure 17.2: Particle count: steady tracking step against the number of particles, zoom-10 HEALPix mesh (8.9 million wet surface nodes), one 128-thread process, fused three-dimensional midpoint kernel. Dashed: the fitted power law with slope 0.88.

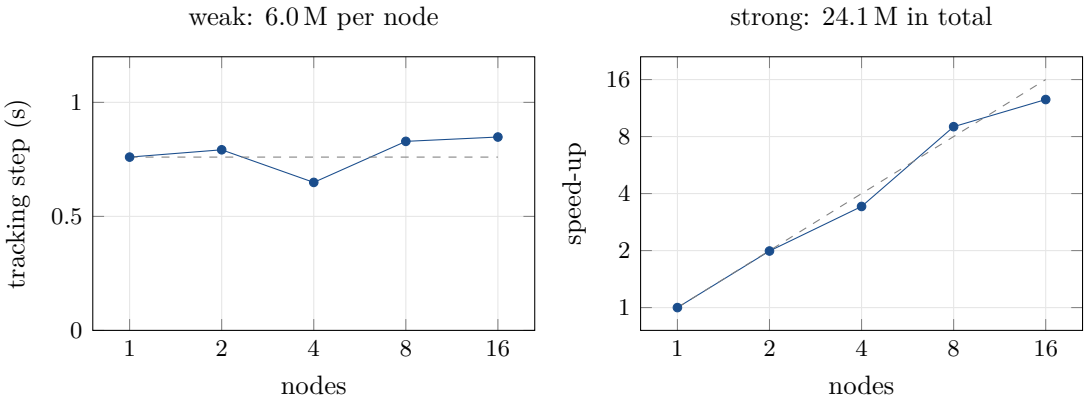


Figure 17.3: Scaling over nodes on the zoom-11 HEALPix mesh, one 128-thread process per node, fused three-dimensional midpoint kernel. Left: weak scaling (dashed, the one-node step). Right: strong scaling of 24.1 million particles (dashed, ideal). The strong curve is not smooth because the cost of a process depends on which particles its contiguous share holds.

4×10^8 particles per process the snapshot read, not the particles, sets the peak (122–146 GiB on this mesh). A small probe therefore measures the read, not the cost per particle; a new configuration is sized from two releases large enough for the particles to dominate.

17.8 Input and output

The read term dominates runs that write. For a full model year of six million three-dimensional particles on a zoom-9 mesh with daily snapshots and six-hour steps, one 128-thread process took 2 hours 12 minutes: of the mean step of 5.33 s, reading took 2.94 s, tracking 0.62 s and writing 1.78 s (7.1 s per record, every fourth step). Input and output make up 88 % of the step. On the zoom-11 mesh one daily snapshot costs about 146 s per process — as much as some 190 tracking steps of six million particles.

Levers on the read term. Confining a study to part of the ocean does not by itself reduce the read on a global mesh; extracting the region into a smaller mesh and forcing once, before the run, reduces the input volume by about two orders of magnitude and improves the thread scaling, because the smaller field fits in cache. The read-ahead (Section 10.4) amortizes the per-read overhead; it is forced to one snapshot when snapshots are shared between processes or when the forcing is cycled. Several processes on one node can share one copy of each processed

snapshot (`numerics.shared_fields_dir`): one process, elected by a file lock, publishes each snapshot with an atomic rename and the others map it read-only. Every measurement placed the cache on the parallel file system: shared within one node it was nearly free and removed the per-process replication (16 GB to 10 GB per process in one layout); shared by several nodes it slowed every step, because remote nodes fault pages in while stepping. Node-local memory has not been measured. The outputs are bit-for-bit identical with and without the shared cache, and any failure of the shared layer falls back to local reads.

Output. The volume grows with the number of particles and records: at most about 66 B per particle and record uncompressed (positions, status, velocity diagnostics and two sampled fields, the velocity diagnostics some 40 %), and about 37 B on disk in the one measured compressed store (six million particles, one year); `output_freq` is the lever, and sampled variables add their own columns. A pulsed release stores the whole pre-allocated array at every record, released or not.

Source. Read window and shared snapshots: `_VelCache` in `tracking.py`, `SharedFieldCache` in `core/shared_fields.py`; region extraction: `utils/extract_fesom_region.py` and `utils/extract_healpix_region.py`; sharding: `utils/sort_seeds.py`, `utils/split_seeds.py`, `utils/concat_trajectories.py`.

17.9 Sharding a release

A release is sharded in three steps: the seeds are sorted spatially (by HEALPix cell, then depth), so that consecutive particles share cached velocity data — worth 17 % on the fused three-dimensional step at a million particles; the sorted file is split into contiguous shares, one per process; and the per-process outputs are merged in shard order afterwards. A deterministic run with a single release, sharded and merged, is data-identical to the unsharded run, stochastic features included: the splitting tool writes each shard's position in the full release into the shard file, and the engine keys every random draw by the particle's index in the full release (Section 12.6). All shards therefore use the same seeds.

17.10 Measuring the cost

Three measurements cover most questions. The verbose per-step phase split of Section 17.1 tells which term dominates a run; the steady step — the median tracking phase over read-free steps after the first, of the slowest process — is the number to compare between runs and configurations, never a mean that mixes steps that read snapshots with steps that do not. The portable single-node benchmark (`benchmarks/single_node/run_bench.py`) builds a synthetic HEALPix store and reports the machine-independent *thread-microseconds per particle*, its steady step (the mean of its last five steps) times the thread count divided by the particles: on the compute node above 15.7–19.9 thread-µs from 16 to 128 threads, on a laptop's five performance cores 5.5. Comparisons are valid only between rows of the same benchmark.

Source. Phase timing: the verbose branch of `run_single_realization` in `tracking.py`; benchmark: `benchmarks/single_node/run_bench.py` and its `README.md`; the diagnosis path for a run slower than these numbers is the `kelp-diagnose` skill (Chapter 18).

Part VII

Working with KELP

Chapter 18

Agent Skills: the Operating Layer

The preceding chapters describe what the engine computes. This chapter describes the layer through which the engine is most often *operated*: a set of *agent skills*, plain Markdown instruction files that an AI coding agent loads when a request matches them, and that turn a described task — "release particles near a point, backward, in three dimensions", "show where the decades differ", "why is this run slow" — into the concrete artefacts the engine consumes: seeds, a configuration, job scripts, an analysis script, a figure script. The skills contain no numerics of their own. They encode the order of operations and the checks that past campaigns showed to be necessary, and they point to the documentation of Chapters 2, 14, 15 and 17 for everything else.

18.1 What a skill is

A skill is a directory under `.claude/skills/` whose entry point is a file `SKILL.md`. The file opens with a two-field header: a *name* and a *description*. The description states only *when* the skill applies — the requests, symptoms and boundary cases that should trigger it — and deliberately does not summarise the procedure, because an agent that finds the procedure in the description tends to act on the summary and skip the body. The body is the procedure: an ordered workflow, the decisions that need judgement with the condition that selects each option, and the checks that must hold. Longer material — templates, complete code recipes, measured tables, the history behind a rule — lives in files under `references/` next to the skill, one level deep, and is read only when the step that needs it names it. Two checks are shipped as runnable programs under `scripts/` (Section 18.4). An agent therefore reads a short entry file for every task and the detail only for the task at hand.

Source. The seven skills are `.claude/skills/kelp-run/`, `kelp-config/`, `kelp-experiment/`, `kelp-analysis/`, `kelp-visualize/`, `kelp-diagnose/` and `kelp-bench/`, each with a `SKILL.md`; reference files in their `references/` subdirectories; the two check scripts in `kelp-run/scripts/`. The figure helpers the visualisation skill relies on are in `gallery/figure_helpers.py`.

18.2 The seven skills

Table 18.1 lists the skills with the request that triggers each and what it hands back. They divide the work along the life of a study: setting up one run, writing its configuration, organising a campaign of many jobs, reading the output, drawing it, diagnosing performance, and benchmarking a machine.

The skills cross-reference one another rather than repeating each other (Figure 18.1). `kelp-run` is the entry point for a single run and defers the YAML mechanics to `kelp-config`; `kelp-experiment` sits above both and owns the campaign order of operations; `kelp-analysis` owns the reading of trajectory output, which `kelp-visualize` uses for its caches; `kelp-diagnose`

Table 18.1: The KELP agent skills. "Hands back" names the artefacts a skill produces; every skill that prepares a computation stops before launching it (Section 18.3).

Skill	Use it when	Hands back
`kelp-run`	a tracking experiment is described (release location, direction, 2-D or 3-D, diffusion, beaching, sampling, time window), or a dataset is used for the first time	seed generator, configuration, verification commands and the run command; for a new dataset the vertical-velocity sign and record-0 checks
`kelp-config`	a YAML configuration is written, edited, reviewed or debugged, or the engine opened the wrong file, ignored a key or failed at startup	the configuration, a validation script for generated or campaign configurations, a decision summary
`kelp-experiment`	an experiment is big (millions of particles, several processes or nodes), long (beyond one job's wall clock), on a large mesh that may need cropping, or a speed and memory showcase	a campaign folder with its rules file, report skeleton, ledger, config generator and config check, job scripts, the resumable chain driver
`kelp-analysis`	anything is read or computed from trajectory output (maps, cohort fractions, section crossings, run comparisons, reversibility, time-step ladders, costs from logs)	an analysis script with the reading pattern, masks and denominators, and a plan whose numbers are placeholders until the script prints them
`kelp-visualize`	a figure, animation or video is made from trajectory output or its caches, or colours, palettes, labels or render speed need changing	a render script (extract, cache, render), its commands, a caption file for each figure
`kelp-diagnose`	a run is slower than documented, silent, scales oddly with mesh size, costs milliseconds per particle, or a proposed speed fix needs vetting	a diagnosis along a decision tree and the measurement legs that decide it
`kelp-bench`	a machine is benchmarked or compared with the reference rows	one table row from the self-contained single-node benchmark

is entered from the phase timings that a verbose run prints and compares them with the measured rows of `docs/performance.md`, and `kelp-bench` produces one machine's row of the portable benchmark (`benchmarks/single_node/`).

18.3 How the skill text is written

The skills are read by agents of different strength, and the way a rule is phrased decides whether it is followed. The text therefore follows a small set of conventions.

Defaults and checks. Each skill that produces a product opens with a short paragraph that separates two kinds of guidance. *Defaults* are the choices that worked in past work — a palette, a background, a layout, which tables to write, how an event is counted, which uncertainty to quote. They are written as "by default ...; if the owner asks for something else, do that", because a default stated as a mandate makes a careful agent refuse what the owner explicitly asked for. *Checks* protect correctness whatever the style: the colour map's ends agree with the label (an assertion in the script), dead particles are masked by status, lead times are measured from each particle's own release, every number carries its denominator, cohorts the run does not cover are dropped or censored, and a claim about a whole run is guarded by an assertion that its last record is the run's end. Each check states its scope in the same sentence, so that relaxing a default cannot silently relax the check. A handback ends with a section that lists every default not used (with the owner's choice) and every check skipped (with the condition that did not hold).

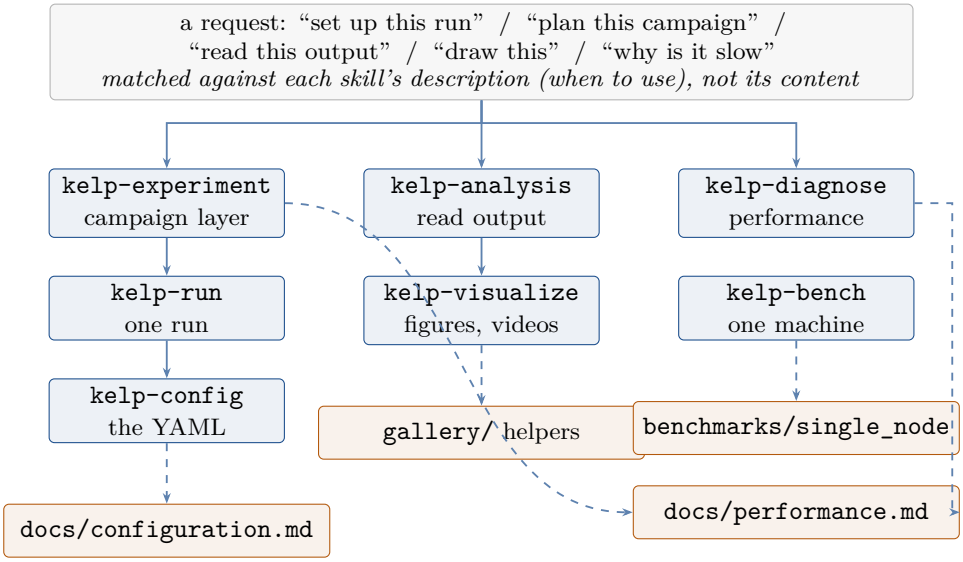


Figure 18.1: How requests reach the skills. A request is matched against the skills' descriptions; the selected `SKILL.md` then points to the neighbouring skill it builds on (solid) and to the documentation it defers to (dashed). Reference files under each skill's `references/` are opened only from the step that names them.

The launch gate. The skills that prepare a run or a batch job (`kelp-run`, `kelp-config`, `kelp-experiment`) stop before starting it; the benchmark skill asks before running on a cluster node. The prepared command is shown, and the agent asks for an explicit confirmation of that command; a request that merely describes the run ("run it on 128 cores") is not a confirmation. The last line of their handback states that nothing was run.

Checks keyed to their condition. A pre-launch check applies where its stated condition holds: the vertical-velocity sign check for a three-dimensional run on a velocity not used before, the record-0 check for a dataset not used before, the grid-hash cell check for a release in a refined region of a non-uniform mesh. A check whose condition does not hold is skipped, with one line saying why; the launch gate holds for every run.

Runnable code, no example numbers as answers. Code blocks in the skills are complete enough to run and were run before they were written down. Measured constants — memory per particle and throughput — live in `docs/performance.md` (a few per-event costs and the output sizing rule in the skills' reference files, labelled as single measurements); a skill states the recipe that derives the number for the task at hand and labels any quoted value as the measurement it came from. A number from a worked example is never a substitute for the reader's own measurement.

Short entry files, reference files on demand. Each `SKILL.md` holds the routing, the ordered workflow, the decision predicates and the checks; long templates and recipes live in `references/`. Two rules keep the split safe: the call line of every helper stays in the entry file (a reader that never opens the reference still calls the helper correctly), and a reference that depends on another links to it.

18.4 The check scripts

Two pre-launch checks recur for every new dataset and are shipped as scripts, run from the clone with the absolute path of the configuration.

Vertical-velocity sign. `w_sign.py` loads the mesh and the dataset through the configured reader with `mode.w_positive` forced to `up`, so that the negated internal velocity is the file's own value (Section 6.1); it averages the file's vertical velocity from 2° S to 2° N and from 150° W to 100° W, at the level nearest 50 m, over records 1–30, and prints the mean with the sign it implies. Equatorial upwelling makes the sign unambiguous there: a positive mean means the file stores upward velocity. A store that does not cover this box stops with a request for another region of known up- or downwelling. The configuration keeps `w_positive: auto` until the printed value is written in, and the run script refuses `auto` for such a run.

Record 0. `record0.py` prints the fraction of exactly zero surface elevation at the first, middle and last record. A model archive whose first record is the initial state of a cold start shows a fraction near one there; the run then starts after the spin-up, which a periodic forcing signal needs hours of records to develop — skipping one record is not a spin-up.

> **Source.** `.claude/skills/kelp-run/scripts/w_sign.py` and `record0.py`; both use `load_config`, `get_reader` and the reader's `load_mesh`, `open_dataset` and `read_velocities`; `w_sign.py` also uses `unrotate_lonlat` for a rotated mesh.

18.5 Skills and the documentation

The skills are procedure; the documentation is reference. Option semantics, defaults and validation rules are authoritative in `docs/configuration.md`; the pipeline in `docs/architecture.md`; costs and sizing in `docs/performance.md`; the restart mechanics in `docs/io_and_time.md`. A skill cites these rather than restating them, so that a change in the engine is documented once. Where a skill and the documentation disagree, the documentation and — behind it — the source take precedence, as throughout this manual.

18.6 Maintaining a skill

A skill changes when a campaign shows that a step was missing, a rule was wrong, or a phrase led a reader astray. Four practices keep the text reliable. Every code block in an edited skill is run before it is committed. A changed rule is exercised by having a separate agent carry out a representative task with the edited skill, and the artefact it produces — not its summary — is reviewed, including a task in which the owner departs from a default on purpose, so that a new rule does not override the owner. The review of an edit lists the errors the edit itself introduced, which is where most new defects come from. Finally, the skills ship with the public engine: they carry engineering practice and generic rules, and no study results.

Appendix A

Configuration Key Reference

This appendix lists the configuration keys consumed by the engine, grouped by their YAML section. It is a reference companion to the chapters: each key is described in context in the relevant chapter, and here we collect them with their types and defaults. The keys and defaults follow the engine's configuration module.

Load rules. A configuration must contain all five top-level sections (`mode`, `numerics`, `mesh`, `data`, `time`); a section that is present but empty must be written as `{}`. Keys that are misspelled or of the wrong case are silently ignored and the default is used — there is no typographic protection — so a key that appears to have no effect should be checked against this table. Relative paths of the mesh, data, seed and output files are resolved against the directory of the configuration file; `numerics.shared_fields_dir` is taken as written. Many values are validated when the file is loaded — the integrator, the interpolation methods `uv_interp`, `w_interp` and `sample_interp`, the diffusion, beaching (when enabled), periodic, adaptive-scheme, grid-hash, rotation and HEALPix keys, `w_positive`, `uv_vertical`, `input_rotation` and the stamp keys of `data` — and an invalid value there raises an error immediately; others, such as `max_levels`, `time_averaging` and the `time` values, are checked when the run starts or when first used. Unknown keys inside the diffusion and beaching blocks are ignored like any other.

`mode` — tracking mode and variable names

Key	Type	Default	Description
`Fdrift2D`	bool	`true`	Kept for older configurations; not read by the engine.
`Fdrift3D`	bool	`false`	Three-dimensional tracking with vertical velocity; alone decides between 2-D and 3-D.
`time_dir`	float	`1.0`	Integration direction: `1.0` forward, `-1.0` backward; any other value is refused.
`Vpuls`	bool	`false`	FESOM-C 2-D only: add the pulsation velocity to the mean flow.
`var_u, var_v`	str	auto	Names of the horizontal velocity variables; set to `unod`/`vnod` for node-layout FESOM2 velocities (requires `uv_interp: barycentric`).
`var_w`	str	auto	Name of the vertical velocity (3-D). FESOM2: if unresolved, the run proceeds in 2-D silently; FESOM-C refuses `w_cart`.

Key	Type	Default	Description
`w_positive`	str	`auto`	Sign convention of the vertical velocity in the file: `auto`, `up` or `down` (Section 6.1).
`var_eta`	str	auto	Name of the sea-surface height.
`uv_vertical`	str	`layer`	Horizontal velocity in depth: `layer` (constant within each layer) or `linear` (linear between the layer centres; a stencil member whose bed lies above the particle is a wall; Section 6.4). `linear` needs 3-D tracking (refused at load otherwise), a z-level or sigma mesh, `euler`, `rk2` or `rk4`, and `uv_interp barycentric` for node layout or `leastsq`, `edge_leastsq` or `barycentric` for element layout; FESOM2 element data need the element bottoms and no ice-shelf cavities (Section 3.5). Refused for `fesom14`, whose velocities on the levels are always linear in depth. Recorded as the output attribute `uv_vertical = "linear"`; a resume must keep it (Section 15.3).
`sigma_layer`	int	surface	2-D layered input: which vertical layer to track (`fesom14`: a 0-based kept level).
`sigma_mean`	bool	`false`	FESOM-C and `fesom14` 2-D: thickness-weighted vertical mean instead of one layer.
`max_levels`	int	all	Keep the first N vertical interfaces from the surface; must be ≥ 2 in 3-D.
`sample_vars`	list	`[]`	Node-based scalar fields to sample along trajectories (e.g. `temp`, `salt`).
`track_failure_reason`	bool	`false`	Record why each particle died (Section 13.2).

`numerics` — step, integrator, interpolation

Key	Type	Default	Description
`dt`	float (s)	auto	Integration step. Defaults to the snapshot interval. In the discrete mode it must equal the record spacing, and irregular record times (calendar months) are refused — use `temporal_interp`, where it may be smaller.
`file_dt`	float (s)	auto	Time between stored snapshots; auto-detected from the time coordinate. When the axis cannot be decoded it defines $t_i = i\,\Delta t$ (Section 14.3).
`integrator`	str	`euler`	`euler` / `rk2` / `rk4` / `rk45` (Chapter 9).
`rk45_tol_m`	float (m)	`1.0`	Adaptive scheme: error tolerance per substep, > 0 (Section 9.3).
`rk45_h_min`	float (s)	$10^{-4}\Delta t$	Adaptive scheme: smallest substep, > 0.
`output_freq`	int	`1`	Record positions every N steps, counted from the start of the run.
`temporal_interp`	bool	`false`	Reconstruct velocity in time between snapshots; permits sub-stepping (Chapter 10).

Key	Type	Default	Description
`temporal_reconstruction`	str	`linear`	`linear` or `pchip` (monotone cubic); active only with `temporal_interp`.
`velocity_float32`	bool	`false`	Hold velocity in single precision; halves snapshot memory.
`prefetch_size`	int	`1`	Snapshots read per bulk read-ahead; forced to 1 under cyclic forcing and with the shared cache.
`shared_fields_dir`	str	`null`	Directory of the node-shared snapshot cache (Section 14.7).
`shared_fields_generation`	str	`null`	Namespace generation of the cache; else `KELP_SHARED_FIELDS_GENERATION`, else the batch job.
`checkpoint_freq`	int	`0`	Write a restart checkpoint every N steps (`0` = off); needs an output path.
`output_flush_every`	int	`0`	NetCDF only: flush every N records so the file is readable mid-run.
`uv_interp`	str	`leastsq`	`leastsq` (element centres), `edge_leastsq` (containing element and edge neighbours) or `barycentric` (vertex blend; required for node velocities) (Chapter 5).
`edge_leastsq_bc`	str	`freeslip`	Wall ghost of `edge_leastsq`: `freeslip` or `noslip`.
`elem_search`	str	`local`	Containing element: `local` (search each step) or `walk` (track across edges; Section 4.6).
`w_interp`	str	`leastsq`	3-D vertical-velocity horizontal combine: `nearest`/`leastsq`/`barycentric`/`idw`.
`sample_interp`	str	`leastsq`	Horizontal method for sampled scalars (same options).
`Dmin`	float (m)	`0.5`	Minimum water-column thickness (sigma meshes only; Section 6.8).
`z0b_min`	float (m)	`0.1`	Bottom clamp offset (sigma meshes only).
`Dch`	float	`1.0`	Reserved; currently unused.
`periodic_lon, periodic_lat`	[lo, hi]	`null`	Periodic windows for channel domains (Section 13.4).
`diffusion`	map	off	Stochastic-diffusion sub-configuration (below).
`beaching`	map	off	Beaching sub-configuration (below).

`numerics.diffusion`

Key	Type	Default	Description
`enabled`	bool	`false`	Master switch; when off, byte-identical to no diffusion.
`scheme`	str	`uniform`	`uniform` / `em` (Euler–Maruyama) / `m1` (Milstein). Only `uniform` for the horizontal kick in 3-D.
`kh_zonal, kh_meridional`	float	`0.0`	Constant diffusivities (`uniform` only), m^2/s.
`var_kh_zonal, var_kh_meridional`	str	auto	Dataset diffusivity variable names (`em`/`m1`).

Key	Type	Default	Description
`dres_m`	float (m)	auto	Central-difference offset for the diffusivity gradient.
`seed`	int	`0`	Counter-based RNG seed (Section 12.6); restart-safe.
`composition`	str	`split`	How the kick composes with the step: `split`/`combined`/`split_rk`.
`kh_eval_position`	str	`end`	`split_rk` only: sample diffusivity at `start` or `end` of step.
`az_enabled`	bool	`false`	Vertical random walk (Section 12.5); needs `enabled` and the `uniform` scheme.
`az_const`	float	`0.0`	Constant vertical diffusivity, m^2/s, ≥ 0.
`var_az`	str	`null`	Vertical diffusivity variable on the interfaces (FESOM2 z-level).

`numerics.beaching`

Key	Type	Default	Description
`enabled`	bool	`false`	Master switch (Section 13.5).
`criteria`	list	`[depth]`	Any of `depth`, `coastal_element`, `on_failure`, `stochastic`; combined inclusively.
`depth_min`	float (m)	`0.3`	Strand when the column height falls below this.
`use_eta`	bool	`false`	Include sea level in the column height (needs an ssh field).
`dry_mask_var`	str	none	Node variable marking dry cells (> 0.5); strands (with the `depth` criterion) and blocks resuspension.
`zone_depth`	float (m)	auto	Near-shore zone for stochastic stranding; defaults to twice `depth_min`.
`t_beach`	float (s)	`86400`	Stochastic stranding timescale: $P = 1 - \exp(-\lvert\Delta t\rvert / t_{\text{beach}})$ per step.
`resuspension`	str	`never`	One of `never`, `auto`, `stochastic`, or `auto_or_stochastic`.
`rewet_hysteresis`	float (m)	`0.1`	Margin above `depth_min` to count as rewetted.
`min_wet_steps`	int	`1`	Consecutive wet steps before automatic release.
`t_resuspend`	float (s)	none	Stochastic-release timescale (required for stochastic resuspension).
`seed`	int	none	Beaching RNG seed; defaults to the diffusion seed.

`mesh` — mesh source

Key	Type	Default	Description
`type`	str	`fesomc`	`fesom2` / `fesom14` / `fesomc` / `nemo` / `healpix`; selects the reader, conventions, and vertical coordinate. `fesom14` needs `uv_interp: barycentric` (Section 3.7).
`meshpath`	str	required	Path to the mesh file or directory (Chapter 3). `fesom14`: an ASCII mesh directory (it holds `nod3d.out`), or level-wise data with cell polygons — a file, or a directory whose first `*.nc` file holding `mode.var_u` is used.
`input_rotation`	[3 floats]	`null`	`fesom14` only (refused at load for every other type): the Euler angles in degrees, from the run's namelist, of the rotated frame an ASCII mesh is stored in; rotated back to geographic on load. Refused for a mesh from cell polygons; independent of `rotation`.
`r_earth`	float (m)	family	Earth radius for mesh-derived areas; defaults per model.
`rotation`	varies	`null`	FESOM2, FESOM 1.4 and HEALPix: `null` (geographic), `fesom2`, or an explicit list of Euler angles (Chapter 8).
`exact_nearest`	bool	`true`	Prove every nearest-node anchor (Section 4.3).
`grid_hash_cell_rule`	str	`occupancy`	Choose the hash cell by measurement (`occupancy`), or keep the reader's (`legacy`) (Section 4.2).
`grid_hash_cell_size`	float (deg)	`null`	Explicit hash cell; overrides the rule.
`healpix_zoom`	int	`null`	HEALPix: zoom level when the store carries no grid attributes.
`healpix_geometry`	str	`dual`	HEALPix: `dual` (nodes at pixel centres) or `pixel` (elements are pixels; 2-D).
`crop_bbox`	[W, E, S, N]	`null`	HEALPix: crop to a box; $W > E$ crosses the dateline.
`levels`	path or list	`null`	HEALPix: interface depths, a level-table path or an inline list; needed in 3-D, and in 2-D it sets the node depths when the store has no bathymetry variable.

`data` — input and output

Key	Type	Default	Description
`ncinput`	str	required	Velocity input: a file, a Zarr store, or (FESOM2) a directory of per-variable files; `fesom14`: a file, a directory or a glob, the files grouped by the variables they hold (Section 14.1).
`ncinput_v`	str	`""`	Separate v-velocity file, if not in `ncinput` (NEMO: overrides grid-V; HEALPix: refused).
`ncinput_w`	str	`""`	Separate vertical-velocity file (3-D), if applicable (HEALPix: refused).

Key	Type	Default	Description
`ncinput_eta`	str	`""`	HEALPix: separate sea-surface-height store.
`inicloud`	str	required	Release positions: rows of `lon lat [depth] [release_sec]`, geographic (Section 13.6).
`particle_offset`	int	`null`	Global index of the file's first particle in a sharded release (Section 17.9); `null` = the file's `# kelp particle_offset N` header, else 0. Keys every random draw.
`output_path`	str	memory	Output; `.zarr` selects Zarr, otherwise NetCDF4 (Chapter 15).
`time_averaging`	str	guess	`mean` (shift timestamps to window midpoints) or `instant`; set explicitly (guess: `mean` for FESOM2 and HEALPix). `fesom14`: leave unset — the reader resolves the record times itself and a set value is refused; the output records `"resolved_by_reader"` (Section 14.5).
`time_stamp`	str	`null`	`fesom14` only (the three stamp keys are refused for every other type): where a stamp sits in its window, `start`, `centre` or `end`, or `instant` for snapshots. Given together with `time_window` or not at all; overrides the files' time bounds or output schedule with a warning.
`time_window`	str	`null`	`fesom14` only: the averaging window (with `instant`, the snapshot cadence): `calendar_month`, `calendar_year`, `calendar_day` or a duration `"<number> <s|min|h|d>"`, computed in the files' calendar.
`time_offset`	str	`null`	`fesom14` only: a shift `"<±number> <s|min|h|d>"` added to the stamps, bounds and epoch of every file first, for raw output of a job that started after midnight; allowed alone.

`time` — window and realizations

Key	Type	Default	Description
`start_time_idx`	int	0	First snapshot index to read.
`n_steps`	int	—	Number of integration steps (preferred way to set the length).
`end_time_idx`	int	—	Legacy alternative to `n_steps`.
`start_date`	str	—	Start as an ISO date in the axis calendar; between snapshots only with `temporal_interp` (Section 14.4).
`duration_days`	float	—	With `start_date`: run length in days, $\lfloor \text{days} \cdot 86400/\Delta t \rfloor$ steps.
`on_exhausted`	str	`stop`	At the end of the forcing: `stop` or `cycle` (wrap).
`num_runs`	int	1	Independent realizations (ensemble; Chapter 12).

Key	Type	Default	Description
`run_interval`	int	`0`	Snapshot-index offset between successive realizations.
`n_pulses`	int	`1`	Re-release the seed set this many times (Section 13.6).
`pulse_interval`	float (s)	`0.0`	Interval between pulses (required when `n_pulses` > 1).

Command line

The command `kelp run config.yaml` runs a configuration, `kelp info config.yaml` prints the mesh and the time range without tracking, and `kelp -version` prints the engine version. The options of `kelp run` override the corresponding keys:

Option	Effect
`-o, -output`	Output path; overrides `data.output_path`.
`-checkpoint-freq N`	Overrides `numerics.checkpoint_freq`.
`-temporal-interp`	Switches the interpolated mode on; without the option the configuration decides.
`-seed N`	Overrides `numerics.diffusion.seed`.
`-flush-every N`	Overrides `numerics.output_flush_every`.
`-v, -verbose`	Per-step timing with the phase split of Section 17.10.

Appendix B

Notation and Symbols

This appendix collects the principal symbols used in the manual. The sign and unit conventions they obey are stated in the front-matter section "Notation and conventions".

Symbol	Meaning
$\mathbf{x} = (\lambda, \phi, z)$	Particle position: longitude, latitude (degrees), depth (m, positive down).
$\mathbf{u} = (u, v, W)$	Velocity: horizontal components ($\mathrm{m\,s^{-1}}$) and vertical velocity W ($\mathrm{m\,s^{-1}}$, positive down).
$t,\ \Delta t$	Time and the (positive) integration step.
s_{fb}	Integration-direction factor, $+1$ forward or -1 backward (the key `time_dir`).
M_ϕ	Metres per degree of latitude, $111\,120\,\mathrm{m\,{}^\circ{}^{-1}}$.
z_0	Metres per degree of longitude, $\max(\cos\phi\, M_\phi, 1)$.
w_i	Barycentric / Wachspress interpolation weight at vertex i.
α	Interpolation fraction, in time (Equation (10.1)) or in depth (Equation (6.7)).
$N,\ E,\ K$	Number of mesh nodes, elements, and vertical interfaces ($K-1$ layers).
z_k	Depth of the k-th vertical interface.
σ_k	Sigma value of the k-th interface (FESOM-C); $\sigma = 1$ surface, $\sigma = 0$ floor.
H	Sigma-mesh water-column thickness, $\max(d + \eta, D_{\min})$.
$d,\ \eta$	Still-water depth and sea-surface height.
$D_{\min},\ z_{0b}$	Minimum column thickness and bottom clamp offset (sigma; keys `Dmin`, `z0b_min`).
K_h	Horizontal eddy diffusivity ($\mathrm{m^2\,s^{-1}}$); $\sigma^2 = 2K_h t$.
R	Earth radius of the mesh (`mesh.r_earth`), used in the local tangent-plane projection.
R_M	Metric radius of the displacement, $M_\phi \cdot 180/\pi$, used by the great-circle kicks (Section 12.2).
$\xi,\ \xi_1,\ \xi_2$	Standard normal variates of the stochastic kicks (Chapter 12).
A_z	Vertical eddy diffusivity ($\mathrm{m^2\,s^{-1}}$), Section 12.5.
c_k	Fraction of the step at which Runge–Kutta stage k samples the flow.
$p_k,\ M$	Data points of a depth-linear horizontal velocity (layer centres or levels; on a sigma mesh layer centres in s) and their number (Section 6.4).

Symbol	Meaning
$\ell_c,\ d_c,\ D_c$	Last usable data point, bed depth and reader-declared data prefix of column c (a node or an element), Equation (6.3).
s	Sigma fraction z/H_p of the particle's column, clipped to $[0, 1]$ (Equation (6.6)).
w_i^+	Barycentric weight counted as zero when negative, in the wall rule (Equation (6.5)).
$\alpha_0,\ \alpha_k$	Bracket fraction of a lazy blend and stage fraction of a stage blend (Chapter 10).
$A_k,\ C_k$	Doubled fan- and corner-triangle areas in the Wachspress weights (Equation (5.9)).

Appendix C

Source-Code Map

This appendix maps each subsystem of the engine to the source files that implement it and to the chapter that describes it, so that a reader starting from the code can find the corresponding text and vice versa. Paths are relative to the package source root, `src/kelp/`, unless they begin with the repository directory `utils/` or `.claude/`.

Subsystem	Principal source	Manual
Run orchestration	`tracking.py`	Chapter 2
Command line	`cli/main.py`	Chapter A
Mesh structure, contract & derived tables	`core/mesh.py`	Chapter 3
FESOM2 and FESOM-C meshes	`readers/fesom2.py`, `readers/fesomc.py`	Chapter 3
FESOM2 element bottoms	`readers/fesom2.py`, `utils/extract_fesom_region.py`	Section 3.5
FESOM 1.4 mesh and reader	`readers/fesom14/` (`reader.py`, `_ascii_mesh.py`, `_polygons.py`, `_layout.py`, `_timeaxis.py`)	Sections 3.7 and 14.1
HEALPix meshes and reader	`readers/healpix.py`	Chapters 3 and 14
Structured grids and NEMO	`readers/structured.py`, `readers/nemo.py`	Chapters 3 and 14
Nearest node: grid hash, cell rule, exact search	`core/mesh.py`, `utils/check_grid_hash_cell.py`	Chapter 4
Containment and element tracking	`core/mesh.py`, `numerics/integrators.py`	Sections 4.5 and 4.6
Horizontal velocity interpolation	`numerics/interpolate.py`	Chapter 5
Polygon weights (triangle / Wachspress)	`core/mesh.py`	Section 5.5
Vertical geometry & W (z-level)	`numerics/vertical.py`	Chapter 6
Vertical geometry & W (sigma)	`numerics/vertical_sigma.py`	Chapter 6
Horizontal velocity in depth (layer or linear)	`numerics/vertical_uv.py`, `numerics/interpolate.py`, `numerics/integrators.py`, `tracking.py`	Section 6.4
Vertical-velocity sign	`readers/__init__.py`	Section 6.1
Offline vertical-velocity reconstruction	`w_reconstruct.py`, `utils/reconstruct_w.py`	Section 6.1
Scalar sampling	`numerics/sampling.py`	Chapter 7
Coordinate frame / rotation	`core/rotation.py`, `readers/fesom2.py`, `readers/healpix.py`, `tracking.py`	Chapter 8
Time integration	`numerics/integrators.py`	Chapter 9
Fused kernels	`numerics/integrators.py`, `numerics/interpolate.py`	Chapter 11
Temporal reconstruction	`numerics/temporal_reconstruct.py`, `core/io.py`	Chapter 10

Subsystem	Principal source	Manual
Stochastic diffusion	`numerics/diffusion.py`	Chapter 12
Particle state & lifecycle	`core/particles.py, core/release.py`	Chapter 13
Seeding helpers	`core/particles_gen.py`	Chapter 13
Beaching	`numerics/beaching.py, tracking.py`	Section 13.5
Periodic windows	`tracking.py`	Section 13.4
Forcing readers & time axis	`readers/, tracking.py`	Chapter 14
Reader hook and stamp options	`readers/__init__.py`	Sections 14.1 and 14.5
Step payload & snapshot cache	`core/io.py, tracking.py`	Section 14.6
Node-shared snapshot cache	`core/shared_fields.py`	Section 14.7
Trajectory output & restart	`writers.py`	Chapter 15
Sharding and merging	`utils/sort_seeds.py`, `utils/split_seeds.py`, `utils/concat_trajectories.py`	Sections 15.4 and 17.9
Plotting and statistics	`visualization/ (plots, animation, presence statistics)`	— (module README)
Region extraction	`utils/extract_fesom_region.py`, `utils/extract_healpix_region.py`	Section 17.8
Configuration	`core/config.py`	Chapter A
Spherical metric / constants	`core/geometry.py`	Section 9.1
Agent skills	`.claude/skills/`	Chapter 18

The depth-linear rule and the FESOM 1.4 reader span several files; Table C.2 lists their principal symbols.

Table C.2: Principal symbols of the depth-linear horizontal velocity and of the FESOM 1.4 reader.

Symbol	File	Manual
`depth_points, DepthPoints, uv_bracket, uv_at, uv_wet`	`numerics/vertical_uv.py`	Section 6.4
`check_depth_linear_record, refuse_elem_cavities`	`numerics/vertical_uv.py`	Section 6.4
`_interp_barycentric_dl, _interp_barycentric_core_dl, _dl_node_wall`	`numerics/interpolate.py`	Sections 5.6 and 6.4
`_interp_barycentric_core_tinterp_dl, _interp_barycentric_core_pchip4_dl`	`numerics/interpolate.py`	Section 6.4
`_interp_leastsq_dl, _interp_leastsq_core_dl`	`numerics/interpolate.py`	Section 6.4
`_interp_edge_leastsq_dl, _interp_edge_leastsq_core_dl`	`numerics/interpolate.py`	Section 6.4
`_interp_bary_elem_core_dl, _dl_elem_wet_mean`	`numerics/interpolate.py`	Section 6.4
`_check_depth_linear_entry, _sigma_frac_for_uv`	`numerics/integrators.py`	Section 6.4
`_fused_euler_step_3d_dl, _fused_rk2_step_3d_bary_tinterp_dl, _fused_rk2_step_3d_bary_pchip_dl`	`numerics/integrators.py`	Sections 6.4 and 17.2
`_check_uv_vertical_option, _validate_uv_compat, _check_linear_element_mesh, _mark_depth_linear`	`tracking.py`	Section 6.4
`_validate_restart_uv_vertical`	`tracking.py`	Section 15.3

Symbol	File	Manual
`_elem_level_metadata`	`readers/fesom2.py`	Section 3.5
`Fesom14Reader (load_mesh, open_dataset, prepare, read_velocities)`	`readers/fesom14/reader.py`	Sections 3.7 and 14.1
`read_ascii_mesh`	`readers/fesom14/_ascii_mesh.py`	Section 3.7
`recover_triangles`	`readers/fesom14/_polygons.py`	Section 3.7
`is_data, detect_form, gather_nodes3d, wet_mask_levelwise, check_record`	`readers/fesom14/_layout.py`	Sections 3.7 and 14.1
`resolve_axis, TimeOptions`	`readers/fesom14/_timeaxis.py`	Section 14.5
`_prepare_dataset, _time_option_kwargs, _describe_record_times`	`readers/__init__.py`	Sections 14.1 and 14.5

The package installs with `pip install -e .` from the repository root and provides the `kelp` command. Every output records the engine version — the package version and the commit — in its attributes, so a result can be traced to the source that produced it. The documentation tree carries a machine-readable map from each subsystem to the source symbols and the revision at which its documentation was last verified (`docs/doc_map.yaml`), and a script that reports which documents have fallen behind their code (`docs/check_doc_sync.py`).

Appendix D

Glossary

FESOM family
The unstructured-mesh ocean models the engine principally targets: FESOM2, FESOM 1.4 and FESOM-C. Used here as a collective term; statements specific to one model name it.

FESOM2
A global ocean model on a purely triangular mesh with a fixed depth-level (z-level) vertical coordinate. The running example uses a FESOM2 mesh.

FESOM-C
A coastal ocean model on a mixed mesh — elements may be triangles or quadrilaterals — with a terrain-following (sigma) vertical coordinate.

FESOM 1.4
The finite-element predecessor of FESOM2, on a triangular mesh with z-levels; a P1 model whose u, v, W, temperature and salinity live on the levels at the nodes. Its mesh is read from the model's ASCII files or recovered from the cell polygons of its output.

Node
A vertex of the mesh. Node-layout fields (e.g. `unod`/`vnod`, scalars, sea-surface height) are defined at nodes.

Element
A mesh face: a triangle (FESOM2) or a triangle or quadrilateral (FESOM-C, HEALPix, NEMO). Element-layout velocities are defined at element centroids.

Interface, layer
The vertical level boundaries (interfaces) and the volumes between them (layers); K interfaces bound $K-1$ layers. The vertical velocity lives at interfaces, the three-dimensional horizontal velocity and scalars at mid-layers (in FESOM 1.4 every field on the levels).

Depth-linear horizontal velocity
The rule `mode.uv_vertical: linear`: the horizontal velocity is read linearly in depth between its data points, each stencil member in its own column, instead of as a constant per layer. FESOM 1.4 velocities are always read so.

Wall at depth
A stencil member whose bed lies above the particle; under the depth-linear rule it is left out of the interpolation, as the coast is.

Partial bottom layer
A layer whose lower level lies below the bed of its column (a FESOM2 partial bottom cell); the depth-linear rule never reads it.

One-ring
The set of elements that share a given node; the stencil used by the localization search and several interpolators.

Barycentric coordinates
The area-ratio weights of a point within a triangle, used to blend the three vertex values; the triangular interpolation stencil.

Wachspress coordinates
The generalization of barycentric coordinates to a convex polygon, used to interpolate over a quadrilateral element; computed in a division-free product form that is exact on edges, which is why quadrilaterals must be convex.

z-level coordinate
A vertical coordinate with fixed interface depths, the same everywhere (FESOM2).

Sigma coordinate
A terrain-following vertical coordinate whose interface depths are fractions of the local column thickness $H = \max(d + \eta, D_{\min})$ (FESOM-C).

Node-layout / element-layout
Whether a velocity field is stored at nodes or at element centroids; node layout admits only the barycentric scheme, element layout all three.

Fused / unfused kernel
The two interchangeable implementations of a step: a single compiled loop over the particles, or separate parallel passes; they agree bit-for-bit where both exist.

PCHIP
Monotone piecewise-cubic Hermite reconstruction of the velocity in time, using four snapshots; raises the realized temporal order above the linear reconstruction's cap.

Counter-based RNG
A stateless random-number generator in which each draw is a hash of an integer key, giving results independent of thread count and reproducible across restarts.

Beaching
The optional stranding of particles in shallow or coastal water, with an optional resuspension; the only reversible non-alive status.

`time_dir`
The integration-direction factor ($+1$ forward, -1 backward), folded into the interpolated velocity so the step size stays positive.

Drifter / particle
A passive Lagrangian tracer advected by the flow; "drifter" is used in the schematics for the point being interpolated to.

HEALPix
A hierarchical equal-area grid of the sphere on which some ocean products are stored; the engine builds a mesh from its pixel centres (dual mesh) or from its pixels (pixel mesh).

Grid hash
The uniform lattice of cells over which mesh nodes are bucketed for the nearest-node search; its cell is chosen by measurement for each run.

Exact anchor
A nearest node proven to be the nearest by the cap certificate of the search, the default since September 2026.

Edge walk
Tracking the containing element from step to step by crossing element edges, instead of searching around the nearest node every time.

Edge least squares
The velocity interpolation that fits a plane to the containing element and its edge neighbours, with mirrored ghost values at the coast.

Adjoint process
The backward-in-time counterpart of an advection–diffusion process: the advection reversed, the diffusive drift and noise unchanged.

Resumable chain
A long run split into consecutive batch jobs, each resuming from the previous one's restart; the concatenated result equals one uninterrupted run.

Shard
One contiguous block of a large release, run as a separate process and merged back afterwards.

Agent skill
A packaged set of instructions, reference files and check scripts that a coding agent loads to carry out one kind of task on the engine (Chapter 18).

Bibliography

J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. Wiley, 3rd edition, 2016.

S. Danilov, D. Sidorenko, Q. Wang, and T. Jung. The finite-volume sea ice–ocean model (FESOM2). *Geoscientific Model Development*, 10(2):765–789, 2017. doi: 10.5194/gmd-10-765-2017.

J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980. doi: 10.1016/0771-050X(80)90013-3.

Michael S. Floater. Mean value coordinates. *Computer Aided Geometric Design*, 20(1):19–27, 2003. doi: 10.1016/S0167-8396(03)00002-5.

F. N. Fritsch and J. Butland. A method for constructing local monotone piecewise cubic interpolants. *SIAM Journal on Scientific and Statistical Computing*, 5(2):300–304, 1984. doi: 10.1137/0905021.

F. N. Fritsch and R. E. Carlson. Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2):238–246, 1980. doi: 10.1137/0717021.

John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'11)*, 2011. doi: 10.1145/2063384.2063405.

Erik van Sebille, Stephen M. Griffies, Ryan Abernathey, et al. Lagrangian ocean analysis: Fundamentals and practices. *Ocean Modelling*, 121:49–75, 2018. doi: 10.1016/j.ocemod.2017.11.008.

A. W. Visser. Using random walk models to simulate the vertical distribution of particles in a turbulent water column. *Marine Ecology Progress Series*, 158:275–281, 1997. doi: 10.3354/meps158275.

Q. Wang, S. Danilov, D. Sidorenko, R. Timmermann, C. Wekerle, X. Wang, T. Jung, and J. Schröter. The Finite Element Sea Ice-Ocean Model (FESOM) v.1.4: formulation of an ocean general circulation model. *Geoscientific Model Development*, 7(2):663–693, 2014. doi: 10.5194/gmd-7-663-2014.

Joe Warren. Barycentric coordinates for convex polytopes. *Advances in Computational Mathematics*, 6:97–108, 1996. doi: 10.1007/BF02127699.

Index

element, 10

grid hash, 18

interface, 10

layer, 10

`nod_in_elem2D`, 12
node, 10